Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan
CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 1 - Introduction to the Various Application Landscape

Introduction

This course aims to provide a comprehensive introduction to modern Web application architecture approaches, frontend and backend technologies, and suitable web application frameworks required for developing modern web apps.

Learning Outcomes
Understand the underlying architecture used for Web applications and identify the various components of the Web Application
Demonstrate the creation of API to accomplish various backend functionalities of an application like database interaction, handling user requests
Design and develop user-friendly and interactive web frontends.
Implement a functional end-to-end web application using client-side and server-side web technologies.

Modular Structure

Module 1: Application Development
Introduction to the Various Application Landscape

Module 2: Understanding the Web Basics
Structure of web applications, Client – Server Communication

Module 3: Web Protocols
HTTP, HTTPS, WebSockets

Module 4: Server Side: Implementing Web Services
REST, gRPC, GraphQL

Module 5: Securing Application
JWT, OAuth

Modular Structure

Module 6: Understanding Frontend Development

Implementing Single Page Applications using JavaScript Tech stack

Module 7: Testing

API Testing, Unit Testing

Module 8: Accessibility and Performance

Inclusive Design , Assistive Technologies, Tools and Metrics for Measuring Performance

Module 9: Latest Advancements

Progressive Web Apps

Web Assembly

*Building ML/AI Powered Applications

Development Environment

Tools/Technologies

- Visual Studio Code
- Browser(Chrome)
- Frontend: ReactJS, Node Ecosystem
- Backend: NodeJS, Express, Django
- Database: MongoDB/Postgresql
- Postman for Testing
- GitHub and GIT
- *Python (ML model development and inference) FastAPI / Flask (for ML API creation)

Module 1: Application Introduction

Activity 1

Open Makemytrip in a browser
Open the BITS Website in a browser
Observe the difference in the address bar!!

Repeat in a different browser
Install the app from one browser and observe

Website-What & Why?

A group of interlinked web pages having a single domain name

- Hosted on a web server
- Accessible over the web with an internet connection
- Easily accessible through browsers
- Can be developed and maintained by individuals/teams for personal or business usage

Use Cases for Static Websites

- portfolios
- personal blogs
- informational websites and
- small business websites with minimal content updates.

- They are also suitable for landing pages or temporary promotions where the content doesn't change frequently.

Web Application

A web application is an application program stored on a remote server and delivered over the internet.

Users can access a web application through a web browser, such as Google Chrome, Mozilla Firefox or Safari.

Common web applications include e-commerce shops, webmail, and social networking sites.

Classifications of Applications

Applications can be classified along multiple dimensions:

- By Functional Domain or Purpose (What the app does)
- By Platform Type (How apps are built and accessed)
- By Backend Architecture & Hosting Model

By Functional Domain or Purpose (What the app does)- The intelligent spectrum

Classifies apps based on their intended functionality or intelligence.

AI-Powered App: Uses artificial intelligence like LLMs, image generation, or chatbots

ML-Powered App: Integrates machine learning models for tasks like prediction or classification

Data Dashboard: Presents analytics or business intelligence in real-time or historical views

Domain-specific application types: E-Commerce, HealthTech, EdTech, CRM.

Productivity / Collaboration Tools – e.g., task managers, shared editors, note apps

Agentic Applications: The newest addition. These apps don't just answer questions; they execute multi-step tasks (e.g., an agent that researches, books, and pays for a business trip autonomously).

What is an ML-Powered Application?

ML-powered apps are applications that integrate machine learning models.
They enhance functionality through prediction, classification, or generation tasks.
Examples include chatbots, recommendation engines, OCR tools, and smart assistants.

Architecture Overview:
[Frontend (React)] → [API (FastAPI)] → [ML Model] → [JSON Response] → [Frontend]

Differences: (The below table generated by ChatGPT)

Aspect	Traditional Applications	ML-Powered Applications	AI-Powered Applications	Agentic Applications
Core Logic	Rule-based, deterministic logic written by developers	Uses trained ML models for prediction or classification	Uses advanced AI models (LLMs, vision, speech) for reasoning and generation	Uses AI agents that plan, decide, and act autonomously
Decision Making	Fully predefined	Data-driven, probabilistic	Context-aware, generative	Goal-driven and adaptive
Autonomy Level	None	Low	Medium	High
Learning Capability	No learning	Learns from data during training	Uses pre-trained models + runtime context	Learns from feedback, environment, and tools
Adaptability	Static unless code changes	Improves via retraining	Flexible via prompts and context	Continuously adapts strategy
User Interaction	Forms and workflows	Mostly indirect	Conversational, interactive	Conversational + proactive
Planning & Reasoning	None	None	Limited reasoning	Explicit planning and multi-step reasoning
Tool Usage	Fixed APIs	Fixed APIs	Limited tool calls	Dynamically selects and uses tools
State Management	Application state only	Application + model state	Application + conversational context	Long-term memory and task state
Determinism	Highly deterministic	Probabilistic	Non-deterministic	Non-deterministic and evolving
Typical Use Cases	CRUD systems, ERPs, CMS	Fraud detection, recommendations	Chatbots, copilots, semantic search	Autonomous assistants, AI workflows, AI operators
Examples	Inventory system	Product recommendation engine	AI shopping assistant	AI agent managing orders, support, and inventory
Risk Level	Low	Medium	Medium–High	High (requires strong guardrails)
Development Complexity	Low–moderate	Moderate	High	Very high
Governance Required	Standard validation	Model evaluation	Output validation & safety checks	Policies, monitoring, human-in-the-loop

By Platform Type (How apps are built and accessed)

Defines how the app is delivered to users and the technologies used.

Native apps are created for one specific platform or operating system.

Web apps are responsive versions of websites that can work on any mobile device or OS because they're delivered using a browser.

Hybrid apps are combinations of both native and web apps, but wrapped within a native app, giving it the ability to have its icon or be downloaded from an app store. A web app (HTML/CSS/JS) wrapped in a native shell runs inside a webview

Cross-Platform Apps are built with one codebase that compiles to native apps for multiple platforms

Progressive Web App (PWA): A web app enhanced with features like offline support, install ability, and push notifications using modern web APIs

Native Apps

Developed specifically for a particular mobile device

Installed directly onto the device itself

Needs to be downloaded via app stores such as

- Apple App Store
- Google Play store, etc.

Built for specific mobile operating system such as

- Apple iOS
- Android OS

An app made for Apple iOS will not work on Android OS or Windows OS

Need to target all major mobile operating systems

– require more money and more effort

Native Apps-Pros and Cons

Pros

- Can be Used offline - faster to open and access anytime
- Allow direct access to device hardware that is either more difficult or impossible with web apps
- Allow the user to use device-specific hand gestures
- Full access to all device features and APIs.
- Gets the approval of the app store they are intended for
- User can be assured of improved safety and security of the app

Native Apps-Pros and Cons

Cons

- More expensive to develop - separate app for each target platform
- The cost of app maintenance is higher - especially if this app supports more than one mobile platform
- Getting the app approved for the various app stores can prove to be a long and tedious process
- Needs to download and install the updates to the apps ontouser'ss mobile device

Native Apps

Native apps are built specifically for one platform using its native programming language and tools.

iOS:

- **Language**: Swift, Objective-C
- **Frameworks**: UIKit, SwiftUI
- **Tools**: Xcode
- **Example App**: Instagram (iOS)

Android:

- **Language**: Java, Kotlin
- **Frameworks**: Android SDK
- **Tools**: Android Studio
- **Example App**: WhatsApp (Android)

Web Apps

Internet-enabled applications

- Accessible via the device's Web browser

Don't need to be downloaded and installed onto a mobile device

Written as web pages in HTML and CSS with interactive parts in JQuery, JavaScript, etc.

Single web app can be used on most devices capable of surfing the web irrespective of the operating system

Web Apps- Pros and Cons

Pros

- Instantly accessible to users via a browser
- Easier to update or maintain
- Easily discoverable through search engines
- Development is considerably more time and cost-effective than development of a native app
- common programming languages and technologies
- It has a much larger developer base.

Cons

- Only have limited scope as far as accessing a mobile device's features is concerned such as device-specific hand gestures, sensors, etc.
- Many variations between web browsers and browser versions and phones
- Challenging to develop a stable web app that runs on all devices without any issues
- Not listed in 'App Stores'
- Unavailable when offline, even as a basic version

Web Apps

Web apps run in web browsers and are built using standard web technologies.

Languages: HTML, CSS, JavaScript

Frameworks: React, Angular, Vue.js

Tools: Web browsers, developer tools

Example App: Google Docs

Progressive Web Application

A **progressive web app** (PWA) is an app that's built using web platform technologies
Progressive web apps combine the best features of traditional websites and platform-specific apps

- Works offline or with limited connectivity
- Can be installed like a native app on desktops or mobile
- Uses web technologies like HTML, CSS, and JavaScript
- Employs service workers, manifest files, and caching

Progressive Web Application

PWAs offer a native app-like experience on the web, including offline capabilities and installation.

Languages: HTML, CSS, JavaScript

Frameworks: Workbox, Lighthouse (for auditing)

Tools: Service Workers, Web App Manifests

PWAs

How ML Can Be Embedded in PWAs

Client-Side ML (On-Device Inference)

Perform ML inference directly in the browser, using JavaScript-based ML libraries.

Edge-Based ML (via Service Workers)

Combine service workers with ML for background tasks.

Hybrid Apps

Hybrid apps combine elements of both native and web applications. They are built using web technologies but run inside a native container.

Languages: HTML, CSS, JavaScript

Frameworks: Ionic, Apache Cordova, PhoneGap

Tools: Web technologies wrapped in native code

Hybrid Apps

Part native apps, part web apps

Like native apps,

- available in an app store
- can take advantage of some device features available

Like web apps,

- Rely on HTML, CSS , JS for browser rendering

The heart of a hybrid mobile application is an application that is written with HTML, CSS, and JavaScript!

Run from within a native application and its own embedded browser, which is essentially invisible to the user

- iOS application would use the WKWebView to display the application
- Android app would use the WebView element to do the same function.

Hybrid Apps- Pros and Cons

Pros

- Don’t need a web browser like web apps
- Can access to a device’s internal APIs and device hardware
- Only one codebase is needed for hybrid apps

Cons

- Much slower than native apps
- With hybrid app development, dependent on a third-party platform to deploy the app’s wrapper
- Customization support is limited.

Cross-platform Application

Cross-platform app development refers to developing software that can run on multiple devices.

Multi-platform compatibility is a pervasively desirable trait.

Product to be available to as many consumers as possible.

Cross-platform Application

Cross-platform native apps use frameworks that allow development for multiple platforms from a single codebase, providing near-native performance.

Frameworks: React Native, Flutter, Xamarin

Tools: IDEs like Visual Studio, Android Studio

Example App:

- **React Native**: Facebook
- **Flutter**: Google Ads
- **Xamarin**: Microsoft Outlook

Native App vs Cross Platform

Native App Development	Cross Platform App Development
Native App Development is costly.	It is cost-effective.
Code cannot be reused	It supports code reusability. Same codebase is used across multiple platforms
Native apps are faster.	Cross Platform apps are slower than native apps

Differences (The below table generated by ChatGPT)

Feature / Aspect	Web App	PWA	Hybrid App	Cross-Platform App	Native App
Codebase	HTML, CSS, JS	HTML, CSS, JS + Web APIs	HTML, CSS, JS	Single (e.g., Dart, JS)	Platform-specific (Swift, Kotlin)
Runs On	Web browser	Web browser (with offline support)	WebView inside native shell	Compiled to native code	Compiled natively
Installation	Not installable	Installable via browser prompt	Via app stores	Via app stores	Via app stores
Access to Native APIs	Very limited	Limited (browser APIs only)	Moderate (via plugins)	High (via native bridges)	Full access
Performance	Low to Medium	Medium to Low	Medium	High / Near-native	Best
UI/UX	Browser-based UI	Web UI (can mimic native)	Web-based look and feel	Native look and feel	Native look and feel
Offline Support	No	Yes (via service workers)	Yes (via embedded browser cache)	Yes	Yes
Distribution	Hosted on web	Hosted on web or installed	App Stores	App Stores	App Stores
Push Notifications	No	Yes (limited browser support)	Yes (via native shell)	Yes	Yes
Update Mechanism	Instantly via server	Instantly via service worker	Through app store update	Through app store update	Through app store update
Use Cases	Informational sites, blogs	Lightweight interactive apps	MVPs, internal business apps	Scalable consumer apps	High-performance or native apps
Examples(*subject to change)	Wikipedia, MDN	MDN PWA , Starbucks PWA	Instagram (early versions), Evernote	Flutter (Google Ads), React Native (Facebook)	WhatsApp, Instagram, Uber

2. By Backend Architecture & Hosting Model

- Monolithic – All-in-one application deployed as a single unit
- Microservices – Decomposed services that scale and deploy independently
- Cloud-Native – Designed for elasticity, scalability, containerization (e.g., Kubernetes)
- Serverless – Event-driven functions managed by cloud providers (e.g., AWS Lambda, Azure Functions)
- SaaS (Software as a Service) – Software delivered over the internet, managed and hosted by a provider (e.g., Google Workspace, Salesforce)
- Edge-Native Applications: These run as close to the user as possible (on 5G towers or local gateways) to provide zero latency. Vital for autonomous vehicles and real-time medical monitoring.

Cloud Native Application

Cloud-native is an approach to developing, deploying, and running applications using modern methods and tools.

The Cloud Native Computing Foundation(CNCF) defines

> *Cloud-native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.*

Why Cloud Native

Monolithic Application

- A monolithic architecture refers to a traditional software design approach where an entire application is built as a single, self-contained unit.

Key characteristics of monolithic architecture include:

- ❑ Single Codebase
- ❑ Tight Coupling
- ❑ Single Deployment Unit
- ❑ Centralized Database
- ❑ Development and Scaling Challenges
- ❑ Longer Development Cycles
- ❑ Limited Fault Isolation

Cloud-Enabled Solutions

What happens when you move a monolithic app from on-premises to cloud?

A cloud-based application is an existing app shifted to a cloud ecosystem.

They are not able to utilize the full potential of the cloud

- Not scalable
- Less automation
- Longer Time to Market

Cloud-Native Applications are designed to run on the cloud computing architecture.

With cloud-native applications, you can take advantage of the automation and scalability that the cloud provides.

Cloud native vs. Cloud enabled

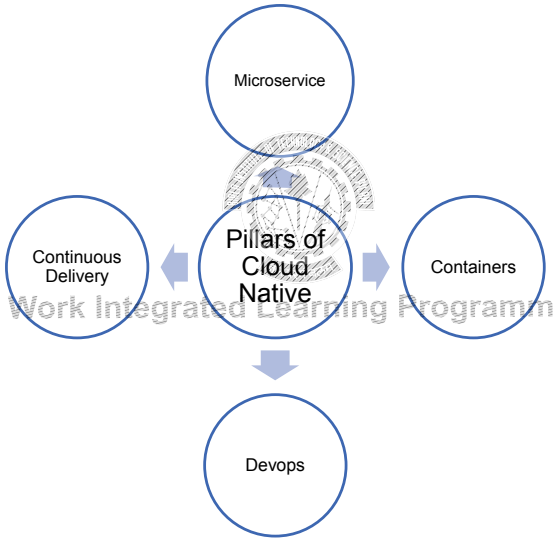
A cloud-enabled application is an application that was developed for deployment in a traditional data center but was later changed so that it could also run in a cloud environment

Cloud-native applications, however, are built to operate only in the cloud.

Developers design cloud-native applications to be

- Scalable
- platform agnostic
- and comprised of microservices

Cloud Native



Building Blocks

Microservices

- ✓ is an architectural approach to developing an application as a collection of small services
- ✓ each service implements business capabilities, runs in its own process and communicates via HTTP APIs or messaging

Containers

- ✓ offer both efficiency and speed compared with standard virtual machines (VMs)
- ✓ Using operating system (OS)-level virtualization, a single OS instance is dynamically divided among one or more isolated containers, each with a unique writable file system and resource quota
- ✓ Low overhead of creating and destroying containers combined with the high packing density in a single VM makes containers an ideal compute vehicle for deploying individual microservices

Building Blocks

DevOps

- ✓ Collaboration between software developers and IT operations to constantly deliver high-quality software that solves customer challenges
- ✓ Creates a culture and an environment where building, testing, and releasing software happens rapidly, frequently, and more consistently

Continuous Delivery

- ✓ is about shipping small batches of software to production constantly through automation
- ✓ makes the act of releasing reliable, so organizations can deliver frequently, at less risk, and get feedback faster from end users

Advantages

Reduced Time to Market
Ease of Management
Scalability and Flexibility
Reduced Cost

Serverless Application

Serverless architecture is an approach to software design that allows developers to build and run services without managing the underlying infrastructure.

- Cloud service providers automatically provision, scale, and manage the infrastructure required to run the code.

Function as a Service

One of the most popular serverless architectures is Function as a Service (FaaS)

Developers write their application code as a set of discrete functions.

Each function will perform a specific task when triggered by an event

When a function is invoked, the cloud provider executes the function

The execution process is abstracted away from the view of developers.

Examples:

- AWS: AWS Lambda
- Microsoft Azure: Azure Functions
- Google Cloud: Cloud Functions

Serverless

Benefits

- Reduced operational cost
- Easier operational management
- Scalability

Drawbacks

- Loss of control
- Vendor lock-in
- Multitenancy problems
- Security concerns

AWS Serverless- Serverless Offering

AWS provides a set of fully managed services that can be used to build and run serverless applications

AWS Lambda

- Allows to run code without provisioning or managing servers

AWS Fargate

- Purpose-built serverless compute engine for containers

Amazon API Gateway

- Fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale

Source : AWS Serverless

Reference Architecture – Web Application

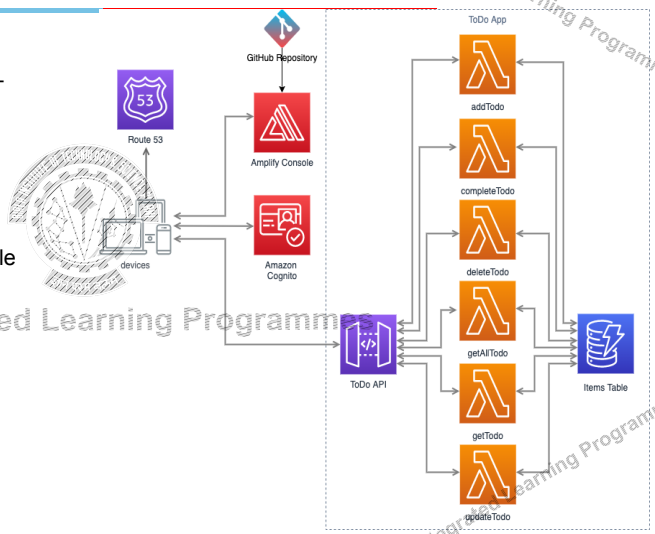
General-purpose, event-driven, web application back-end that uses

AWS Lambda, Amazon API Gateway for its business logic

Uses Amazon DynamoDB as its database

Uses Amazon Cognito for user management

All static content is hosted using AWS Amplify Console



Example: AWS Serverless Architecture

Source : AWS

Differences (The below table generated by ChatGPT)

Aspect	Monolithic	Microservices	Cloud-Native	Serverless	SaaS	Edge-Native
Definition	All components packaged and deployed as a single unit	Application split into independently deployable services	Applications designed specifically for cloud environments	Applications built using event-driven cloud functions	Software delivered over the internet as a service	Applications executed near the data source or user
Deployment Unit	Single deployable artifact	Multiple independent services	Containers and microservices	Individual functions	Provider-managed application	Edge devices, gateways, or local nodes
Scalability	Scales as a whole	Scales per service	Elastic and auto-scalable	Automatic per function	Scaled by provider	Limited, location-specific
Latency	Moderate	Moderate	Moderate	Moderate	Internet-dependent	Very low (near-zero)
Infrastructure Management	Fully managed by organization	Managed by organization	Orchestrated via cloud platforms	Fully managed by cloud provider	Fully managed by provider	Partially managed (edge + cloud)
Technology Stack	Single tech stack	Polyglot allowed	Container-based, cloud services	Cloud-specific runtimes	Abstracted from user	Lightweight, hardware-aware
Fault Isolation	Poor (one failure affects all)	Strong	Strong	Strong	Strong	Strong at local level
Cost Model	Fixed infrastructure cost	Variable	Pay-as-you-go	Pay-per-execution	Subscription-based	Infrastructure + operational cost
Development Complexity	Low	High	High	Moderate	Low for users, high for providers	High
Operational Complexity	Low	Very high	High	Low	Very low for customers	High
Vendor Lock-in	Low	Low	Medium	High	High	Medium
Offline Capability	Possible	Possible	Limited	No	No	Yes
Security Surface	Smaller	Large	Large	Large	Abstracted	Localized but sensitive
Best Use Cases	Small apps, legacy systems	Large-scale distributed systems	Modern scalable web apps	Event-driven workloads	Enterprise productivity software	Real-time, mission-critical systems
Examples	Legacy ERP	Netflix backend	Kubernetes-based apps	AWS Lambda APIs	Salesforce	Autonomous vehicles

Self Reading

Recommendation 1:https://railsware.com/blog/native-vs-hybrid-vs-cross-platform/

Recommendation 2: https://litslink.com/blog/mobile-applications-development-native-web-cross-platform

References

1. https://railsware.com/blog/native-vs-hybrid-vs-cross-platform/
2. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
3. https://www.cncf.io/about/what-is-cloud-native/
4. https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html
5. **Full Stack Web Development" by Philip Ackermann**
   ISBN: 9781801070639

Thank you

Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan
CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 2 – Application Architectures

Web Application

Web application follows the Layered Architecture.

Two-layer systems

- Typical client-server system
- The client held the user interface and other application code, and the server was usually a relational database.
- Embed the logic directly into the UI screens
- Alternative: put the domain logic in the database as stored procedures

Layered Pattern

Context: Separation of concern

Problem: Modules of the system may be independently developed and maintained, supporting portability, modifiability, and reuse.

Solution: The layered pattern divides the software into units called layers.

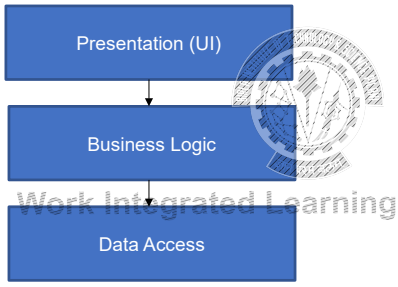
Each layer is a grouping of modules that offers a cohesive set of services.

Each partition is exposed through a interface.

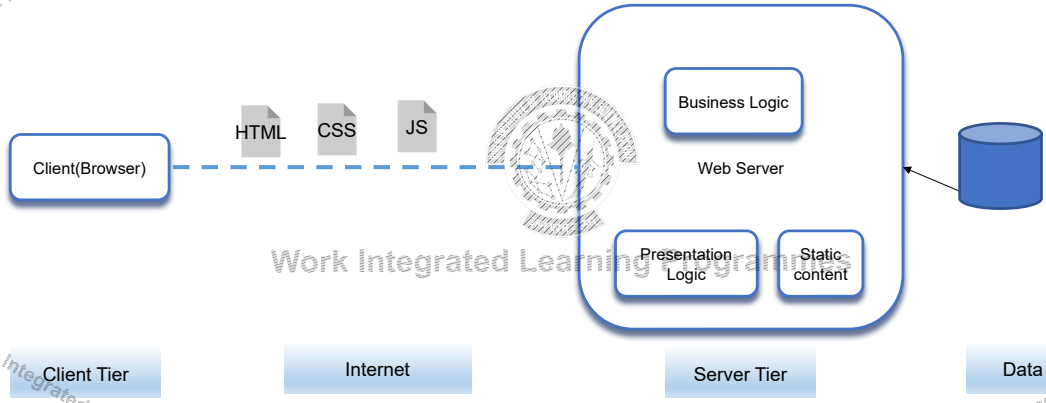
Layers interact according to a strict ordering relation and unidirectional.

3-Tier Architecture

A typical Web Application:



Traditional Web Application



Client Server Architecture

Request – Response Model
Providers of a resource or service, Server
Service requester/Consumer - Client

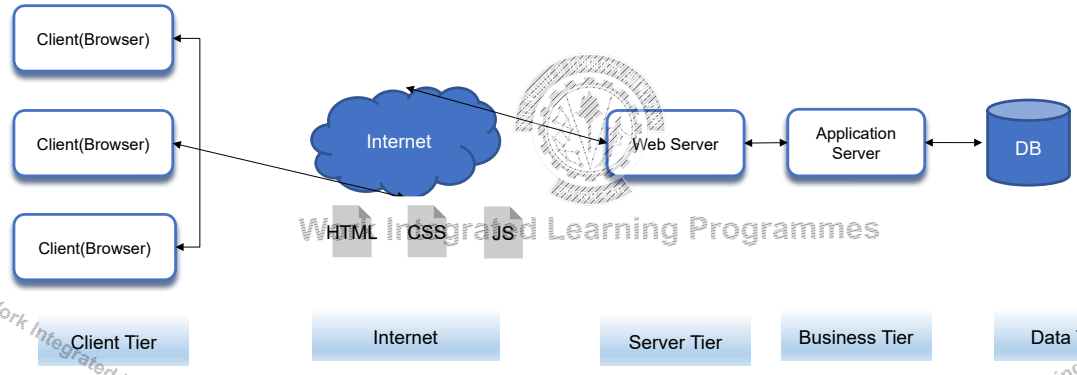
Client Server Architecture

Benefits

- ✓ **Higher security**
- ✓ **Centralized data access**.
- ✓ **Ease of maintenance**.

The client-server architectural style has evolved into the more general 3-tier (or N-tier) architectural style for the web

N-Tier Web Application



Variants of Client Server Pattern:

Peer-to-Peer (P2P) applications
Application servers

Variations

- ✓ Web browsers don't block until the data request is served up - Asynchronous
- ✓ In some client-server patterns, servers can initiate certain actions on their clients.
- ✓ Service calls over a request/reply connector are bracketed by a "session"

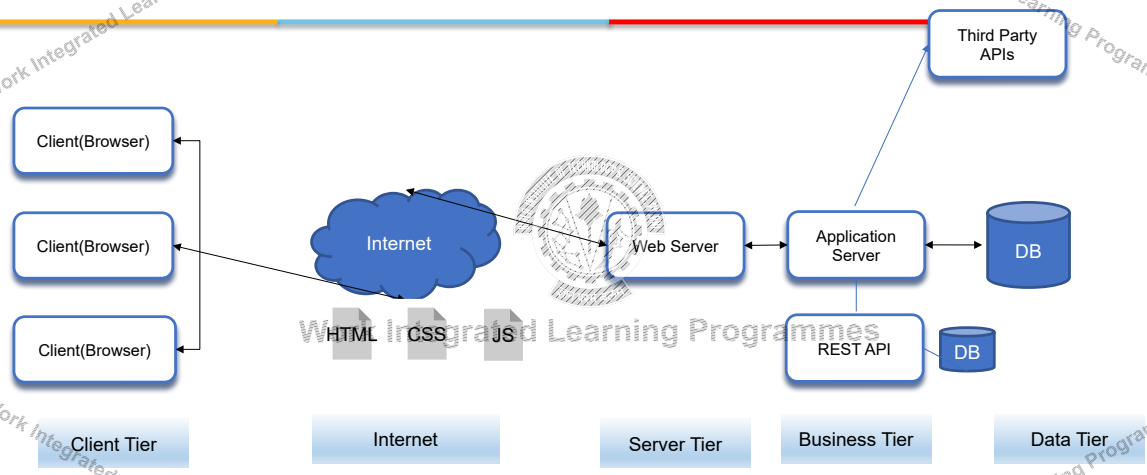
Characteristics

Each tier is completely independent.

The nth tier only has to know how to handle a request from the n+1th tier

Tiers make it easier to ensure security and to optimize performance and availability in specialized ways.

Service Based Web Application



Service Oriented Architecture

Service-oriented architecture (SOA) is a business-centric IT architectural approach that supports integrating your business as linked, repeatable business tasks or services.

SOA exposes business functionalities as services to be consumed by applications.

Services are

- loosely coupled
- Autonomous

Robert C. Martin's Single Responsibility Principle

Gather together the things that change for the same reasons. Separate those things that change for different reasons.

Monolithic Application

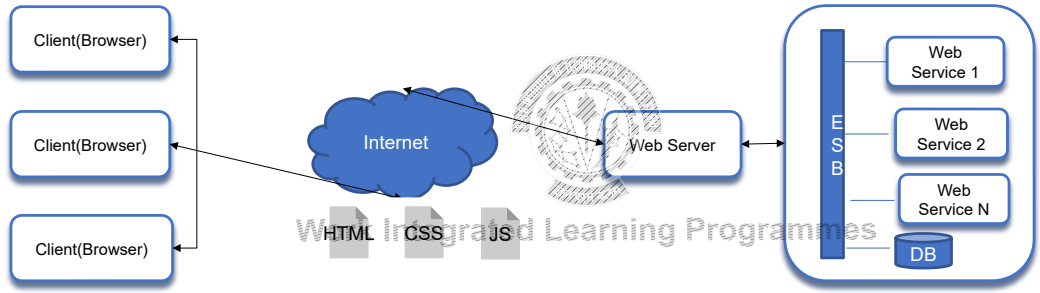
The application is deployed as a single monolithic application.

For example, a Java web application consists of a single WAR file that runs on a web container such as Tomcat

Difficulty to scale

Redeployment of the entire application in case of change

SOA Based Web Application



Microservice Architecture

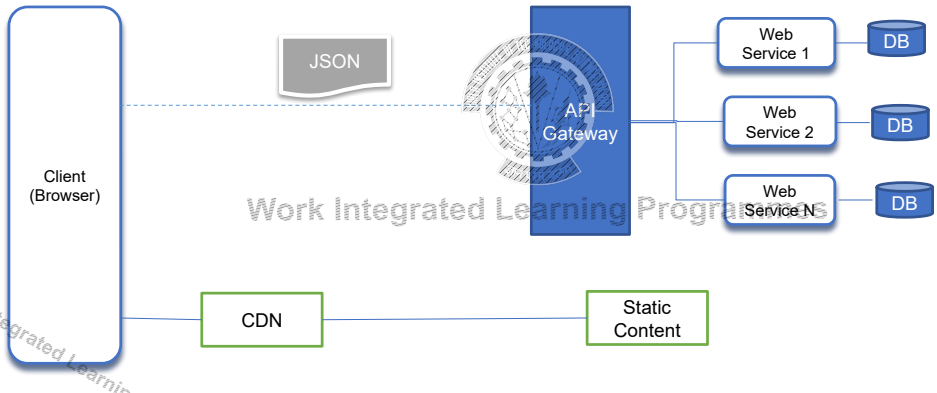
Microservice Application

- ✓ is composed of many small, independent services
- ✓ each service implements a single business capability
- ✓ are loosely coupled, communicating through API contracts
- ✓ can be built by a small, focused development team

More complex to build and manage

- ✓ requires a mature development and DevOps culture
- ✓ can lead to higher release velocity, and a more resilient architecture

Microservices Application



API

API stands for ‘Application Programming Interface’

API: an interface to access whatever resource it points to: data, server software, or other applications

In an internet-connected world, web and mobile applications are designed for humans to use

While APIs are designed for other digital systems and applications to use

What is a API?

An API is a software intermediary that allows two applications to talk to each other

- ✓ API is the messenger that delivers your request to the provider that you're requesting it from and then delivers the response back to you

An API is independent of their respective implementations.

Changing the underlying implementation ca be done without affecting the users

APIs make it possible to integrate different systems together

- ✓ like Customer Relationship Management systems, databases, or even school learning management systems

What is a API?

In this metaphor, a customer is like a user, who tells the waiter what she wants

The waiter is like an API,

- ✓ receiving the customer's order
- ✓ translating the order into easy-to-follow instructions that the kitchen then uses to fulfill that order—often following a specific set of codes
- ✓ or input, that the kitchen easily recognizes

The kitchen is like a server that creates the order in the manner the customer wants it, hopefully!

When the food is ready, the waiter picks up the order and delivers it to the customer.

Similarly, the API delivers the response.

Public APIs vs. Private APIs

Public API

- Twitter API, Facebook API, Google Maps API, and more
- Granting Outside Access to Your Assets
- Provide a set of instructions and standards for accessing the information and services being shared
- Making it possible for external developers to build an application around those assets
- Much more restricted in the assets they share, given they're sharing them publicly with developers around the web

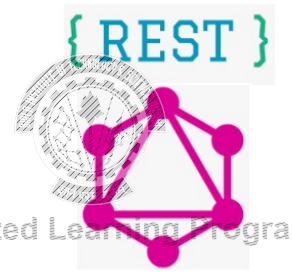
Private API

- Self-Service Developer & Partner Portal API
- Far more common (and possibly even more beneficial, from a business standpoint)
- Give developers an easy way to plug right into back-end systems, data, and software
- Letting engineering teams do their jobs in less time, with fewer resources
- All about productivity, partnerships, and facilitating service-oriented architectures

API Paradigm

Over the years, multiple API paradigms have emerged such as

- ✓ REST
- ✓ RPC
- ✓ GraphQL
- ✓ WebHooks
- ✓ and WebSockets



Broadly can be classified as

- ✓ Request-Response APIs
- ✓ Event Driven APIs

Request-Response APIs

Typically expose an interface through a HTTP-based web server

APIs define a set of endpoints

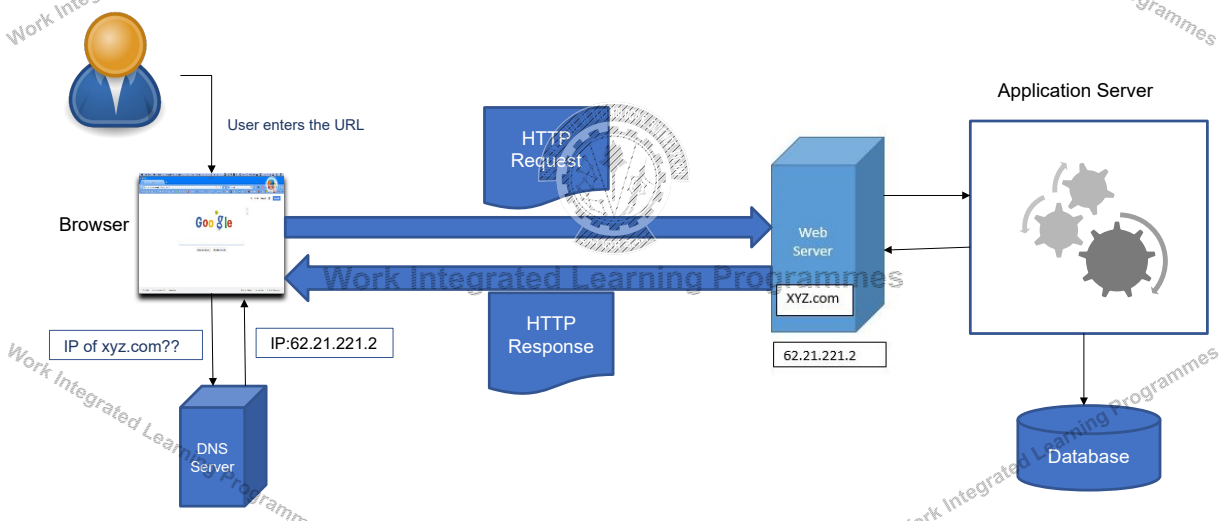
- ✓ Clients make HTTP requests for data to those endpoints
- ✓ Server returns responses

The response is typically sent back as JSON or XML

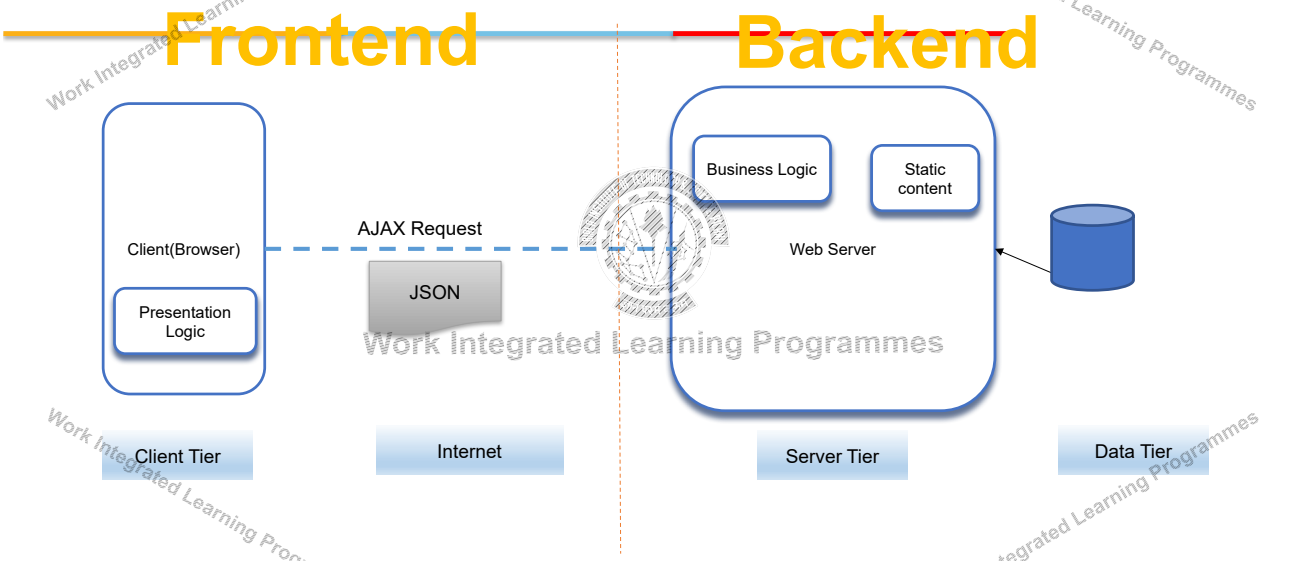
Three common paradigms used to expose request–response APIs:

- ✓ REST
- ✓ RPC
- ✓ and GraphQL

Working of the Web



Modern Web Application



AJAX

Asynchronous JavaScript and XML (Ajax) refer to a concept that is used to develop web applications in a better way.

Ajax defines a method of initiating client-to-server communication without page reloads.

It provides a way to enable partial page updates.

In an Ajax web application, the user is not interrupted in interactions with the web application.

Model of a Web application

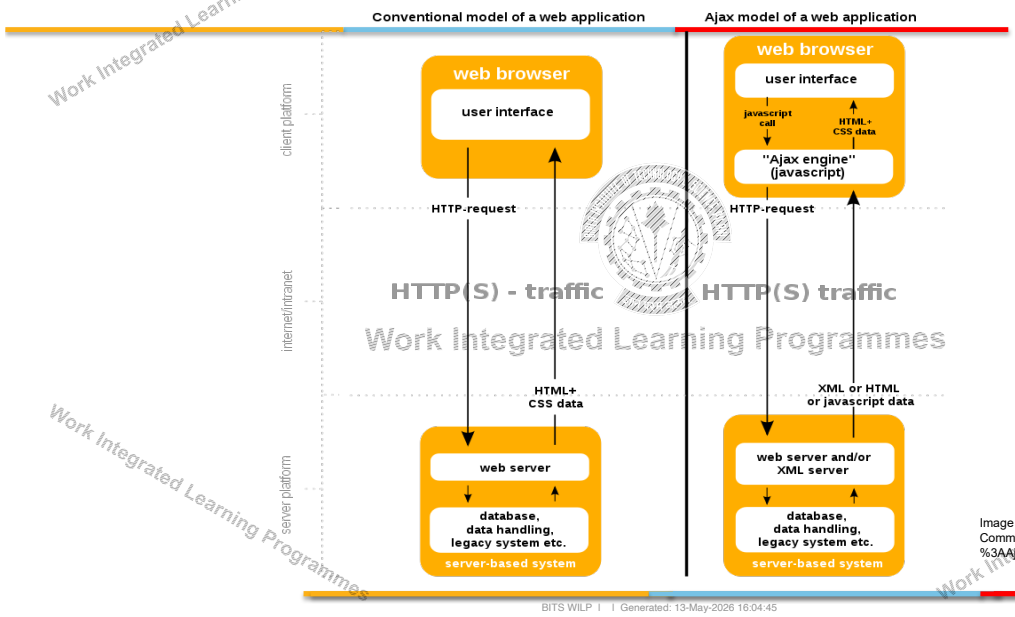


Image source: By DanielSHaischt, via Wikimedia Commons - https://commons.wikimedia.org/wiki/File%3AAjax-vergleich.svg, CC BY-SA 3.0,

Parts of an URL

http://abc.company.com:80/a/b/c.html?user=John&year=2020#p2

The scheme identifies the protocol used to fetch the content.
Host name name of a machine to connect to.
The server's port number allows multiple servers to run on the same machine.
The hierarchical portion is used by the server to find content.
The Query parameters provide additional parameters
Fragment : Have the browser scroll the page to a specific part of the webpage
fragment

Different types of links

Full URL: News

Absolute URL: <a href="/stock/quote.html">

- same as http://www.xyz.com/stock/quote.html

Relative URL (intra-site links): <a href="b/March.html">

- same as http://www.xyz.com/news/2008/March.html

Define an anchor point (a position that can be referenced with # notation):

- <a id="sec3">

Go to a different place in the same page: <a href="#sec3">

URL Encoding

How are special characters sent in the URL?

- http://www.xyz.com/companyInfo?name=A&B CO
- Any character in a URL other than A-Z, a-z, 0-9, or any of -_.~ must be represented as %xx, where xx is the hexadecimal value of the character:
- http://www.xyz.com/companyInfo?name=A%26B%20CO

Escaping is a commonly used technique.

Self Reading

https://www.ibm.com/think/topics/monolithic-vs-microservices

https://litslink.com/blog/single-page-vs-multi-page-applications-benefits-drawbacks-and-pitfalls

BITS Pilani
Pilani Campus

Thank you

Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan
CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 3 – Web Application

Domain Name System

The Domain Name System (DNS) is the phonebook of the Internet.

Humans access information online through domain names such as google.com or facebook.com.

The network of devices interacts through Internet Protocol (IP) addresses.

DNS translates domain names to IP addresses so browsers can load Internet resources.

Each device connected to the Internet has a unique IP address, which other machines use to find the device.

DNS servers eliminate the need for humans to memorize IP addresses

Domain Name System

The Domain Name System resolves the names of internet sites with their underlying IP addresses.

A DNS server is a computer server that contains a database of public IP addresses and their associated hostnames.

DNS is a distributed database implemented in a hierarchy of name servers.

It is an application layer protocol for message exchange between clients and servers.

Domain Name System

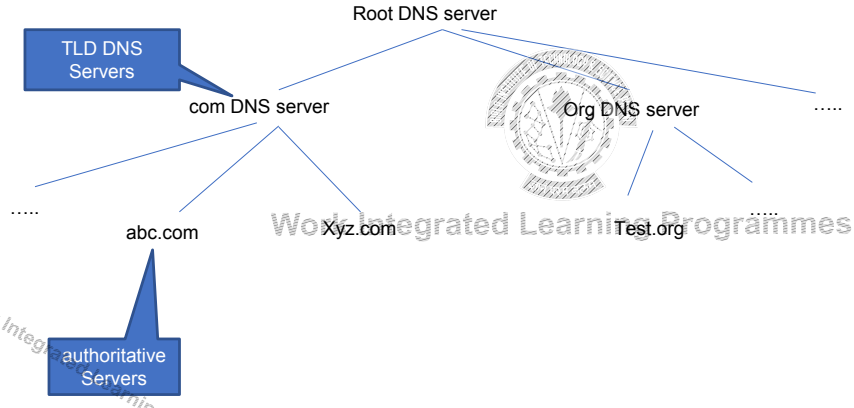
The **DNS recursor** is a server designed to receive queries from client machines through applications such as web browsers.

The **root server** is the first step in translating (resolving) human-readable host names into IP addresses.

TLD nameserver - The top level domain server (TLD)

The authoritative nameserver is the last stop in the nameserver query

Domain Name System Lookup



Work Integrated Learning Programmes

Content Delivery Network

A content delivery network (CDN) is a geographically distributed group of servers that caches content close to end users.

A CDN allows for the quick transfer of assets needed for loading Internet content, including HTML pages, JavaScript files, stylesheets, images, and videos.

Is a CDN the same as a web host?

CDN can't replace the need for proper web hosting, It improve Web performance!!

CDN

A CDN improves website load times.

The globally distributed nature of a CDN reduces the distance between users and website resources.

A CDN caches content (such as images, videos, or webpages) in proxy servers that are located closer to end users than origin servers.

CDNs also reduce the amount of data transferred by reducing file sizes using minification and file compression tactics.

Load balancing distributes network traffic evenly across several servers, making it easier to scale rapid boosts in traffic.

Benefits of using a CDN

Improving website load times
Reducing bandwidth costs
Increasing content availability and redundancy
Improving website security

Client-Side Programming-Frontend / User Interface

Involves everything users see on their screens.

Major frontend technology stack components:

- HTML, CSS and JS

Hypertext Markup Language (HTML) and Cascading Style Sheets (CSS)

- HTML tells a browser how to display the content of web pages
- CSS styles that content
- Bootstrap is a helpful framework for managing HTML and CSS

JavaScript (JS)

- Makes web pages interactive
- Many JavaScript libraries (such as jQuery, React.js)
- frameworks (such as Angular, Vue, Backbone, and Ember)

Server-Side Programming-Backend

Major server-side technology stack components:

- Programming language, Framework, web server and databases

Server-side programming languages used to create the logic of applications

Frameworks offer lots of tools for simpler and faster development of applications

Options

- Ruby (Ruby on Rails)
- Python (Django, Flask, Pylons)
- PHP (Laravel)
- Java (Spring)
- Scala (Play)
- Javascript (Express Node.js)

Other Components

Storage

- Apps needs a place to store its data
- Two types of databases:

○relational and non-relational

- Most common databases for web development:

○MySQL (relational)
○PostgreSQL (relational)
○MongoDB (non-relational, document)

Other Components

Caching system
- Used to reduce the load on the database and to handle large amounts of traffic
- Memcached and Redis are the most widespread.

Web servers/Load balancers/Proxy servers
- Needs a server to handle requests from clients' computers
- Two major players: Apache, Nginx
- Cloud Based Servers (EC2, Serverless, ELB)

Example TechStack

MEAN / MERN / MEVN Stack
- MongoDB: A NoSQL database that stores data in a flexible, JSON-like format.
- Express.js: A minimal and flexible Node.js web application framework that provides robust features for web and mobile applications.
- Frontend Framework
  Vue.js: A JavaScript framework for building user interfaces.
  Angular: A TypeScript-based open-source front-end web application framework maintained by Google.
  React: A JavaScript library for building user interfaces, developed by Facebook.
- Node.js: A JavaScript runtime built on Chrome's V8 JavaScript engine. It allows developers to use JavaScript for server-side scripting.

Python Stack:
- Flask or Django: Flask is a lightweight Python web framework, and Django is a high-level Python web framework.
- SQLAlchemy: An SQL toolkit and Object-Relational Mapping (ORM) library.
- Django REST framework: If using Django for building APIs.
- Database: Various options, including SQLite, PostgreSQL, or MySQL.
- HTML/CSS/JavaScript or React/Angular for frontend

innovate achieve lead

BITS Pilani
Pilani Campus

Rendering Patterns

Client-side rendering (CSR)

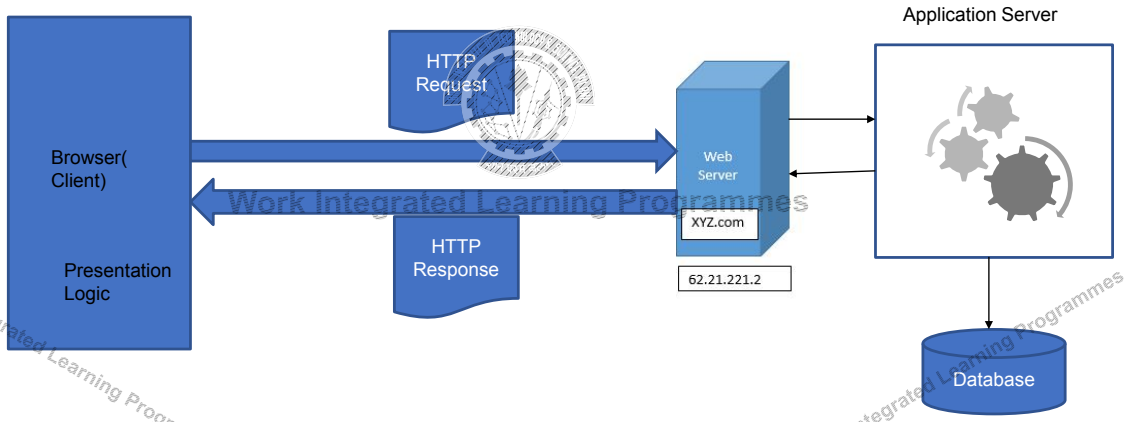
innovate achieve lead

Client-side rendering (CSR) is a technique for rendering web content on the client-side, i.e., in the user's browser.

With CSR, the client requests a minimal HTML file from the server containing the necessary JavaScript and CSS files.

When the client loads the JavaScript files, the JavaScript code is executed, which renders the content in the browser.

Client Side rendering



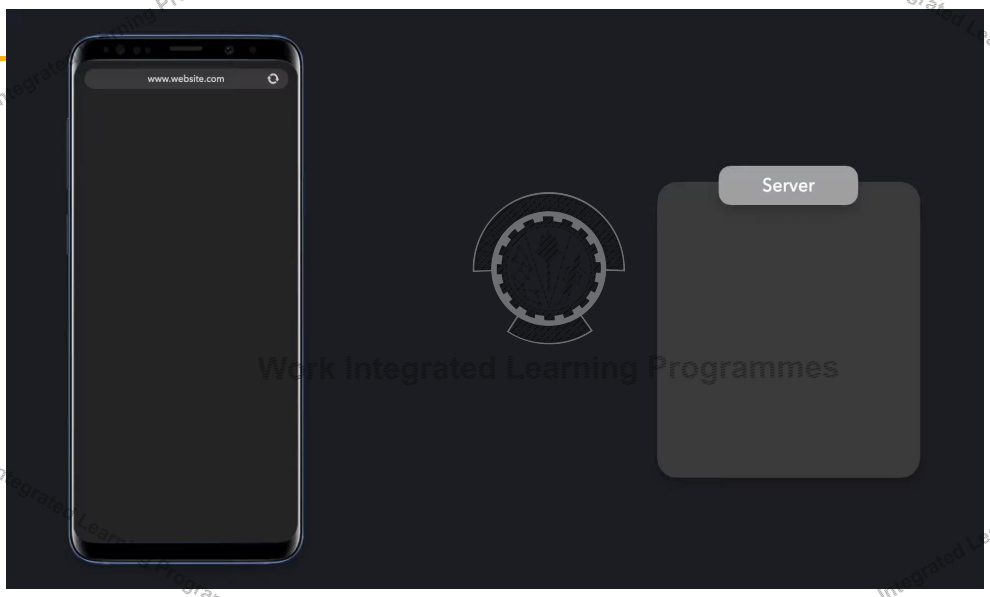
Server-Side Rendering

Server-side rendering (SSR) is a technique for rendering web content on the server-side, i.e., before the page is sent to the client.

Server-Side Rendering is also named Pre-Rendering because the fetching of external data and transformation of components, content, and data into HTML happens before the result is sent to the client.

There are several benefits of using server-side rendering:

- Better SEO
- Faster Initial Page Load
- Improved Accessibility
- Better Performance on Low-Powered Devices



Static Site Generation

Static site generation (SSG) is a popular approach to website development that involves generating a website's content ahead of time and delivering it as static HTML files to end-users.

In SSG, the HTML is generated once, at build time.

The HTML is stored in a CDN or elsewhere and re-used for each request.

A static site generator combines the content and templates into a collection of static HTML files.

This process may also include optimizing images, minifying code, and generating metadata.

Static Site Generation

Benefits:

- Performance
- SEO
- Cost

Limitations:

- Not suitable for all types
- No Real-time data

Some popular static site generators include Jekyll, Hugo, and Gatsby.

Example- Case study- Ecommerce(Generated by ChatGPT)

Page Type	Rendering Method	Why?
Homepage	✓ SSG (Static Site Generation)	✓ Prebuild for fast load, ✓ SEO-friendly
Product Listing Page (Category Page)	✓ SSR (Server-Side Rendering)	✓ Always fresh data, ✓ SEO
Product Details Page	✓ SSG (with ISR for periodic updates)	✓ Prebuild for speed, ✓ SEO-friendly, ✓ Updates periodically
User Dashboard (Orders, Wishlist)	✓ CSR (Client-Side Rendering)	✗ Not SEO-relevant, ✓ Personal user data, ✓ Faster navigation
Cart & Checkout	✓ CSR (Client-Side Rendering)	✗ No SEO needed, ✓ User-specific, ✓ Immediate updates
Admin Dashboard	✓ CSR (Client-Side Rendering)	✗ No SEO needed, ✓ Better interactivity
Search Page	✓ SSR (Server-Side Rendering)	✓ Fresh, dynamic results, ✓ Good for SEO

References

1. https://www.patterns.dev/vanilla/rendering-patterns/

Thank you

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani

Pilani Campus

innovate achieve lead

BITS Pilani

Pilani Campus

Lecture No: 4 – Web Protocols

Module 3: Web Protocols

HTTP
- HTTP Request- Response and its structure
- HTTP Methods;
- HTTP Headers
- Connection management - HTTP/1.1, HTTP/2

Synchronous and asynchronous communication

Communication with Backend
- AJAX, Fetch API
- Webhooks
- Server-Sent Events

Polling

Bidirectional communication - Web sockets

BITS Pilani
Pilani Campus

HTTP

HyperText Transfer Protocol(HTTP)

HTTP is a request-response client-server protocol

HTTP is a stateless protocol.

HTTP is an application-level protocol for distributed, collaborative, hypermedia information systems.

Each HTTP communication (request or response) between a browser and a Web server consists of two parts:

- A header and
- A body

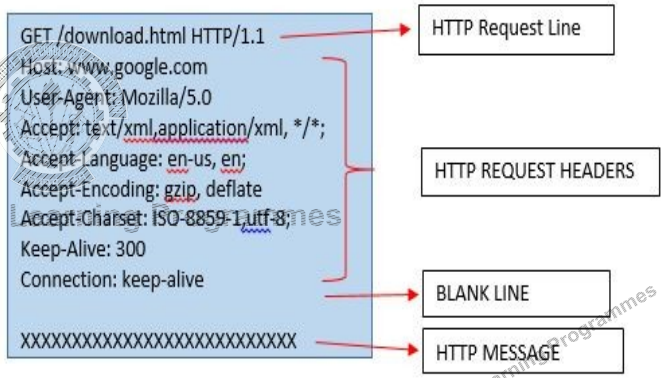
The header contains information about the communication.

The body contains the data of the communication (optional).

HTTP Request

The general form of an HTTP request is:

1. HTTP Request Line
2. Header fields
3. Blank line
4. Message body (Optional)



Request Line

The first line of the header is called the request line:
request-method-name /request-URI /HTTP-version
Examples of request line are:
GET /test.html HTTP/1.1
HEAD /query.html HTTP/1.0
POST /index.html HTTP/1.1

Request Headers

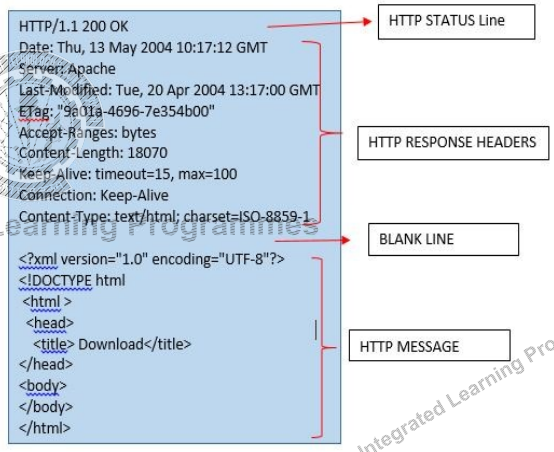
The request headers are in the form of name:value pairs. Multiple values, separated by commas, can be specified.

request-header-name: request-header-value1, request-header-value2, ...
Examples of request headers are:
Host: www.xyz.com
Connection: Keep-Alive
Accept: image/gif, image/jpeg, */*
Accept-Language: us-en, fr, cn

HTTP Response

The general form of an HTTP response is:

1. Status line
2. Response header fields
3. Blank line
4. Response body



Status Line

The first line is called the status line.

HTTP-version status-code reason-phrase

status-code: a 3-digit number generated by the server to reflect the outcome of the request.

reason-phrase: gives a short explanation to the status code.

Common status code and reason phrase are "200 OK", "404 Not Found", "403 Forbidden", "500 Internal Server Error".

Examples of status line are:

HTTP/1.1 200 OK

HTTP/1.0 404 Not Found

HTTP/1.1 403 Forbidden

STATUS CODE

Status code is a three-digit number; first digit the specifies the general status

1 => Informational
2 => Success
3 => Redirection
4 => Client error
5 => Server error

Common HTTP Response Status Codes

200 OK Indicates a nonspecific success
201 Created Sent primarily by collections and stores but sometimes also by controllers, to indicate that a new resource has been created
204 No Content Indicates that the body has been intentionally left blank
301 Moved Permanently Indicates that a new *permanent* URI has been assigned to the client's requested resource
303 See Other Sent by controllers to return results that it considers optional
401 Unauthorized Sent when the client either provided invalid credentials or forgot to send them
402 Forbidden Sent to deny access to a protected resource
404 Not Found Sent when the client tried to interact with a URI that the REST API could not map to a resource
405 Method Not Allowed Sent when the client tried to interact using an unsupported HTTP method
406 Not Acceptable Sent when the client tried to request data in an unsupported media type format
500 Internal Server Error Tells the client that the API is having problems of its own

Response Headers

The response headers are in the form name:value pairs:
response-header-name: response-header-value1, response-header-value2, ...
Examples of response headers are:
Content-Type: text/html
Content-Length: 35
Connection: Keep-Alive
Keep-Alive: timeout=15, max=100
The response message body contains the resource data requested.

HTTP Methods (Verbs)

GET - Fetch a URL
HEAD - Fetch information about a URL
PUT - Store to an URL
POST - Send form data to a URL and get a response back
DELETE - Delete a resource in URL

GET and POST (forms) are commonly used.

HTTP Methods

HTTP defines a set of **request methods** to indicate the desired action for a given resource.

The request methods are sometimes referred to as *HTTP verbs*.

GET - Fetch a URL

HEAD - Fetch information about a URL

PUT - Store to an URL

POST - Send form data to a URL and get a response back

DELETE - Delete a resource in URL

GET and POST (forms) are commonly used.

HTTP Methods

GET

The GET method requests a representation of the specified resource. Requests using GET should only retrieve data.

The GET request method is said to be a safe operation, which means it should not change the state of any resource on the server.

The GET method is used to request any of the following resources:

A webpage or HTML file.

An image or video.

A JSON document.

A CSS file or JavaScript file.

An XML file.

HTTP Methods

POST:

The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.

The POST HTTP request method sends data to the server for processing.

The data sent to the server is typically in the following form:

- Input fields from online forms.
- XML or JSON data.
- Text data from query parameters.

A POST operation is not considered a safe operation, as it has the power to update the state of the server and cause potential side effects to the server's state when executed.

The HTTP POST method is not required to be idempotent either, which means it can leave data and resources on the server in a different state each time it is invoked.

HTTP Methods

HEAD

The HEAD method requests a response identical to a GET request but without the response body.

The HTTP HEAD method returns metadata about a resource on the server.

The HTTP HEAD method is commonly used to check the following conditions:

- The size of a resource on the server.
- If a resource exists on the server or not.
- The last-modified date of a resource.
- Validity of a cached resource on the server.
- The following example shows sample data returned from a HEAD request:


```
HTTP/1.1 200 OK
Date: Fri, 19 Aug 2023 12:00:00 GMT
Content-Type: text/html
Content-Length: 1234
Last-Modified: Thu, 18 Aug 2023 15:30:00 GMT
```

HTTP Methods

PUT

The HTTP PUT method is used to replace a resource identified with a given URL completely.

The HTTP PUT request method includes two rules:

A PUT operation always includes a payload that describes a completely new resource definition to be saved by the server.

The PUT operation uses the exact URL of the target resource.

If a resource exists at the URL provided by a PUT operation, the resource's representation is completely replaced.

A new resource is created if a resource does not exist at that URL.

The payload of a PUT operation can be anything that the server understands, although JSON and XML are the most common data exchange formats for RESTful web services.

HTTP Methods

DELETE

The DELETE method deletes the specified resource.

CONNECT

The CONNECT method establishes a tunnel to the server identified by the target resource.

OPTIONS

The OPTIONS method describes the communication options for the target resource.

TRACE

The TRACE method performs a message loop-back test along the path to the target resource.

PATCH

The PATCH method applies partial modifications to a resource.

HTTP Request

HTTP Request methods		
	SAFE	**IDEMPOTENT**
GET	Yes	Yes
POST	No	No
PUT	No	Yes
PATCH	No	No
DELETE	No	Yes
TRACE	Yes	Yes
HEAD	Yes	Yes
OPTIONS	Yes	Yes
CONNECT	No	No

OPTIONS /example/resource HTTP/1.1

Host: www.example.com HTTP/1.1 200 OK

Allow: GET, POST, DELETE, HEAD, OPTIONS

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS

Access-Control-Allow-Headers: Authorization, Content-Type

HTTP HEADERS

HTTP headers allow the client and the server to pass additional information with the request or the response.

An HTTP header consists of its name followed by a colon ':', then by its value.

Headers can be grouped according to their contexts:

General header: Headers applying to both requests and responses but with no relation to the data eventually transmitted in the body.

Request header: Headers containing more information about the resource to be fetched or about the client.

Response header: Headers with additional information about the response, like its location or about the server itself (name and version etc.).

Entity header: Headers containing more information about the body of the entity, like its content length or its MIME-type.

HTTP Headers –Content

The Accept header

The Accept header lists the MIME types of media resources that the agent is willing to process.

Each combined with a quality factor, a parameter indicating the relative degree of preference between the different MIME types.

A media type also known as MIME type is a two-part identifier for file formats and format contents transmitted on the Internet.

Form: type/subtype

Examples: `text/plain`, `text/html`, `image/gif`, `image/jpeg`

Accept: image/gif, image/jpeg, */*

The Accept-Charset header

It indicates to the server what kinds of character encodings are understood by the user-agent.

Accept-Charset:utf-8

HTTP Headers -Content

The Accept-Encoding header

The Accept-Encoding header defines the acceptable content-encoding (supported compressions).

The value is a q-factor list (e.g.: br, gzip;q=0.8) that indicates the priority of the encoding values.

Compressing HTTP messages is one of the most important ways to improve the performance of a Web site.

Accept-Encoding:gzip, deflate

The Accept-Language header

It is used to indicate the language preference of the user.

Accept-Language: en-us

The User-Agent header

It identifies the browser sending the request.

HTTP Headers- Caching

Cache-Control header

The Cache-Control general-header field is used to specify directives for caching mechanisms in both requests and responses.

Caching directives are unidirectional, i.e., directive in a request is not implying that the same directive is to be given in the response.

Standard Cache-Control directives that can be used by the client in an HTTP request.

- Cache-Control: max-age=<seconds>
- Cache-Control: no-cache
- Cache-Control: no-store
- Cache-Control: no-transform

Standard Cache-Control directives that can be used by the server in an HTTP response.

- Cache-Control: must-revalidate
- Cache-Control: no-cache
- Cache-Control: no-store
- Cache-Control: no-transform
- Cache-Control: public
- Cache-Control: private

HTTP Headers- Caching

Expires Header

The Expires header contains the date/time after which the response is considered stale.

Invalid dates, like the value 0, represent a date in the past and mean that the resource is already expired.

If there is a Cache-Control header with the "max-age" directive in the response, the Expires header is ignored.

Expires: Wed, 21 Oct 2015 07:28:00 GMT

HTTP Headers –Caching Etag

The ETag HTTP response header is an identifier for a specific version of a resource.
If the resource at a given URL changes, a new Etag value must be generated.
ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4“

The client will send the Etag value of its cached resource along in an If-None-Match header field:
If-None-Match: "33a64df551425fcc55e4d42a148795d9f25f89d4“

The server compares the client's ETag (sent with If-None-Match) with the ETag for its current version of the resource
If both values match, the server send back a 304 Not Modified status, without any body
Tells the client that the cached version of the response is still good.

HTTP Headers-Caching Last-Modified Header

The Last-Modified response HTTP header contains the date and time at which the origin server believes the resource was last modified.
It is used as a validator to determine if a resource received or stored is the same.
Less accurate than an ETag header

Redirection

In HTTP, a redirection is triggered by the server by sending special responses to a request: redirects.

HTTP redirects are responses with a status code of 3xx.

A browser, when receiving a redirect response, uses the new URL provided in the location header.

Location: /index.html

Permanent redirections

301 Moved Permanently

Temporary Redirections

302 Temporary Redirect

HTTP Headers-Cookies

An HTTP cookie is a small piece of data that a server sends to the user's web browser.

The browser may store it and send it back with the next request to the same server.

The Set-Cookie HTTP response header sends cookies from the server to the user agent.

Set-Cookie: <cookie-name>=<cookie-value>

The Cookie HTTP request header contains stored HTTP cookies previously sent by the server with the Set-Cookie header.

Cookie: name=value; name2=value2; name3=value3

HTTP Headers-CORS

Same-Origin Policy (SOP): By default, browsers **block requests** from one origin to another for security reasons.

Same origin = scheme + domain + port are identical.
Example: http://example.com:80/page → can access other resources from the same host/port.

Different origin (cross-origin):
http://example.com → http://api.example.com (different subdomain).
http://example.com → https://example.com (different protocol).
http://example.com:3000 → http://example.com:4000 (different port).
Without CORS, cross-origin requests are blocked.

HTTP Headers-CORS

When a browser makes a **cross-origin HTTP request**, it checks CORS headers in the response.
Common CORS Response Headers:
Access-Control-Allow-Origin: Specifies which origins are allowed.
Example: Access-Control-Allow-Origin: * → Any origin can access.
Access-Control-Allow-Origin: https://myapp.com → Only myapp.com can access.

Access-Control-Allow-Methods: Defines allowed HTTP methods.
Example: GET, POST, PUT, DELETE.

Access-Control-Allow-Headers: Specifies which custom headers a client can use.
Access-Control-Allow-Credentials: Indicates if cookies/authorization headers are allowed (true or false).

HTTP Headers-CORS

Simple Requests
- Methods: GET, POST, or HEAD.
- Browser directly sends the request, server replies with CORS headers.

Preflight Requests
- For methods like PUT, DELETE, or custom headers.
- Browser sends an **OPTIONS request** first to ask the server:
- If server responds with correct CORS headers → actual request is sent.

HTTP Connection

Module 3: Web Protocols

HTTP
- HTTP Request- Response and its structure
- HTTP Methods;
- HTTP Headers
- Connection management - HTTP/1.1 and HTTP/2

Synchronous and asynchronous communication
Communication with Backend
- AJAX, Fetch API
- Webhooks
- Server-Sent Events

Polling
Bidirectional communication - Web sockets

HTTP Headers- Connection Management

The Connection general header controls whether or not the network connection stays open after the current transaction finishes.

If the value sent is keep-alive, the connection is persistent and not closed, allowing for subsequent requests to the same server to be done.

Connection: keep-alive

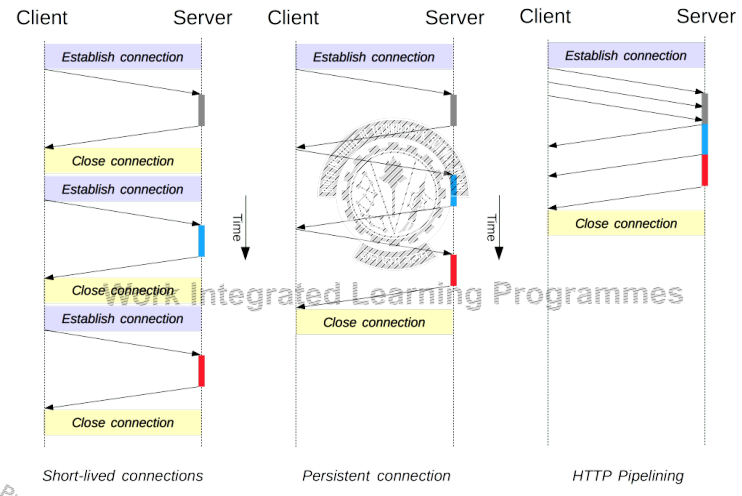
Connection: close

Keep-alive Header

The Keep-Alive general header allows the sender to hint about how the connection may be used to set a timeout and a maximum amount of requests.

Keep-Alive: timeout=5, max=1000

HTTP Connection 1.x



Reference: https://developer.mozilla.org/en-US/docs/Web/HTTP/Connection_management_in_HTTP_1.x

HTTP/1 Problems

HTTP/1.x clients need to use multiple connections to achieve concurrency and reduce latency;

-> Head of line Problem

HTTP/1.x does not compress request and response headers, causing unnecessary network traffic;

HTTP/1.x does not allow effective resource prioritization, resulting in poor use of the underlying TCP connection

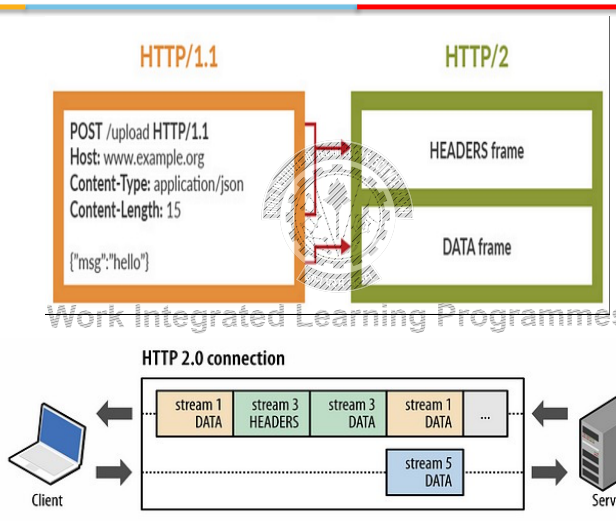
HTTP/2

Hypertext Transfer Protocol version 2, Draft 17

HTTP/2 enables a more efficient use of network resources and a reduced perception of latency by introducing header field compression and allowing multiple concurrent exchanges on the same connection… Specifically, it allows interleaving of request and response messages on the same connection and uses an efficient coding for HTTP header fields. It also allows prioritization of requests, letting more important requests complete more quickly, further improving performance.

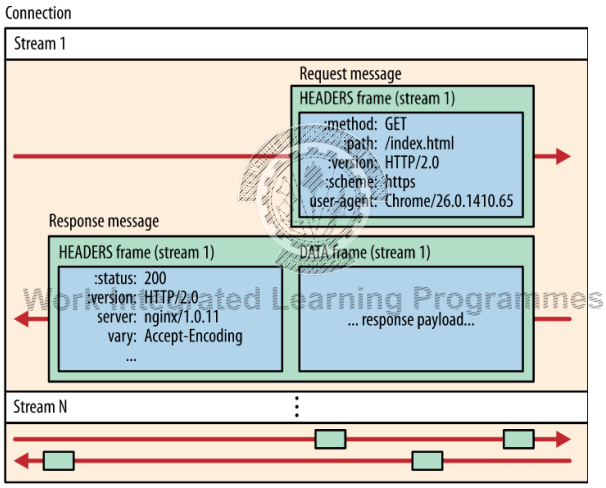
The resulting protocol is more friendly to the network, because fewer TCP connections can be used in comparison to HTTP/1.x. This means less competition with other flows, and longer-lived connections, which in turn leads to better utilization of available network capacity. Finally, HTTP/2 also enables more efficient processing of messages through use of binary message framing.

HTTP/2



Reference: https://web.dev/performance-http2/

HTTP/2



Reference: https://web.dev/performance-http2/

Frames -> Messages -> Streams -> A single TCP connection

HTTP/2

Streams, messages, and frames #

The introduction of the new binary framing mechanism changes how the data is exchanged between the client and server.

Stream: A bidirectional flow of bytes within an established connection, which may carry one or more messages.

Message: A complete frame sequence that maps to a logical request or response message.

Frame: The smallest unit of communication in HTTP/2, each containing a frame header, which at a minimum, identifies the stream to which the frame belongs.

HTTP/2

HTTP/2 does not modify the semantics of HTTP, meaning the core concepts found in HTTP/1.1, such as methods, status codes, URIs, and header fields, remain the same.

Instead, HTTP/2 modifies how the data is formatted (framed) and transported between the client and server, both of which manage the entire process, and hides protocol complexity within a framing layer. As a result, all existing applications can be delivered over the protocol without modification.

Features and Benefits of HTTP/2

HTTP/2 also has problems, We now have HTTP/3 as well!

It's a **binary** protocol

It used header **compression** HPACK to reduce the overhead size

It allows servers to "**push**" responses proactively into client caches instead of waiting for a new request for each resource

It reduces additional **round trip times (RTT),** making your website load faster without any optimization.

HTTP/2 supports response **prioritization**.

QUIC

QUIC (Quick UDP Internet Connections) is a new encrypted-by-default Internet transport protocol.

Built-in security (and performance)

The initial QUIC handshake combines the typical TCP three-way handshake and the TLS 1.3 handshake.

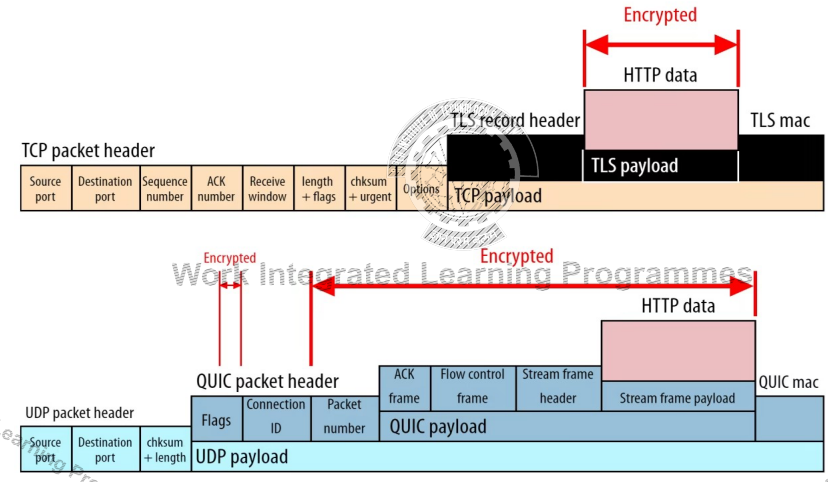
QUIC handshake only takes a single round-trip between client and server to complete

It also encrypts additional connection metadata

Reduced head-of-line blocking

Improved congestion control

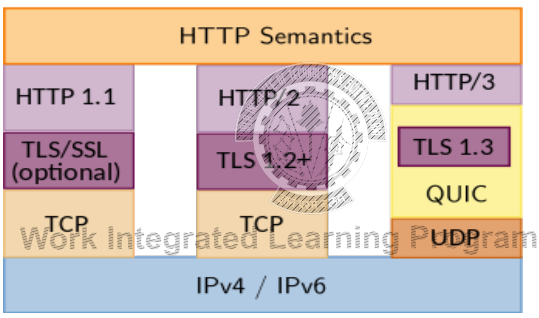
Connection Metadata - encrypted



Benefits - QUIC

QUIC is more secure for its users.
QUIC's connection set-up is faster.
QUIC can evolve more easily.

HTTP Versions



References

https://www.cloudflare.com/en-gb/learning/performance/http2-vs-http1.1/
https://ably.com/topic/http3

Thank You!

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani
Pilani Campus

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture 5:HTTPS

HTTPS

Hypertext Transfer Protocol Secure is a secure version of HTTP.

This protocol enables secure communication between a client (e.g. web browser) and a server (e.g. web server) by using encryption.

HTTPS URLs begin with **https** instead of **http**.

HTTPS uses a well-known TCP port 443.

Make sure you always send requests over HTTPS and never ignore invalid certificates.

HTTPS is the only thing protecting requests from being intercepted or modified!!

TLS handshake

HTTPS uses **Transport Layer Security (TLS)** protocol or its predecessor **Secure Sockets Layer (SSL)** for encryption.

The TLS handshake enables the TLS client and server to establish the secret keys with which they communicate.

Agree on the version of the protocol to use.

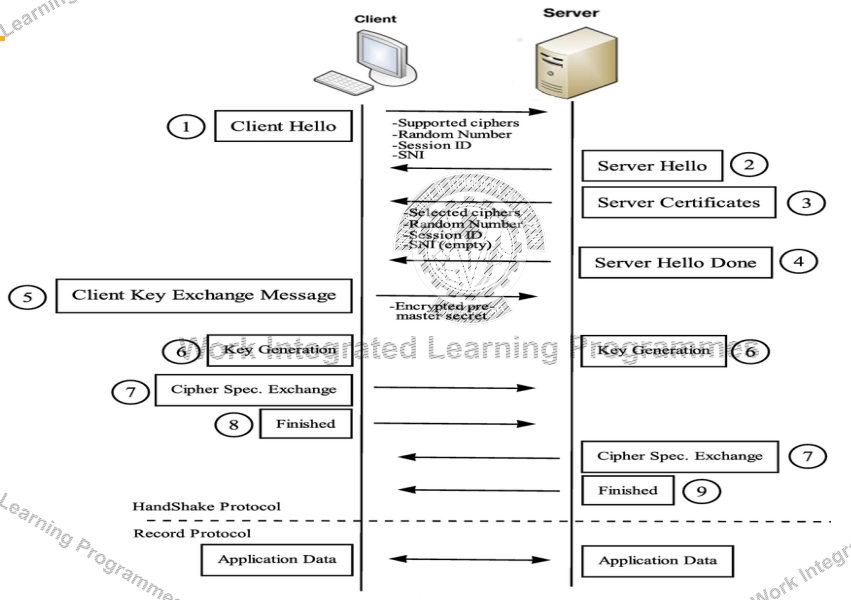
Select cryptographic algorithms.

Authenticate each other by exchanging and validating digital certificates.

Use asymmetric encryption techniques to generate a shared secret key, which avoids the key distribution problem.

SSL or TLS then uses the shared key for the symmetric encryption of messages, which is faster than asymmetric encryption.

The TLS handshake



https://www.researchgate.net/figure/TLS-handshake-protocol_fig1_298065605

TLS Handshake

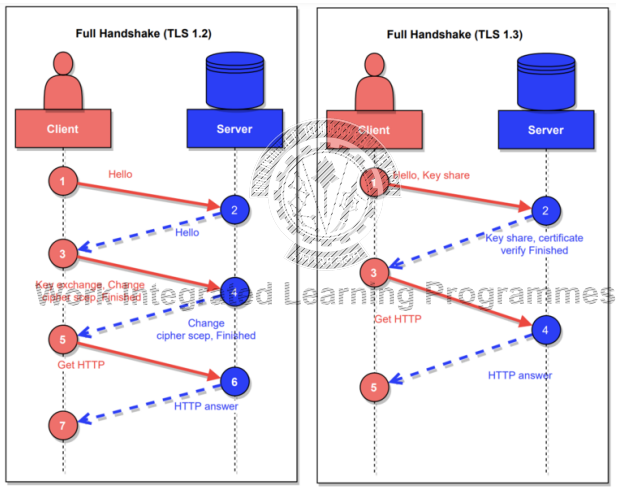


Image Ref: https://commons.m.wikimedia.org/wiki/File:Handshake_in_TLS.png

Certificates

.The HTTPS protocol is based on the server having a public key certificate, sometimes called an SSL certificate.

When a secure connection is established, the server will send the client its TLS/SSL certificate.

The client will then check the certificate against a trusted Certificate Authority, essentially validating the server's identity.

A TLS/SSL certificate essentially binds an identity to a pair of keys which are then used by the server to encrypt as well as sign the data.

A Certificate Authority is an entity that issues TLS/SSL or Digital certificates

HTTP

- HTTP Request- Response and its structure
- HTTP Methods;
- HTTP Headers
- Connection management - HTTP/1.1 and HTTP/2

Synchronous and asynchronous communication

Communication with Backend

- AJAX, Fetch API
- Webhooks
- Server-Sent Events

Polling

Bidirectional communication - Web sockets

Calling API from Frontend

Fetching data from the server

How do you call a service from a client?

The browser makes one or more HTTP requests to the server for the files needed to display the page, and the server responds with the requested files.

The browser reload the page with the new data.

So, instead of the traditional model, many websites use JavaScript APIs to request data from the server and update the page content without a page load.

This is called AJAX

The Fetch API enables JavaScript on a page to request an HTTP server to retrieve specific resources.

Fetch API

The Fetch API provides an interface for fetching resources

Provides a generic definition of Request and Response objects

The fetch() method takes one mandatory argument, the path to the resource you want to fetch.

It returns a promise that resolves to the response of that request (successful or not).

You can optionally pass an init options object as a second argument (used to configure req headers for other types of HTTP requests such as PUT, POST, DELETE)

Fetch API Example

```
fetch("https://www.boredapi.com/api/activity")
    .then(response => {
        return response.json();
    })
    .then(data => {
        console.log(JSON.stringify(data));
        document.getElementById("h1id").innerText = data.activity;
    }
    )
    .catch(error => {
        alert('There was a problem with the request.');
    });
```

Axios

Axios is a promise-based HTTP client for JavaScript.
It allows you to

- Make XMLHttpRequests from the browser
- Make http requests from node.js
- Supports the Promise API
- Automatic transforms for JSON data

Axios

Axios provides more functions to make other network requests as well, matching the HTTP verbs that you wish to execute, such as:

- axios.post(<uri>, <payload>)
- axios.put(<uri>, <payload>)
- axios.delete(<uri>, <payload>)

Fetch vs Axios

Fetch API is built into the window object, and therefore doesn't need to be installed as a dependency or imported in client-side code.

Axios needs to be installed as a dependency.

However, it automatically transforms JSON data.

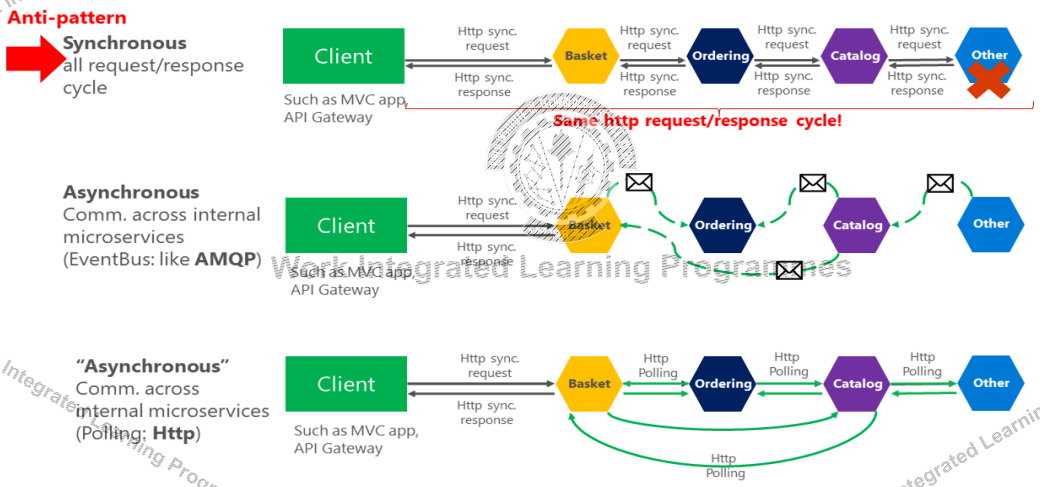
If you use .fetch() there is a two-step process when handing JSON data.

The first is to make the actual request and then the second is to call the .json() method on the response.

Lecture 6: Synchronous vs Asynchronous communication

Synchronous vs Asynchronous communication

Synchronous vs. async communication across microservices



Long running Transactions

API that performs tasks that could run longer than the request timeout limit.

The server typically replies with a 202 Accepted Response indicating that the request is accepted and is in progress.

HTTP is a **unidirectional** protocol, which means the client always initiates the communication.

So client has to periodically check the server , if the work has been completed.

Approaches

How Server to Client Real Time Notification Response Works?

Polling
Webhooks
Message Queues

Communication to Frontend

- Server Sent Events
- Websockets

HTTP Polling

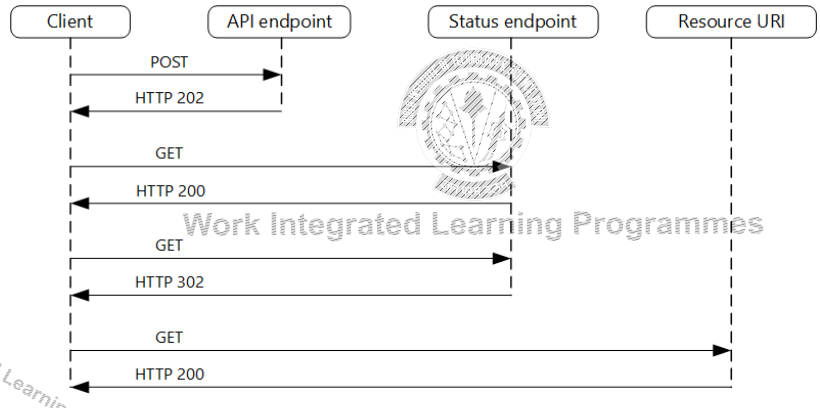
HTTP Polling

HTTP Polling is a mechanism where the client requests the resource regularly at intervals.

If the resource is available, the server sends the resource as part of the response.

If the resource is not available, the server returns an empty response to the client.

HTTP Polling



HTTP Polling

- An HTTP 202 response should indicate the location and frequency that the client should poll for the response.
- It should have the following additional headers:
  - Location: A URL the client should poll for a response status.
  - Retry-After: This header is designed to prevent polling clients from overwhelming the back-end with retries.

WebHooks

Webhooks

A webhook is an HTTP-based callback function that allows lightweight, event-driven communication between 2 application programming interfaces (APIs).

This callback is an HTTP POST that sends a message to a URL when an event happens.

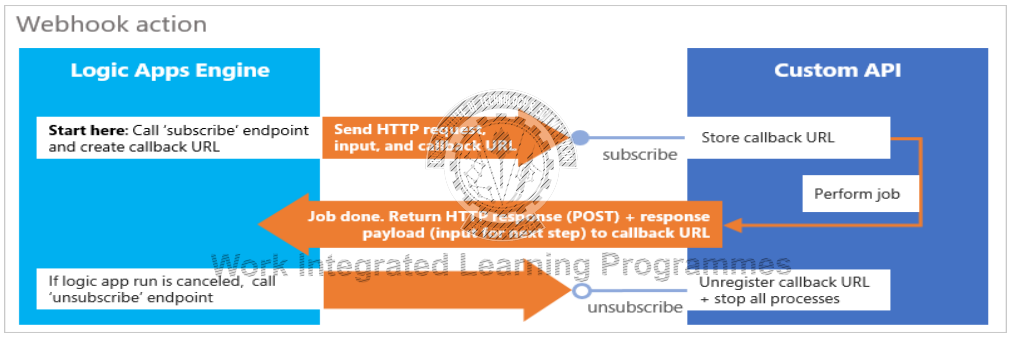
To set up a webhook, the client gives a unique URL to the server API and specifies which event it wants to know about.

Once the webhook is set up, the client no longer needs to poll the server;

When the specified event occurs, the server will automatically send the relevant payload to the client's webhook URL.

Webhooks are often called reverse APIs or push APIs because they put communication responsibility on the server rather than the client.

Webhooks



Webhooks

Eliminate the need for polling

Are quick to set up.

Automate data transfer.

Are good for lightweight, specific payloads.

Webhooks

client-side" application is the one making the request to the API on the "server-side".

The "client-side" must be running a server, and the "server-side" must be running a server.

The "client-side" application makes an API request to the "server-side" server, and sends the "server-side" server a "webhook" to call once the "server-side" wants to notify the "client-side" application of some "event".

Once the "event" occurs, and the "server-side" application calls the "webhook" url, the server that is running on the "client-side" application will "receive" that "webhook" notification.

Front end applications eg. pure React JS, AngularJS, Mobile Apps, cannot use webhooks directly.

Webhooks basically are APIs implemented by API consumers but defined and used by API providers to send notifications of events.

innovate achieve lead

BITS Pilani
Pilani Campus

Server Sent Events

Server-Sent Events

innovate achieve lead

Server-sent events (SSE) represent a unidirectional communication channel from the server to the client, allowing servers to push updates in real time.

Unlike other technologies like WebSockets, SSE operates over a single HTTP connection.

Key Features of SSE

Simplicity: Easy to implement and use.

Unidirectional Flow: Data flows from server to client only.

Automatic Reconnection: SSE connections automatically attempt to reconnect in case of interruptions.

Server-Sent Events

Establishing the SSE Connection

To initiate an SSE connection, the server sends a special text/event-stream MIME type response.

On the client side, the EventSource API is used to handle incoming events.

Client Side

To begin receiving events from the server , create a new EventSource object with the URL of a script that generates the events.

Create an EventSource object to establish the SSE connection

```
const eventSource = new EventSource('http://localhost:3000');
```

To receive message events, attach a handler for the message event:

```
eventSource.onmessage = (event) => {
    const data = JSON.parse(event.data);
document.getElementById('output').innerHTML = `Received data:
${data.message}`;
  };
```

This code listens for incoming message events and appends the message text to a list in the document's HTML.

Server Side

The server-side script that sends events must respond using the MIME type text/event-stream.

Each notification is sent as a block of text terminated by a pair of newlines.

Event stream format

data: {"message":"Server time: Thu Jan 18 2024 00:37:43 GMT+0530 (India Standard Time)"}

A pair of newline characters separate messages in the event stream.

Use Cases and Applications

Real-Time Notifications
Live Feeds and Dashboards
Collaborative Editing and Chat Applications
AI and LLM Text Streaming

Advantages

Simplicity and ease of use
Reduced network overhead
Automatic reconnection
Cross-domain support

Limitations: TCP connection is kept open, no bidirectional communication

WebSockets

WebSockets

innovate achieve lead

WebSocket is a communications protocol that supports bidirectional communication over a single TCP connection.

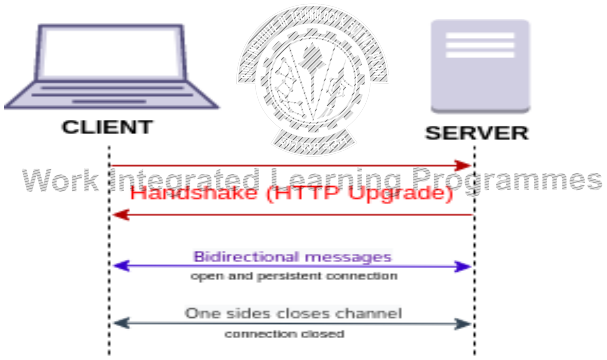
It is a living standard maintained by the WHATWG and a successor to The WebSocket API from the W3C.

The WebSocket protocol enables full-duplex interaction between a web browser (or other client application) and a web server

WebSockets Handshake

The WebSocket protocol specification defines ws (WebSocket) and wss (WebSocket Secure) as two new uniform resource identifier (URI) schemes.

ws : [// authority] path [? query]



https://commons.wikimedia.org/wiki/File:Websocket_connection.png#/media/File:Websocket_connection.png

WebSocket - Upgrade

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
Origin: http://example.com
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept:
HSmrc0sMlYUkAGmm5OPpG2HaGWk=
Sec-WebSocket-Protocol: chat
```

Headers

The Sec-WebSocket-Key header is calculated by base64 encoding a random string of 16 characters, each with an ASCII value between 32 and 127.

A different key must be randomly generated for each connection.

The Sec-Websocket-Accept header is included with the initial response from the server.

The server reads the Sec-WebSocket-Key, appends UUID 258EAFA5-E914-47DA-95CA- to it, re-encodes it using base64, and returns it in the response as the parameter for Sec-Websocket-Accept

The client sends the Sec-WebSocket-Protocol header to ask the server to use a specific subprotocol.

Why MCP chose SSE over WebSockets

The creators of MCP (Anthropic) specifically chose SSE for remote connections for a few practical reasons:

Firewall Friendly: SSE runs over standard HTTP/HTTPS (Port 80/443), which is less likely to be blocked by corporate firewalls compared to WebSockets.

Simplicity: SSE is easier to implement on the server side using standard web frameworks and doesn't require the complex "handshake" or persistent state management that WebSockets do.

Native HTTP Headers: Unlike WebSockets, standard HTTP requests allow for easy inclusion of authentication headers (like Bearer tokens), which is crucial for security

Several developers have built custom "WebSocket-to-MCP" bridges and servers

Summary

HTTP Polling
Webhooks
Server-Sent Events
Websockets

References

https://www.cloudflare.com/en-gb/learning/performance/http2-vs-http1.1/
https://ably.com/topic/http3

innovate achieve lead

BITS Pilani
Pilani Campus

Thank You!

BITS Pilani
Pilani Campus

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 7 REST

Module 4: Server Side: Implementing Web Services

innovate achieve lead

- REST
- Principles of REST
- REST constraints
- Service Design with REST
  - Interaction Design with HTTP
  - Interface Design (URI)
  - Representation and Metadata design
- Implementing REST API
  - Using a Framework
  - URL Mapping
  - Routing Requests-Redirection
  - Implementing a web server
  - Processing request, response, data
- Storing data in databases
  - Models
  - Object Relational Mapper
  - Interaction with DBs
- API versioning and documentation
-

REST Services

REST - Concepts

REST stands for REpresentational State Transfer
REST is an architectural style.

A RESTful web service

- ✓ exposes information about itself in the form of information about its resources
- ✓ enables the client to take actions or perform CRUD operations on those resources.

Resource

- ✓ Can be anything(docs, data, media, images) the web service can provide information about
- ✓ Each resource has a unique identifier - can be a name or a number

When a RESTful API is called, the server will transfer to the client a representation of the state of the requested resource!

REST - example

When a developer calls Instagram API to fetch a specific user (the resource)

- ✓ the API will return the state of that user, including
  - their name
  - the number of posts that user posted on Instagram so far
  - how many followers they have
  - and more

The representation of the state can be in a JSON format

- ✓ probably for most APIs this is indeed the case
- ✓ but can also be in XML or HTML format

Resources

The main building blocks of a REST architecture are the resources

Representations: It can be any way of representing data (binary, JSON, XML, etc.).

A single resource can have multiple representations.

Identifier: A URL that retrieves only one specific resource at any given time.

Metadata: Content type, last-modified time, and so forth.

Control data: Is-modifiable-since, cache-control.

REST and HTTP

The fundamental principle of REST is to use the **HTTP** protocol for data communication.

RESTful web service makes use of HTTP for determining the action to be carried out on the particular resources

REST gets its motivation from HTTP. Therefore, it can be said as a structural pillar of the REST

Commonly Used ones

- ✓ GET - to read (or retrieve) a representation of a resource
- ✓ POST - utilized to create new resources
- ✓ PUT - to update a resource
- ✓ PATCH - to modify a resource - only needs to contain the changes to the resource, not the complete resource
- ✓ DELETE - to delete a resource identified by a URI.

Rarely Used ones

- ✓ Head – requests the headers.
- ✓ Options - requests permitted communication options for a given URL or server

REST

Server responds based on the identifier and the operation

- ✓ An identifier for the resource you are interested in
  - URL for the resource, also known as the endpoint
  - URL stands for Uniform Resource Locator
  - A REST service exposes a URI for every piece of data the client might want to operate on.(Scoping Information)
- ✓ The operation you want the server to perform on that resource
  - in the form of an HTTP method, or verb
  - common HTTP methods are GET, POST, PUT, and DELETE
  - One way to convey method information in a web service is to put it in the HTTP method(Method Information)

Method Information

In a SOAP, REST, RPC
How the client can convey its intentions to the server?

REST Get Request

The response to an HTTP GET request for http://www.oreilly.com/index.html

```
HTTP/1.1 200 OK
Date: Fri, 17 Nov 2006 15:36:32 GMT
Server: Apache
Last-Modified: Fri, 17 Nov 2006 09:05:32 GMT
Etag: "7359b7-a7fa-455d8264
Accept-Ranges: bytes
Content-Length: 43302
Content-Type: text/html
X-Cache: MISS from www.oreilly.com
Keep-Alive: timeout=15, max=1000
Connection: Keep-Alive

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
...
<title>oreilly.com -- Welcome to O'Reilly Media, Inc.</title>
-----
```

An HTTP GET request for http://www.oreilly.com/index.html

```
GET /index.html HTTP/1.1
Host: www.oreilly.com
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US;
rv:1.7.12)...
Accept:
text/xml,application/xml,application/xhtml+xml,text/html
;q=0.9,...
Accept-Language: us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-15,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

Image Ref: Book T2: Restful Web services *Leonard Richardson and Sam Ruby*

SOAP Request

A sample SOAP RPC call

```
POST search/beta2 HTTP/1.1
Host: api.google.com
Content-Type: application/soap+xml
SOAPAction: urn:GoogleSearchAction
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Body>
<gs:doGoogleSearch xmlns:gs="urn:GoogleSearch">
<q>REST</q>
...
</gs:doGoogleSearch>
</soap:Body>
</soap:Envelope>
```

Part of the WSDL description for Google's search service

```
<operation name="doGoogleSearch">
<input message="typens:doGoogleSearch"/>
<output message="typens:doGoogleSearchResponse"/>
</operation>
```


XML-RPC Example

```
POST /rpc HTTP/1.1
Host: www.upcdatabase.com
User-Agent: XMLRPC::Client (Ruby 1.8.4)
Content-Type: text/xml; charset=utf-8
Content-Length: 158
Connection: keep-alive
<?xml version="1.0" ?>
<methodCall>
<methodName>lookupUPC</methodName>
...
</methodCall>
```

Example 1-12. An XML document describing an XML-RPC request

```
<?xml version="1.0" ?>
<methodCall>
<methodName>lookupUPC</methodName>
<params>
<param><value><string>001441000055</
string></value></param>
</params>
</methodCall>
```

Scoping Information

http://flickr.com/photos/tags/penguin

- *http://api.flickr.com/services/rest/?method=flickr.photos.search&tags=penguin*

http://www.upcdatabase.com/upc/00598491'

One obvious place to put it is in the URI path.

A RESTful, resource-oriented service exposes a URI for every piece of data the client might want to operate on.

Method and Scoping information

In RESTful web service,

- ✓ the method information goes into the HTTP method.
- ✓ The scoping information goes into the URI.

Given the first line of an HTTP request to a RESTful web service you should understand basically what the client wants to do.

"GET /reports/docs HTTP/1.1"

If the HTTP method doesn't match the method information, the service isn't RESTful.

The service is not resource-oriented if the scoping information isn't in the URI.

Key principles

Everything is a resource
Each resource is identifiable by a unique identifier (URI)
Use the standard HTTP methods
Resources can have multiple representations
Communicate statelessly

REST Constraints

For a web service to be RESTful, it has to adhere to 6 constraints:

- ✓ Client - Server separation
- ✓ Stateless
- ✓ Cacheable
- ✓ Uniform interface
- ✓ Layered system
- ✓ Code-on-demand (Optional)

Client - Server separation

The client and the server act independently, each on its own

- ✓ Interaction between them is only in the form of
  - requests initiated by the client only
  - responses, which the server send to the client only as a reaction to a request

The server sits there waiting for requests from the client to come

- ✓ doesn't start sending away information about the state of some resources on its own
- ✓ Responds only when a request comes in

Stateless

Stateless means the server does not remember anything about the user who uses the service

- ✓ doesn't remember if the user already sent a GET request for the same resource in the past
- ✓ doesn't remember which resources the user of the API requested before
- ✓ and so on...

Each individual request contains all the information the server needs to perform the request and return a response, regardless of other requests made by the same user.

The client is responsible for sending any state information to the server whenever it's needed.

No session stickiness or session affinity on the server for the calling request

The statelessness constraint helps services to be more scalable and reliable.

Cacheable

Data the server sends contain information about whether or not the data is cacheable

If the data is cacheable, it might contain some version number

- ✓ version number is what makes caching possible

The client knows which version of the data it already has (from a previous response)

- ✓ the client can avoid requesting the same data again and again

client should also know if the current version of the data is expired,

- ✓ will know it should send another request to the server to get the most updated data about the state of a resource

Uniform interface

There are four guiding principles suggested by Fielding that constitute the necessary constraints to satisfy the uniform interface

- Identification of resources
- Manipulation of resources
- Self-descriptive messages
- Hypermedia as the engine of application state

The request to the server has to include a resource identifier

Each request to the web service contains all the information the server needs to perform the request

- ✓ Each response the server returns contain all the information the client needs to understand the response

The response the server returns include enough information so the client can manipulate the resource

Hypermedia as the engine of application state

- ✓ Application mean the web application that the server is running
- ✓ Hypermedia mean the hyperlinks, or simply links, that the server can include in the response
- ✓ means that the server can inform the client , in a response, of the ways to change the state of the web application

HATEOS

Hypermedia as the Engine of Application State

Without HATEOAS:

```
Request:
[Headers]
user: jim
roles: USER
GET: /items/1234
Response:
HTTP 1.1 200
{
    “id” : 1234,
    “description” : “FooBar TV”,
    “image” : “fooBarTv.jpg”,
    “price” : 50.00,
    “owner” : “jim”
}
```

With HATEOAS:

```
Request:
[Headers]
user: jim
roles: USER
GET: /items/1234
Response:
HTTP 1.1 200
{
    “id” : 1234,
    “description” : “FooBar TV”,
    “image” : “fooBarTv.jpg”,
    “price” : 50.00,
    “links” : [
                “rel” : “modify”,
                “href” : “/items/1234”
            },
            {
                “rel” : “delete”,
                “href” : “/items/1234”
            }
        ]
    }
}
```


Layered system

Between the client who requests a representation of a resource’s state, and the server who sends the response back

- ✓ there might be a number of servers in the middle
- ✓ servers might provide a security layer, a caching layer, a load-balancing layer etc.,
- ✓ layers should not affect the request or the response

The client is agnostic as to how many layers, if any, there are between the client and the actual server responding to the request.

Code-on-demand (Optional)

Is optional — a web service can be RESTful even without providing code on demand

The client can request code from the server

- ✓ response from the server will contain some code
- ✓ when the response is in HTML format, usually in the form of a script

The client then can execute that code

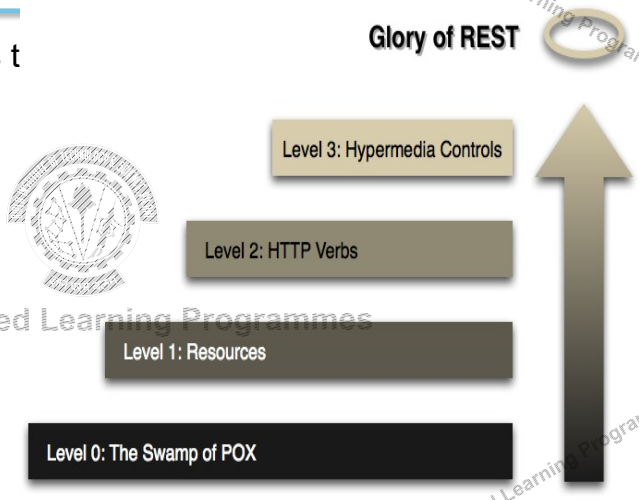
REST Maturity Model

Richardson used three main factors t
factors are

URI,

HTTP Methods,

HATEOAS (Hypermedia)



innovate achieve lead

BITS Pilani
Pilani Campus

REST API DESIGN

REST API Design

innovate achieve lead

API design is more of an art.

Some best practices for REST API design are implicit in the HTTP standard.

- When should URI path segments be named with plural nouns?
- Which request method should be used to update the resource state?
- How do I map *non-CRUD* operations to my URIs?
- What is the appropriate HTTP response status code for a given scenario?
- How can I manage the versions of a resource's state representations?
- How should I structure a hyperlink in JSON?

API Design

REST APIs are designed around *resources* , which is any kind of object, data, or service that is accessible to the client.

A resource has an *identifier* , a URI that uniquely identifies that resource.

The resource URIs should be based on nouns (the resource) and not verbs (the operations on the resource).

API Design Elements

The following aspects of API design are all important, and together they define your API:

- The representations of your resources and the links to related resources.
- The use of standard (and occasionally custom) HTTP headers.
- The URLs and URI templates define your API's query interface for locating resources based on their data.
- Required behaviors by clients—for example, DNS caching behaviors, retry behaviors

- Interaction Design with HTTP(response codes, request methods)
- Identifier Design with URIs
- Metadata Design(Media Types, content negotiation)
- Representation Design(Message body format, Hypermedia Representation)

Interaction Design

Interaction Design with HTTP

REST APIs embrace all aspects of the HyperText Transfer Protocol including its request methods, response codes, and message headers

Each HTTP method has specific, well-defined semantics within the context of a REST API's resource model.

The purpose of GET is to retrieve a representation of a resource's state.

HEAD is used to retrieve the metadata associated with the resource's state.

PUT should be used to add a new resource to a store or update a resource.

DELETE removes a resource from its parent.

POST should be used to create a new resource within a collection and execute controllers.

Interaction Design

Interaction Design with HTTP

Rule: GET and POST must not be used to tunnel other request methods

Rule: GET must be used to retrieve a representation of a resource

Rule: HEAD should be used to retrieve response headers

Rule: PUT must be used to both insert a stored resource

Rule: POST must be used to create a new resource in a collection

Rule: POST must be used to execute controllers

Rule: DELETE must be used to remove a resource from its parent

Rule: OPTIONS should be used to retrieve metadata that describes a resource's available interactions

Interaction Design

HTTP response success code summary

200 OK Indicates a nonspecific success

201 Created Sent primarily by collections and stores but sometimes also by controllers to indicate that a new resource has been created

204 No Content Indicates that the body has been intentionally left blank

301 Moved Permanently Indicates that a new *permanent* URI has been assigned to the client's requested resource

303 See Other Sent by controllers to return results that it considers optional

401 Unauthorized Sent when the client either provided invalid credentials or forgot to send them

402 Forbidden Sent to deny access to a protected resource

404 Not Found Sent when the client tried to interact with a URI that the REST API could not map to a resource

405 Method Not Allowed Sent when the client tried to interact using an unsupported HTTP method

406 Not Acceptable Sent when the client tried to request data in an unsupported media type format

500 Internal Server Error Tells the client that the API is having problems of its own

Identifier Design

REST APIs use Uniform Resource Identifiers (URIs) to address resources

URI Format

URI = scheme "://" authority "/" path ["?" query] ["#" fragment]

Forward slash separator (/) must be used to indicate a hierarchical relationship.

Consistent subdomain names should be used for your APIs

Identifier Design

Resource Modelling

The URI path conveys a REST API's resource model,

Each forward slash-separated path segment corresponds to a unique resource within the model's hierarchy.

http://api.library.restapi.org/books/harry-potter

http://api.library.restapi.org/books

http://api.library.restapi.org/sections/fiction

http://api.library.restapi.org/sections

Identifier Design

document,

http://api.library.restapi.org/books/Pride and Prejudice

http://api.library.restapi.org/books/harry-potter/harry potter and the philosopher's stone

collection,

Each URI below identifies a collection resource:

http://api.library.restapi.org/books

http://api.library.restapi.org/sections

controller.

POST /alerts/245743/resend

Few antipatterns

GET /deleteUser?id=1234

GET /deleteUser/1234

DELETE /deleteUser/1234

POST /users/1234/delete

Identifier Design

Rules

- A singular noun should be used for document names
- A plural noun should be used for collection names
- A verb or verb phrase should be used for controller names
- Variable path segments may be substituted with identity-based values http://api.library.restapi.org/section/{sectionId}/books/{bookId}/version/{versiond}
- CRUD function names should not be used in URIs

Identifier Design

URI Query Design

As a component of a URI, the query contributes to the unique identification of a resource.

Rule: The query component of a URI may be used to filter collections

Rule: The query component of a URI should be used to paginate collections

Represent relationships clearly in the URI

Filter and Paginate

/orders?minCost=n

/orders?limit=25&offset=50

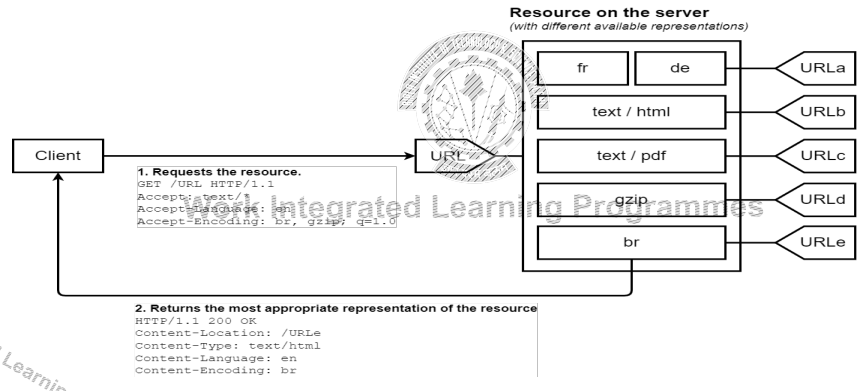
Metadata Design

Important HTTP headers

- Content type
- Content length
- Last-modified
- E-tag
- Location

Metadata Design

Media type negotiation should be supported when multiple representations are available



Representation Design

Representation is the technical term for the data that is returned

Users can view the representation of a resource in different formats, called media types

JSON is the preferred format

XML and other formats may optionally be used for resource representation

A consistent form should be used to represent media types, links, and errors

Representation Design

Keep the choice of media types and formats flexible to allow for varying application use cases, and client needs for each resource.

Prefer to use well-known media types for representations.

What data to include in JSON-formatted representations

An example of a representation of a person resource:

```
{
"name" : "John",
"id" : "urn:example:user:1234",
"link" : {
"rel" : "self",
"href" : "http://www.example.org/person/john"
},
"address" : {
"id" : "urn:example:address:4567",
"link" : {
"rel" : "self",
"href" : "http://www.example.org/person/john/address"
}
```


Best Practices - Summary

Use only nouns for a URI;

GET methods should not alter the state of resource;

Use plural nouns for a URI;

Use sub-resources for relationships between resources;

Use HTTP headers to specify input/output format;

Provide users with filtering and paging for collections;

Version the API;

Provide proper HTTP status codes.

Best Practices

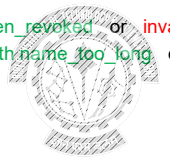
Consistent API s - reduces the cognitive load on developers.

Meaningful errors

Example:

Authentication failed because token is revoked : token_revoked or invalid_auth

Value passed for name exceeded max: length_name_too_long or invalid_name



API Design

"*The API should enable developers to do one thing really well.It's not as easy as it sounds, and you want to be clear on what the API is not going to do as well.*"

- Ido Green, developer advocate at Google

RESTful Services Example - Customer

HTTP Verb	CRUD	Entire Collection (e.g. /customers)	Specific Item (e.g. /customers/{id})
POST	Create	201 (Created), 'Location' header with link to /customers/containing new ID.	404 (Not Found), 409 (Conflict) if resource already exists..
GET	Read	200 (OK). list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer. 404 (Not Found), if ID not found or invalid.
PUT	Update/Replace	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
PATCH	Update/Modify	405 (Method Not Allowed), unless you want to modify the collection itself.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

Processing a Refund- Resource Realization

Processing a Refund
Step 1: Checking the Resource (GET) GET /v1/orders/99
Step 2: The Action: create a new resource POST /v1/refunds
Step 3: Process the refund POST /v1/refunds/re-552/process
Step 4: Repeat Step 1 to Check

Treat Refund as a Resource!!
Other Similar: Transfer

Non CRUD operation

If you need to perform non crud operation on a resource, you can also create it as a subresource

Example:
Resend
Reboot, Resize
archive

Nesting

Use Nesting for Collections If you want to find all items belonging to a parent, nesting is very intuitive.

GET /v1/posts/2/comments (Find all comments for this specific post).

GET /v1/users/1/orders (Find all orders for this specific user).

Use Flattening for Individual Items Once you have the ID of the item, stop nesting.

GET /v1/comments/3 (Get details for this comment).

PATCH /v1/orders/99 (Update this specific order).

Include the "parent" information in the **Response Body**, not the **URI**.

Never go deeper than two levels of nouns

References

RESTful Web APIs by Leonard Richardson and Mike Amundsen , Oreilly Media, 2013

Web Development with Node and Express by Ethan Brown , Oreilly Media 2nd Edition , 2019

Thank You!

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani
Pilani Campus

innovate achieve lead

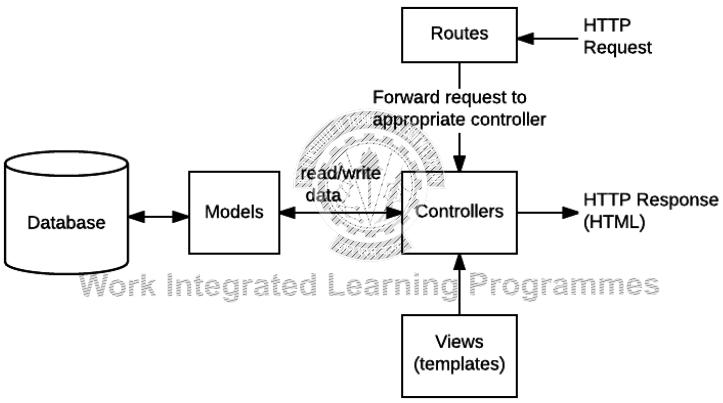
BITS Pilani
Pilani Campus

Lecture No: 8 REST Design

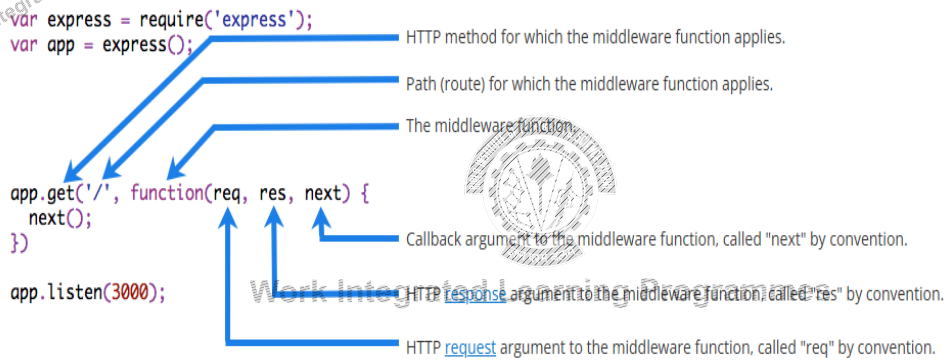
Module 4: Server Side: Implementing Web Services

- REST
- Principles of REST
- REST constraints
- Service Design with REST
  - Interaction Design with HTTP
  - Interface Design (URI)
  - Representation and Metadata design
- Implementing REST API
  - Using a Framework
  - URL Mapping
  - Routing Requests-Redirection
  - Implementing a web server
  - Processing request, response, data
- Storing data in databases
  - Models
  - Object Relational Mapper
  - Interaction with DBs
- API versioning and documentation
-

Express Framework Application



Express Framework Application



API Versioning

Breaking changes primarily fit into the following categories:

- Changing the request/response format (e.g. from XML to JSON)
- Changing a property name (e.g. from name to productName) or data type on a property (e.g. from an integer to a float)
- Adding a required field on the request (e.g. a new required header or property in a request body)
- Removing a property on the response (e.g. removing description from a product)

API Change Management

Effective change management in the context of an API is summarized by the following principles:

Continue support for existing properties/endpoints

Add new properties/endpoints rather than changing existing ones

Thoughtfully sunset obsolete properties/endpoints

API Change Management

We can think of levels of scope change within a tree analogy:

Leaf - A change to an isolated endpoint with no relationship to other endpoints

Branch - A change to a group of endpoints or a resource accessed through several endpoints

Trunk - An application-level change, warranting a version change on most or all endpoints

Root - A change affecting access to all API resources of all versions

As you can see, moving from leaf to root, the changes become progressively more impactful and global in scope.

API versioning representation

There are four different ways that we can implement the versioning

Versioning through the URI path

- http://www.example.org/v1/customer/1234
- Ideal when major changes are introduced that completely break backward compatibility.

Versioning through query parameters

- http://www.example.org/customer/1234?version=v3
- Ideal for simple APIs where backward compatibility is maintained across versions.

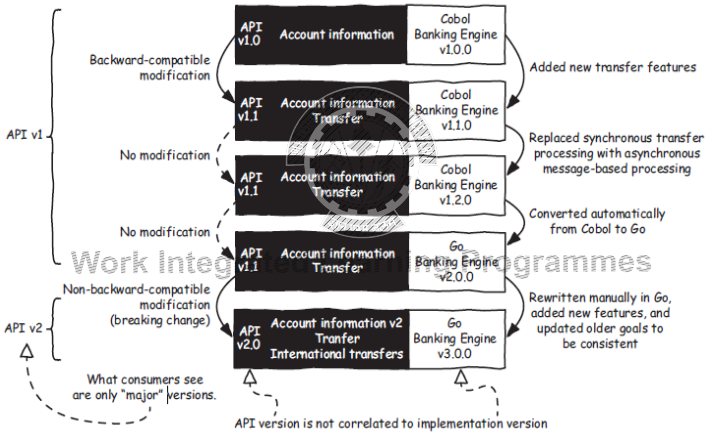
Versioning through custom headers

- Custom-Header: api-version=1

Versioning through content-negotiation

- Accept: application/vnd.adventure-works.v1+json

API Versioning



Semantic Versioning

format: *MAJOR.MINOR.PATCH*.

The MAJOR digit is incremented only on breaking changes, such as adding a new mandatory parameter

The MINOR digit is incremented when new features are added in a backward-compatible manner, like adding new HTTP methods or resource paths in a REST API.

The PATCH digit is incremented when the modifications made involve backward-compatible bug

This makes sense for an implementation, but not for an API.

Semantic versioning applied to APIs consist of just two digits: *BREAKING.NONBREAKING*.

This two-level versioning is interesting from the provider's perspective; it helps to keep track of all the different backward-compatible and non-backward-compatible versions of an API.

API Documentation

API documentation is a technical content deliverable containing instructions on using and integrating with an API effectively.

API description formats like the OpenAPI/Swagger Specification have automated the documentation process, making it easier for teams to generate and maintain them.

OpenAPI Specification (formerly Swagger Specification) is an API description format for REST APIs.

OAS

The *OpenAPI Specication* (OAS) is a popular REST API description format

An OpenAPI file allows you to describe your entire API, including:

- Available endpoints (/users) and operations on each endpoint (GET /users, POST /users)
- Operation parameters Input and output for each operation
- Authentication methods
- Contact information, license, terms of use and other information.
- API specifications can be written in YAML or JSON

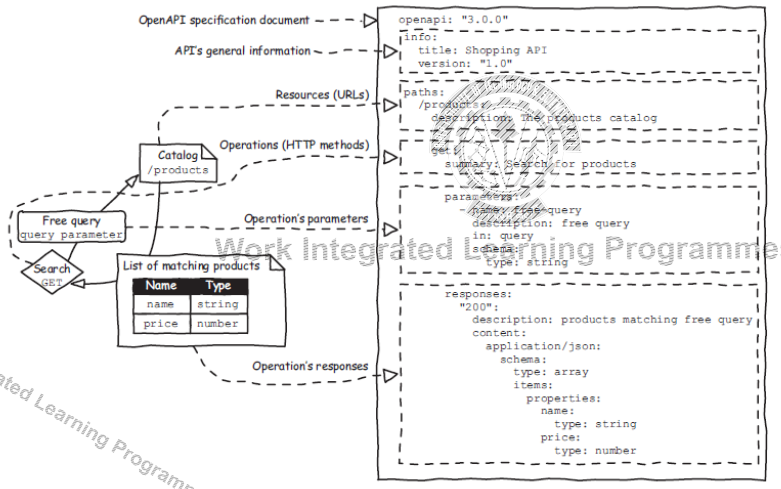
Swagger

Swagger is a set of open-source tools built around the OpenAPI Specification that can help you design, build, document and consume REST APIs.

http://swagger.io/docs/specification/basic-structure/

API Documentation

An OAS document describing the search for products goal of the Shopping API



API Documentation

RAML and Blueprint are other alternatives

API Blueprint is a markdown format to generate documentation. It was developed by Apiary in 2013 and then acquired by Oracle.

RAML, which stands for RESTful API Modeling Language, is an API design format developed by MuleSoft in 2013 and then acquired by Salesforce.

Swagger is an API description format developed by Wordnik in 2010 and then acquired by SmartBear. It was later renamed OpenAPI and donated to the Linux Foundation.

References

RESTful Web APIs by Leonard Richardson and Mike Amundsen , Oreilly Media, 2013

Web Development with Node and Express by Ethan Brown , Oreilly Media 2nd Edition , 2019

Thank You!

Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 9 GraphQL

GraphQL

GraphQL is

- ✓ a query language for your APIs and
- ✓ a server-side runtime for executing queries.

A GraphQL service is created by defining types, fields on those types and functions for each field

A GraphQL query asks only for the data that it needs.

GraphQL

GraphQL is typically served over HTTP via a single endpoint which expresses the full set of capabilities of the service.

GraphQL services typically respond using JSON,

REST- disadvantages

Drawbacks

- Overfetching
- Underfetching

REST APIs need more flexibility.

As the client needs change, new endpoints have to be created.

Eg: to fetch the list of books

GraphQL

A GraphQL operation is either

- a query (read),
- mutation (write), or
- Subscription (continuous read).

This GraphQL operation is interpreted against the GraphQL schema at the backend, and resolved with data for the frontend application.

After a GraphQL service runs , it can receive GraphQL queries to validate and execute.

The service first checks a query to ensure it only refers to the types and fields defined, and then runs the provided functions to produce a result.

GraphQL- Queries

GraphQL Query:

```
query {
  posts {
    title
        author {
      name
      email
    }
  }
}
```

GraphQL Response:

```
{
  "data": {
    "posts": [
      {
        "title": "Introduction to GraphQL",
        "author": {
          "name": "John Doe",
          "email": "john.doe@example.com"
        }
      },
      {
        "title": "Getting Started with React",
        "author": {
          "name": "Jane Smith",
          "email": "jane.smith@example.com"
        }
      }
      // Additional posts...
    ]
  }
}
```


Queries

GraphQL queries can traverse related objects and their fields, letting clients fetch lots of related data in one request, instead of making several roundtrips as one would need in a classic REST architecture.

Queries- Arguments and Variables

Every field and nested object can get its own set of arguments.
Arguments can be of many different types.
The arguments to fields can be dynamic
Replace the static value in the query with $variableName
Declare $variableName as one of the variables accepted by the query
Pass variableName: value in the separate, transport-specific (usually JSON) variables dictionary

```
{
  "personid": "1000"
}

human(id: $personid)
```


Queries- Arguments and Variables

Query:

```
{
 human(id: "1000") {
    name
    height(unit: FOOT)
  }
}
```

Response:

```
{
  "data": {
    "human": {
      "name": "Luke Skywalker",
      "height": 5.6430448
    }
  }
}
```

Queries

Default values
Directives
Reusable units called *fragments*

Mutations

Mutations are used to modify data on the server.
To write new data, we use *mutations*.
Mutations are defined like queries.
The Mutation is a root object type.


```
mutation {
  addUser(name: "Alice", age: 30) {
    id
    name
    age
  }
}
```

Schemas and Types

Every GraphQL service defines a set of types describing the possible data you can query for that service.

Then, when queries come in, they are validated and executed against that schema.

Schema

Schema define the data types that your API will expose.

GraphQL APIs are defined by a schema, which serves as a contract between the client and the server.

The schema specifies the types of data that can be queried and the relationships between them.

GraphQL schema documents are text documents that define the types available in an application

Types

The core unit of any GraphQL Schema is the type.
These types represent the structure of the data available in the API.
A type has *fields* that represent the data associated with each object.
Each field returns a specific type of data.
A schema is a collection of type definitions

```
type User {
    id: ID!
    name: String!
    age: Int!
  }
```


Schema and Types

Two types are special within a schema
Query and
Mutation
Every GraphQL service has a query type and may or may not have a mutation type.
They are special because they define the *entry point* of every GraphQL query

```
const schema = buildSchema(`
  type User {
    id: ID!
    name: String!
    age: Int!
  }

  type Query {
    getUser(id: ID!): User
    getUsers: [User]
  }
  type Mutation {
    addUser(name: String!, age: Int!): User
  }
`);
```

Resolvers

Resolvers are functions that determine how data is retrieved or modified.

They map to the types and fields defined in the schema.

Resolver functions return data in the type and shape specified by the schema.

Resolvers can fetch or update data from a REST API, database, or anyother service.

GraphQL Service

A GraphQL service can be written in any programming language,

It is conceptually split into two major parts,

- structure and
- Behavior

The structure is defined with a strongly typed schema.

The behavior is naturally implemented with functions and are called resolver functions.

Graphs

In GraphQL, the foundation is a graph-based approach to modeling business domain.

By employing GraphQL, the business domain is constructed as a graph,

Graphs are used formally to represent a collection of interconnected objects.

Schema outlines different types of nodes and their connections or relationships.

Examples

Facebook is an undirected graph.

Think of your GraphQL schema as an expressive shared language for your development team and end-users.

Creating a robust schema involves examining the everyday language used to describe your business.

GraphQL language

At the core of a GraphQL communication is a request object.

The source text of a GraphQL request is often referred to as a document.

A document contains text that represents a request through operations like queries, mutations, and subscriptions.

In addition to the main operations, a GraphQL document text can contain fragments that can be used to compose other operations.

Summary

GraphQL

Schemas and Types

Queries and Mutations

Resolvers

innovate achieve lead

BITS Pilani
Pilani Campus

Thank you

BITS Pilani
Pilani Campus

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

Lecture No: 9 gRPC

gRPC- An RPC library and framework

Problems with RPC- laid the motivation for gRPC

In 2015, Google released gRPC , a modern, open source, high-performance remote procedure call (RPC) framework that can run anywhere.

An interprocess communication technology that allows you to connect, invoke, operate, and debug distributed heterogeneous applications as easily as making a local function call.

gRPC

What does the "g" in gRPC actually stand for?

- Its not Google
- Google changes the meaning of the "g" for each version
- Refer to the README doc

gRPC- Architecture

- A service Definition is created.
- Using that service definition, the server-side code known as a *server skeleton* is generated
- On the server side, the server implements the service, and runs a gRPC server to handle client calls.
- On the client side, the client has a stub that provides the same methods as the server.
- gRPC uses **Protocol Buffers** as
  - Interface Definition Language (**IDL**) and
  - message interchange format.

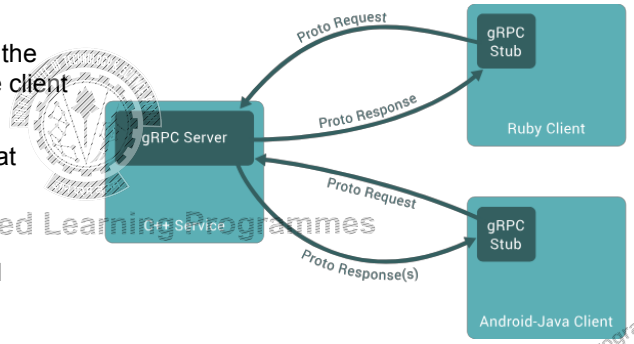


Image Source: https://grpc.io/docs/what-is-grpc/introduction/

Protocol Buffers

gRPC uses protocol buffers as the Interface Definition Language to define the service interface

Protocol Buffers are a language-neutral, platform-neutral extensible mechanism for serializing structured data.

The service interface definition is specified in a proto file -an ordinary text file with a .proto extension.

The service definition specified in the .proto file, is used by both the server and client sides to generate the code.

Protocol Buffers

Protocol buffer data is structured as messages.

Each message is a small logical record of information containing a series of name-value pairs called fields.

Example:

```
message Person {
  string name = 1;
  int id = 2;
  string description = 3;
}
```

Protocol Buffers

You define gRPC services in ordinary proto files, with RPC method parameters and return types specified as protocol buffer messages.

Example:

```
service ProductInfo {
   rpc addProduct(Product) returns (ProductID);
   rpc getProduct(ProductID) returns (Product);
}
message Product {
   string id = 1;
   string name = 2;
   string description = 3;
}
message ProductID {
   string value = 1;
}
```


gRPC Service

The service definition can be used to generate the server or client-side code using the protocol buffer compiler protoc.

Since gRPC service definitions are language agnostic, you can generate clients and servers for any supported language

gRPC Service

Server Side:

- Implement the service logic of the generated service skeleton by overriding the service base class.
- Run a gRPC server to listen for requests from clients and return the service responses.

Client Side:

- The client stub provides the same methods as the server, which your client code can invoke.
- The client stub translates them to remote function invocation network calls that go to the server side.

Why gRPC?

Efficient for inter-process communication

- gRPC implements protocol buffers on top of HTTP/2

Simple, well-defined service interfaces and schema

- contract-first approach

Strongly typed- protocol buffers to define gRPC services

Polyglot

Duplex streaming

Built-in commodity features

gRPC in Real World

With the adoption of gRPC, Netflix has seen a massive boost in developer productivity.

Etcd uses a gRPC user-facing API to leverage the full power of gRPC.

Dropbox has switched to gRPC

gRPC- Disadvantages

It may not be suitable for external-facing services.

Drastic service definition changes are a complicated development process

The ecosystem is still evolving.

gRPC- Flow

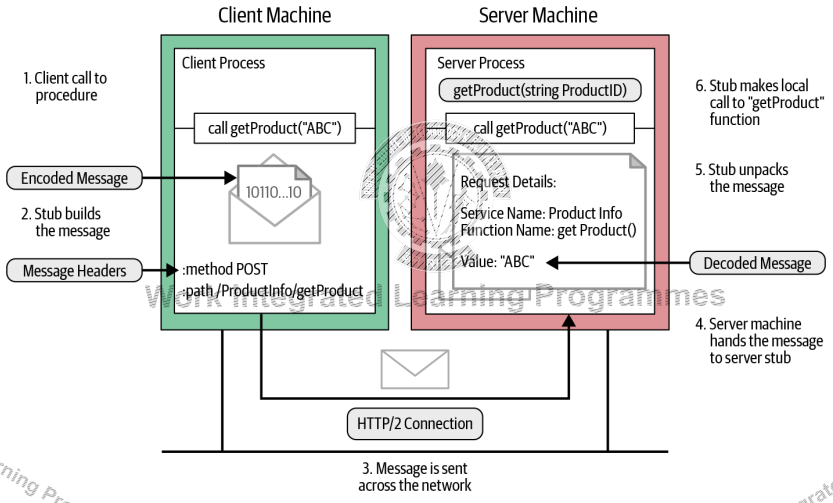


Image Source: gRPC: Up and Running by Kasun Indrasiri, Danesh Kuruppu

Message Encoding

message ProductID {
string value = 1;
}
Let's say we need to get product details for product ID 15:

Message Field

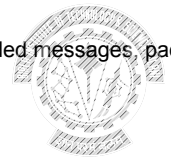
Tag	Value

Message Encoding

- Tag value = (field_index << 3) | wire_type

Wire type

- 0 for Varint int32, int64, uint32, uint64, sint32, sint64, bool, enum
- 1 for 64-bit fixed64, sfixed64, double
- 2 for Length-delimited string, bytes, embedded messages, packed repeated fields
- 3 for Start group groups (deprecated)
- 4 for End group groups (deprecated)
- 5 for 32-bit fixed32, sfixed32, float



Tag value = (00000001 << 3) | 00000010

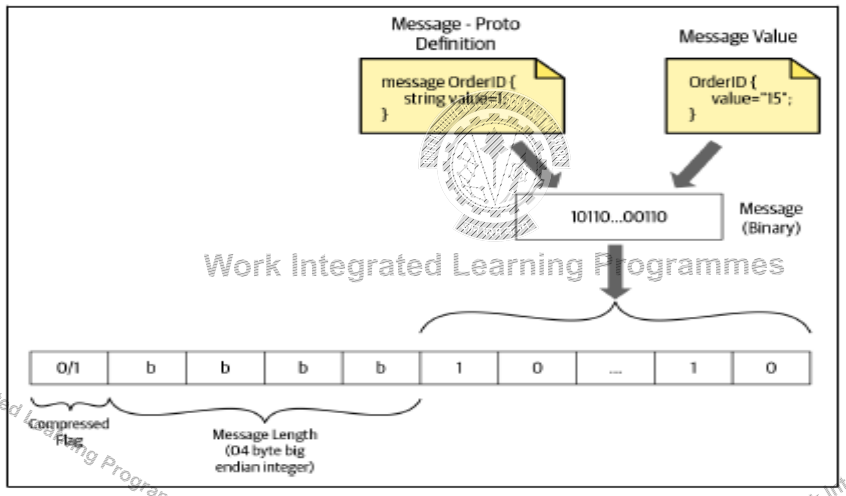
= 000 1010

Length-Prefixed Message Framing

gRPC uses a message-framing technique called length-prefix framing.

Length-prefix is a message-framing approach that writes the size of each message before writing the message itself.

Length Prefixed Messaging



gRPC over HTTP/2

gRPC uses HTTP/2 as its transport protocol to send messages over the network.

This is one of the reasons why gRPC is a high-performance RPC framework.

The gRPC channel represents a connection to an endpoint, which is an HTTP/2 connection.

Messages that are sent in the remote call are sent as HTTP/2 frames.

A frame may carry one gRPC length-prefixed message, or if a gRPC message is quite large it might span multiple data frames.

Communication patterns

The four fundamental communication patterns used in gRPC-based applications are

- unary RPC (simple RPC),
- server-side streaming,
- client-side streaming, and
- bidirectional streaming

Simple RPC (Unary RPC)

In simple RPC, the client invokes a remote function of a server.

The client sends a single request to the server and gets a single response that is sent along with status details and trailing metadata.

Example:

OrderManagement service for an online retail application

```
rpc getOrder(google.protobuf.StringValue) returns (Order);
    message Order {
    string id = 1;
    repeated string items = 2;
    string description = 3;
    float price = 4;
    }
```

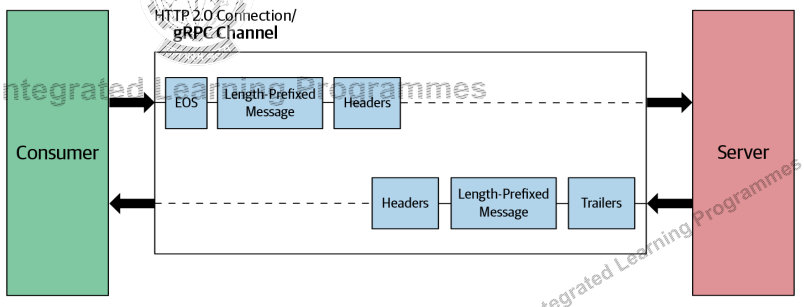


Image Source: gRPC: Up and Running by Kasun Indrasiri, Danesh Kuruppu

Server-Streaming RPC

In server-side streaming RPC, the server sends back a sequence of responses after getting the client's request message.
This sequence of multiple responses is known as a "stream."
After sending all the server responses, the server marks the end of the stream by sending the server's status details as trailing metadata to the client.
Example: searchOrder method of the Order Management service.
rpc searchOrders(google.protobuf.StringValue) returns (stream Order);

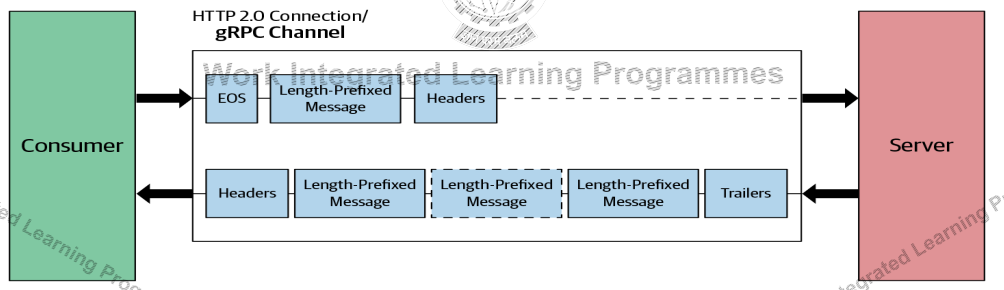


Image Source: gRPC: Up and Running by Kasun Indrasiri, Danesh Kuruppu

Client-Streaming RPC

In client-streaming RPC, the client sends multiple messages to the server instead of a single request.
The server sends back a single response to the client.
However, the server does not necessarily have to wait until it receives all the messages from the client side to send a response.
Example: updateOrders method of the Order Management service.
rpc updateOrders(stream Order) **returns** (google.protobuf.StringValue);

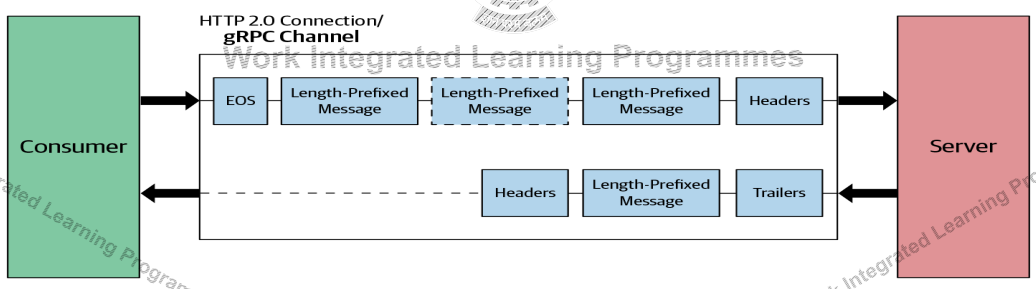


Image Source: gRPC: Up and Running by Kasun Indrasiri, Danesh Kuruppu

Bidirectional-streaming RPC

In bidirectional-streaming RPC, the client is sending a request to the server as a stream of messages.
The server also responds with a stream of messages.
The call has to be initiated from the client side,
Example: updateOrders method of the Order Management service.
rpc processOrders(stream google.protobuf.StringValue) returns (stream CombinedShipment);

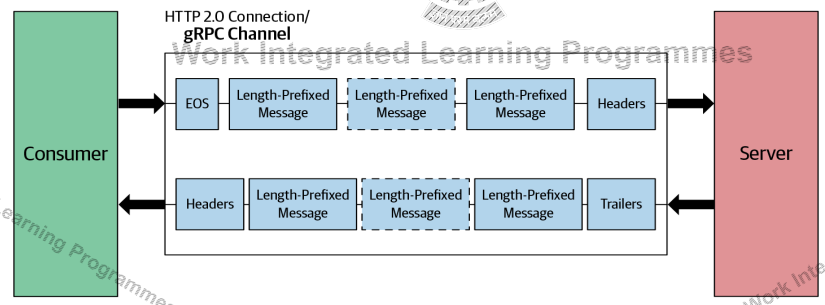


Image Source: gRPC: Up and Running by Kasun Indrasiri, Danesh Kuruppu

Summary

- gRPC
- Protobuffers
- gRPC Communication Patterns

References

BOOK: gRPC: Up and Running by Kasun Indrasiri and Danesh Kuruppu

BITS Pilani
Pilani Campus

Thank You!

Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 10 Understanding Frontend Development

Frontend technologies

What happens when you load your webpage in the browser?

The code (the HTML, CSS, and JavaScript) are running inside an execution environment (the browser).

HTML is the markup language that we use to structure and give meaning to our web content.

CSS is a language of style rules that we use to apply styling to our HTML content, for example, setting background colors and fonts and laying out our content in multiple columns.

JavaScript is a scripting language that enables you to create dynamically updating content and adds interactivity to your webpage.

Javascript

JavaScript is the programming language of the web.

JavaScript is a programming language that adds interactivity to websites.

JavaScript is a single-threaded interpreted language.

Every browser has its own JavaScript engine. Google Chrome has the V8 engine, Mozilla Firefox has SpiderMonkey.

The core client-side JavaScript language consists of some common programming features like variables, objects, functions etc.,

Host Environment

Browsers- Initially only implemented in web browsers
Now it can be used on Server Side

Client Side(Browser)	Server Side(NodeJS)
• Different browsers provide their own JS engines • Allows interaction with web page(HTML and CSS) • Interact with browser API s (History, Location)	• Google' s V8 was extracted to run anywhere- Node.js • Node is a fast C++-based JavaScript interpreter • Work with file systems, Web servers • Knowledge Reuse

JavaScript implementations

Mozilla's SpiderMonkey is used in Firefox. Other non-browser usage includes MongoDB, CouchDB, and more. This was the first ever JavaScript engine, created by Brendan Eich at Netscape.

Google's V8, used in Chrome and Chromium-based browsers such as Opera. Other non-browser usage includes Node.js, Deno, Electron, and more.

Apple's JavaScriptCore (also known as SquirrelFish/Nitro), used in Safari and other WebKit-based browsers.

Execution in Javascript

JavaScript is a single-threaded programming language.

An Execution Context is an abstract concept of an environment where the JavaScript code is evaluated and executed.

JavaScript execution requires the cooperation of two pieces of software: the JavaScript engine and the host environment.

Execution in Javascript

The JavaScript Engine consists of two main components:

Memory Heap: this is where the memory allocation happens

Call Stack: this is where your stack frames are as your code executes. As its name implies, the call stack has a LIFO (Last in, First out) structure, which stores all the execution context created during the code execution.

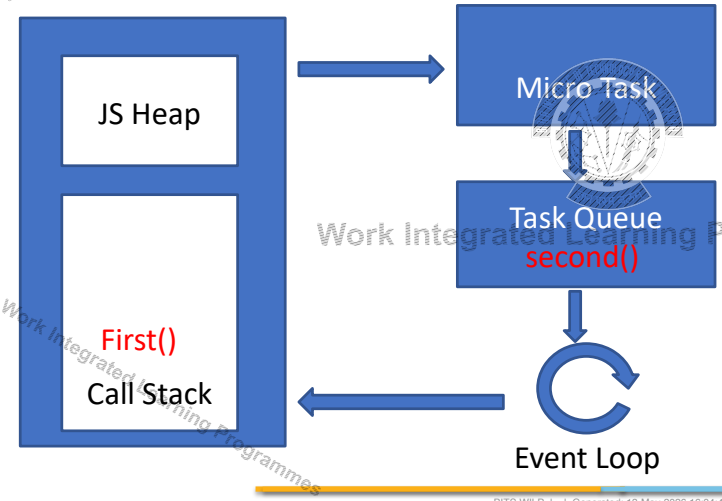
JavaScript Execution in browser

JavaScript runs single-threaded with an event loop.

- Call Stack executes synchronous code.
- Async tasks (e.g., setTimeout, fetch) are handled by the browser.
- Their callbacks are queued:
- Microtask queue (Promises)
- Macrotask queue (Timers, I/O)
- Event Loop executes queued callbacks once the stack is empty.

JavaScript Execution in browser

Asynchronous Execution




```
<script>
    const second = () => {
      console.log('Hello there!');
    }
    const first = () => {
      console.log('Hi there!');
      setTimeout(second, 1000);
      console.log('The End');
    }
    first();
</script>
```

JavaScript Execution in browser

Feature	Microtask (Promise)	Macrotask (setTimeout)
Priority	High	Lower
Runs	Right after current synchronous code	After all microtasks finish
Typical Use	Promise.then, queue Microtask	setTimeout, I/O, DOM events

Visualization

https://www.jsv9000.app/

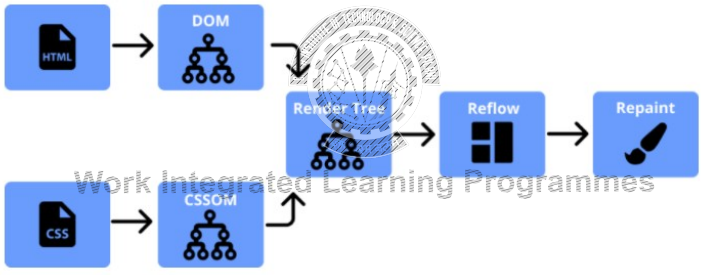
Web Page Rendering

When a browser loads a web page, two major processes happen in sequence/parallel but are carefully coordinated

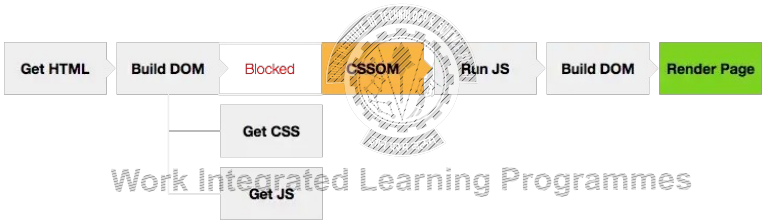
Rendering pipeline: Converts HTML, CSS, JS into pixels on screen

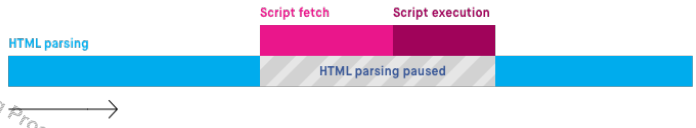
JavaScript execution: Runs your scripts, event handlers, timers

The Critical Rendering Path



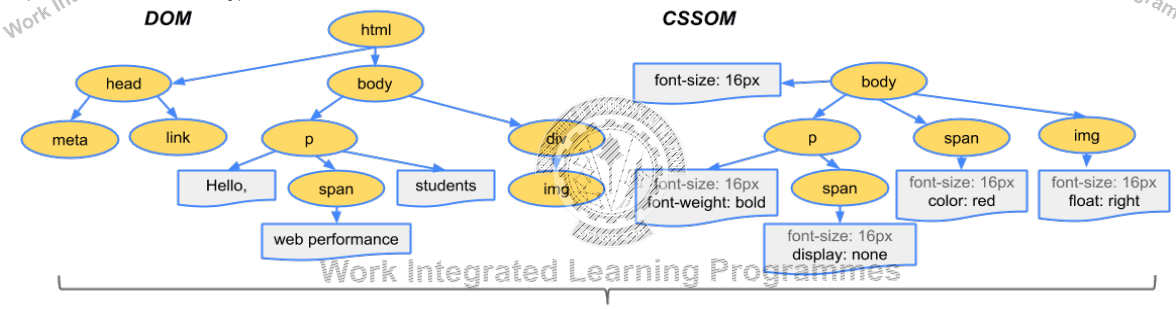
Critical Rendering Path

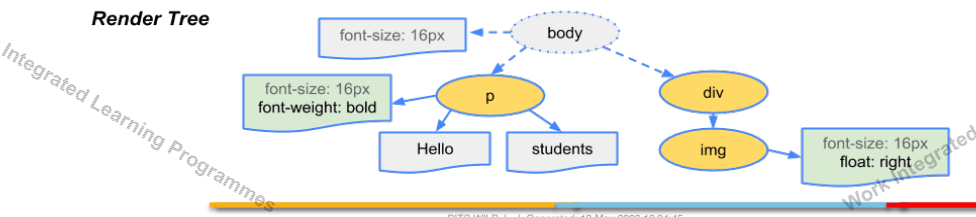




CRP

https://web.dev/articles/critical-rendering-path/render-tree-construction





Placement of <script> tag

<script> tag:

When placed in Head:

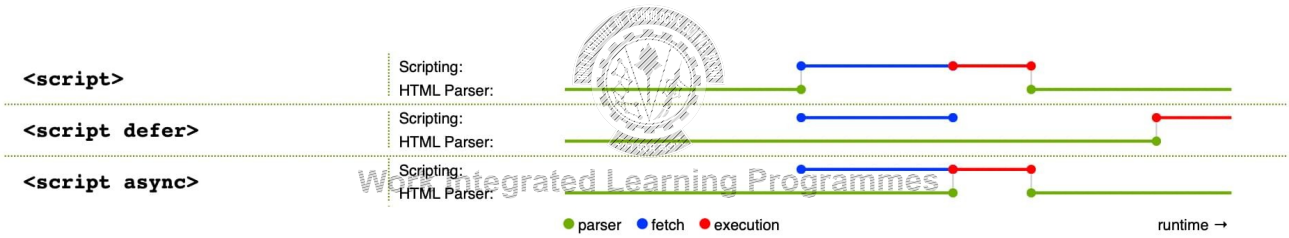
when the parser finds this line, it goes to fetch the script and executes it

Blocks the DOM construction

Introduces a lot of delay in loading the HTML

A very common solution to this issue is to put the script tag at the bottom of the page, just before the closing </body> tag.

Defer and Aysnc



This image is from the HTML spec,

Async

When to NOT use the async attribute for loading scripts

- Loading scripts that want to access the DOM with async should therefore be used with caution.
- Load several scripts with async, but they are actually interdependent — so it is important in which order they should be executed, because e.g. one of the two scripts is a library that the second script wants to access.

When to use the async attribute for loading scripts

- When we load external JavaScripts from another server, analysis tool like google analytics
- Applies to all other scripts that cannot directly access the DOM

Defer

When to use the defer attribute for loading scripts

this attribute is ideal for all scripts that are guaranteed to access the DOM

if we have several scripts that access each other, they must be executed in the order in which we want them to be

Reflow and Repaint

- The Repaint occurs when changes are made to the appearance of the elements that change the visibility, but doesn't affect the layout
  Eg: Visibility, background color, outline
- Reflow means re-calculating the positions and geometries of elements in the document. The Reflow happens when changes are made to the elements, that affect the layout of the partial or whole page.
- Resizing the window
- Changing the font
- Adding or removing a stylesheet
- Content changes, such as a user typing text in
- an input box

Example

```
var bodyStyle = document.body.style;
bodyStyle.padding = "20px";           // reflow, repaint
bodyStyle.border = "10px solid red";      // reflow, repaint

bodyStyle.color = "blue";                  // repaint only, no dimensions changed
bstyle.backgroundColor = "#cc0000"; // repaint
bodyStyle.fontSize = "2em";          // reflow, repaint
// new DOM element - reflow, repaint
document.body.appendChild(document.createTextNode('Hello!'));
```

Minimize Repaint and Reflow

Don't change individual styles, one by one.

Don't ask for computed styles excessively. If you need to work with a computed value, take it once, cache to a local var and work with the local copy

Batch DOM changes together

Performance Metrics

First Paint: The time to start of first paint operation. Note that this change may not be visible; it can be a simple background color update or something even less noticeable.

First Contentful Paint (FCP): The time until first significant rendering (e.g., of text, foreground or background image, canvas or SVG, etc.). Note that this content is not necessarily useful or meaningful.

First Meaningful Paint (FMP): The time at which useful content is rendered to the screen.

Largest Contentful Paint (LCP):The render time of the largest content element visible in the viewport.

Speed index: Measures the average time for pixels on the visible screen to be painted.

Time to interactive: Time until the UI is available for user interaction (i.e., the last long task of the load process finishes).

Optimizing the CRP

The process of website performance optimization involves changes to the website that reduce:

1) The amount of data that has to be transferred
2) The number of resources the browser has to download (especially the blocking ones)
3) The length of CRP

Node JS Ecosystem

Node.js

Node.js is an open-source and cross-platform JavaScript runtime environment.

Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser.

A Node.js app runs in a single process, without creating a new thread for every request.

Why Node.Js is single threaded

Node.js runs your JavaScript code in a single thread, which

It doesn't block the main thread for slow tasks.

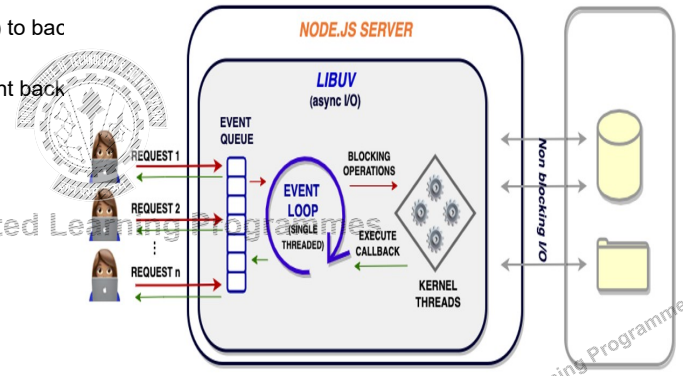
Instead, it uses non-blocking I/O and delegates slow work to background threads.

This makes Node.js lightweight, efficient, and scalable — perfect for web servers, APIs, and real-time apps.

Why Node.Js is single threaded

The event loop is a special mechanism in Node.js that:

- Receives tasks (like I/O requests).
- Passes slow tasks (e.g., file reads, DB calls) to bac
- Continues running other code.
- When the slow task finishes, the result is sent back



Differences between Node.js and the Browser

Both the browser and Node.js use JavaScript as their programming language.

Feature	Node.js	Browser
Purpose	Server-side JavaScript	Client-side JavaScript
Global Object	global	window
Access to DOM	No	Yes
File System Access	Yes (using fs module)	No (for security reasons)
Networking (TCP/UDP/HTTP)	Full access (sockets, HTTP servers, etc.)	Limited to HTTP/HTTPS via fetch, XHR
Modules	Uses CommonJS (require) or ES Modules	Uses ES Modules (import)
Execution Environment	Runs on a server, terminal, or command line	Runs in a web browser (Chrome, Firefox, etc.)
Security	Has full system access, so requires caution	Sandboxed (limited access to protect user data)
APIs Available	Node APIs (fs, http, path, etc.)	Web APIs (DOM, fetch, localStorage, etc.)
Use Cases	Backend apps, APIs, scripts, tools	UI, interactivity, animations, form handling
Event Loop	Handled by Node.js with libuv	Handled by browser (e.g., V8 in Chrome)
Threading	Single-threaded with background thread pool	Single-threaded with Web Workers for multithreading

Node JS Ecosystem

Node.js Core
Node.js was built using the V8 JavaScript Engine
Compiles JavaScript to machine code

Node.js Runtime
event-driven, non-blocking I/O model

NPM (Node Package Manager)
Command-line tool
Manages Dependencies

Express.js
Express.js is a popular, minimalistic web application framework for Node.js.

NPM

npm is the standard package manager for Node.js.
npm installs, updates and manages downloads of dependencies of your project.
Dependencies are pre-built pieces of code, such as libraries and packages, that your Node.js application needs to work.
If a project has a package.json file, by running npm install, it will install everything the project needs, in the node_modules folder, creating it if it's not existing already.

Modules

CommonJS (CJS) – The Traditional Way (Used in Node.js)

- Used by default in Node.js.
- Uses require() to import.
- Uses module.exports to export.

ECMAScript Modules (ESM) – The Modern Standard (Used in Browsers and Node.js)

- Official JavaScript module syntax (supported by both browser and Node).
- Uses import / export.
- You must use .mjs extension or set "type": "module" in package.json.

Webpack

Webpack is a powerful **module bundler**

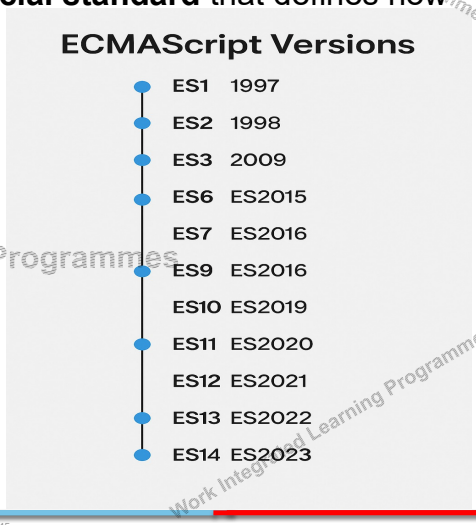
Webpack is a tool that takes your **source code** (JavaScript, CSS, images, etc.) and **bundles** it into a single file (or multiple files) that can be easily deployed to a web server.

Its main purpose is to **optimize** and **organize** your code, making it

ECMAScript

ECMAScript (often shortened to ES) is the **official standard** that defines how the **JavaScript language** should work.

Modern JavaScript = ES6 and beyond.



Transpiling

Transpiling refers to the process of converting ECMAScript 6 (ES6) code or the latest code into an older version of JavaScript that is more widely supported by browsers and environments.

Babel is the most popular transpiler for ES6.

References

https://www.jsv9000.app/
https://eloquentjavascript.net/11_async.html
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Event_loop

Thank You!

Full Stack Application Development- SE ZG503

BITS Pilani
Pilani Campus

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 11 Frontend

NPM

A **package manager** automates installing, updating, and managing third-party code (packages).

For the Node.js ecosystem, the standard is **npm (Node Package Manager)**.

npm is a command-line tool that fetches packages from the central npm registry.

It comes bundled automatically when you install Node.js.

Initializing the project

Before installing packages, you must create a package.json file.

This file is the heart of your project—it tracks metadata and dependencies.

Run this command in your project's root folder:

- npm init -y

The -y flag accepts all the default prompts, creating the file instantly.

To add a dependency, use the npm install command.

This does two things:

- Downloads the package into a new node_modules folder.
- Updates package.json and creates package-lock.json to record the exact version.

Managing Dependencies

Not all packages are needed for your final, live application.

Regular Dependencies: Required for the app to work in production.

Development Dependencies: Tools used only during development (e.g., Vite, ESLint).

- npm install vite --save-dev (or -D)

This separation keeps your final production code lean.

npm uses **Semantic Versioning (MAJOR.MINOR.PATCH)**.

Example: 5.2.1

- **MAJOR (5):** Breaking changes.
- **MINOR (2):** New features, backward-compatible.
- **PATCH (1):** Bug fixes, backward-compatible.

Your package.json will often have a ^ (e.g., "vite": "^5.2.1"), which allows safe patch and minor updates.

Managing Dependencies

package.json: Your project's manifest. Tracks which packages you need and at what version range. **Commit this to Git.**

package-lock.json: An exact, reproducible "snapshot" of all your dependencies. Guarantees every developer has the identical setup. **Commit this to Git.**

node_modules/: The actual downloaded code for all your packages. This folder can be huge. **NEVER commit this to Git.** (Use a .gitignore file to exclude it.)

Preparing apps for production

Preparing a React app for production involves a series of optimization, configuration, and deployment steps.

First, we need to create a production build using Build Tools
When you run: `npm run build`.

For Example, Vite executes the production build process.

1. **Reads your vite.config.js**
2. **Uses Rollup (a bundler) under the hood**
3. **Processes and optimizes your code and assets**
4. **Generates output in the dist/ folder**

Build Process

Vite uses Rollup for its production builds

It supports tree-shaking (removes unused code)

Outputs optimized, modern, ES module-based bundles

Allows fine-grained chunking (code splitting)

Build Process

1. Reads your app's main.jsx as entry
2. Scans all dependencies using Rollup
3. Transpiles JSX and modern JS using esbuild
4. Splits code into multiple chunks
5. Removes unused code (tree-shaking)
6. Bundles JS, extracts CSS
7. Minifies JS and CSS
8. Rewrites index.html with hashed asset URLs
9. Outputs everything into the dist/ folder
10. Optional: Analyze the Build
11. Test the Production Build Locally

```
npm install -g serve
serve -s dist
```


Files in the dist/ Folder

The entry point for your app: index.html

Vite rewrites <script type="module" src="/src/main.jsx"> to the hashed file

Contains all compiled, optimized, hashed files: /assets folder

File names like index-abc123.js have content hashes to:

- Enable long-term caching
- Ensure changes invalidate the cache

innovate achieve lead

BITS Pilani
Pilani Campus

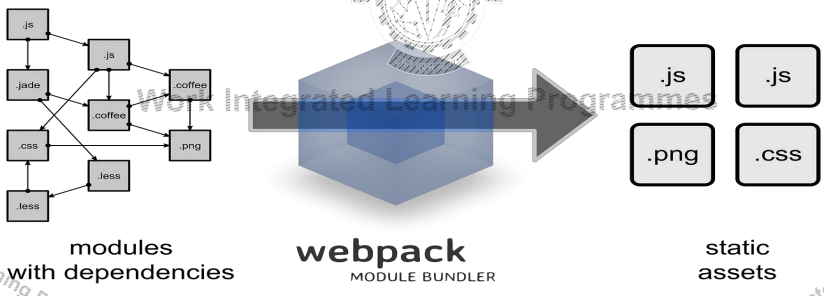
Lecture No: 11.1 Webpack

Webpack

innovate achieve lead

Webpack is a robust and highly scalable open-source Javascript module bundler.

Webpack is not limited to bundling javascript files and can bundle other file types such as CSS, SASS, images, typescript, etc.



Webpack- Dependency Graph

To perform module bundling, Webpack conducts a process called dependency resolution

Webpack looks through all the dependencies for the given modules and generates a **dependency graph**.

A dependency graph represents how the modules are interdependent.

Webpack uses this dependency graph to resolve dependency issues for the modules that depend on one another and generate one or more bundles.

Entry Point

Webpack configuration file: A Webpack configuration file is a file that contains all the configurations for Webpack.

The entry point contains the file path where Webpack begins its module bundling, and the entry point will act as the root to the dependency graph that Webpack generates. There can be multiple entry points, and a single entry point can also have multiple file paths passed as an array.

Output: Output is the file location where Webpack stores the final bundled file.

Webpack execution

When you run webpack, it:

- Loads the webpack.config.js (or default config).
- Webpack begins processing from the entry point.
- Every time it sees an import or require, it treats it as a dependency and recursively parses those files.
- As it parses, it builds a dependency graph
- Every file it encounters is a module.
- Webpack normalizes the module using its rules/loaders.
- Loaders transform files from their original format to JavaScript that the bundler can include in the final bundle.

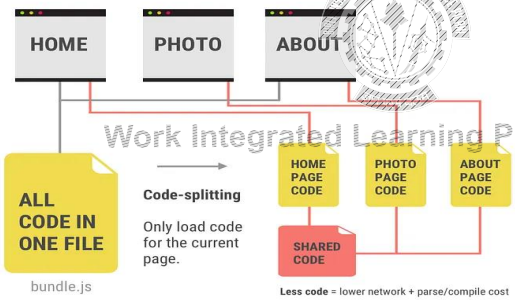
Webpack execution

- Webpack groups modules into chunks (e.g., one chunk per entry point or dynamic import).
- Each chunk contains all the modules needed for that part of the app.
- All chunks are stitched into a bundle (or multiple bundles).
- Writes the generated bundles and assets to the specified output folder.
- Optionally minifies or optimizes them (especially in production mode).
- Emits source maps if enabled.

Code splitting

Code Splitting is an optimization technique.

Code splitting is the process of splitting your project source code into multiple modules to be loaded in parallel or whenever needed.



Code Splitting

Webpack identifies import() as a code split point.

Webpack automatically extracts shared modules between chunks to avoid duplication.

Tree shaking

In production mode: Webpack includes all exports in the output bundle but flags unused ones as **"unused export"**.

Tree shaking is the process of **removing unused code** (a.k.a. “dead code”) from your final JavaScript bundle.

Think of it like pruning branches from a tree: if a function or variable is **imported** but never used, Webpack (along with a minifier like Terser) will shake it out.

Naming Convention for output files

Chunk name, usually based on your Webpack entry point:

main.3f1c2.js

content hash is a short hash string (e.g., MD5/SHA-256-based) generated from the file’s content.

If main.js changes, the hash changes → new filename.

This helps in caching

sourcemap

In production, your JavaScript is:

- **Minified**: Remove whitespaces/comments, shorten longnames
- **Bundled**: Many files combined into one
- **Transpiled**: From newer syntax (ES6+, TypeScript) to older JS

Debugging in the browser is difficult

Source map: A file that maps minified code back to original source

Source map : main.3a9c1.chunk.js.map

Hot Module Replacement (HMR)

Hot Module Replacement (HMR) is a development tool that exchanges, adds, or removes modules while an application is running without requiring a full page reload

The application asks the HMR runtime to check for updates.

The runtime asynchronously downloads the updates and notifies the application.

The application then asks the runtime to apply the updates.

The runtime synchronously applies the updates.

Deployment Options

Platform	Type	Benefits
Vercel	Serverless / CDN	Automatic CI/CD, super-fast edge delivery, SPA support out of the box
Netlify	Serverless / CDN	CI/CD, routing configuration, serverless functions, easy deploy
Firebase Hosting	Static hosting	Simple setup, integrates with Firestore/Functions
GitHub Pages	Static hosting	Free, easy for public projects, simple to configure
Render	Static hosting	Simple like Netlify, supports PR previews
Custom server	Apache, Nginx	Full control, manual caching/compression
Cloud Storage	AWS S3 + CloudFront, Azure, GCP	Global CDN, scalable, flexible configuration

WebAPIs

Client-side JavaScript API

Client-side JavaScript has many APIs available.

They are not part of the JavaScript language itself.

But they are built on top of the core JavaScript language.

Client-side JavaScript API s fall into two categories

- Browser APIs
- Third Party APIs

Browser API

A Browser API can extend the functionality of a web browser.

All browsers have a set of built-in Web APIs to support complex operations, and to help accessing data.

For example, the Web Audio API, Geolocation API

Third-party APIs

Third-party APIs are not built into the browser by default.

Third-party APIs are constructs built into third-party platforms (e.g. Google Maps API, Facebook APIs)

They allow you to use some of those platform's functionality in your own web pages

Common APIs

APIs for manipulating documents
- Example: DOM API

APIs that fetch data from the server
- Example: Fetch API

APIs for drawing and manipulating graphics
- Example Canvas API

Audio and Video APIs
- Example: Web Audio API

Device APIs
- Example: Geolocation API

Client-side storage APIs
- Example: Web Storage API

Client-side storage

There are numerous methods for storing data locally in the users' browser

- Cookies
- Web Storage (Local and Session Storage)
- IndexedDB
- Centralized data store ; State management libraries can be used.

Cookies

A cookie is a small piece of data that a server sends to the user's web browser.

The browser may store it and send it back with the next request to the same server.

It remembers stateful information for the stateless HTTP protocol.

They're the earliest form of client-side storage commonly used on the web.

Web Storage API

The Web Storage API provides mechanisms by which browsers can store key/value pairs

The two mechanisms within Web Storage are as follows:

- sessionStorage maintains a separate storage area for each given origin that's available for the duration of the page session
- localStorage does the same thing, but persists even when the browser is closed and reopened.

The two types of storage areas are accessed through global objects named "window.localStorage" and "window.sessionStorage".

Web Storage API

- Data is stored as key/value pairs, and all data is stored in string form.
- Data is added to storage using the setItem() method.
- setItem() takes a key and value as arguments.
- If the key does not already exist in storage, then the key/value pair is added.
- If the key is already present, then the value is updated.
- sessionStorage.setItem("foo", 3.14);
- localStorage.setItem("bar", true);

Reading Stored Data

- To read data from storage, the getItem() method is used.
- getItem() takes a lookup key as its sole argument. If the key exists in storage, then the corresponding value is returned.
- If the key does not exist, then null is returned.

- var number = sessionStorage.getItem("foo");
- var boolean = localStorage.getItem("bar");

Indexed DB

- IndexedDB is a transactional database embedded in the browser.
- The database is organized around the concept of collections of JSON objects similar to NoSQL databases
- IndexedDB is useful for applications that store a large amount of data (for example, a catalog of DVDs in a lending library) and applications that don't need persistent internet connectivity to work (for example, mail clients, to-do lists, and notepads).

Each IndexedDB database is unique to an origin

IndexedDB is built on a transactional database model.

Indexed DB

Database - This is the highest level of IndexedDB. It contains the object stores, which in turn contain the data you would like to persist.

Object store - An object store is an individual bucket to store data. Similar to tables in traditional relational databases.

Operation - An interaction with the database.

Transaction - A transaction is wrapper around an operation, or group of operations, that ensures database integrity.

Indexed DB Example

https://mdn.github.io/dom-examples/indexeddb-api/index.html

References

https://www.jsv9000.app/
https://eloquentjavascript.net/11_async.html
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Event_loop

Thank You!

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani

Pilani Campus

innovate achieve lead

BITS Pilani

Pilani Campus

Lecture No: 12 ReactJS

Module 6: Understanding Frontend Development

Designing and Structuring Webpages
Making pages Interactive with JavaScript
Using Browser based Web APIs- DOM, Web Storage
Client-side JavaScript Frameworks: Features and Advantage, MEAN, MERN
Implementing Single Page Applications using JavaScript Tech stack

- The Node Ecosystem
- Project Setup
- Creating and styling components
- Managing Component hierarchies and lifecycle
- Managing State
- Routing

Building, deploying and Hosting

React - Introduction

React is a JavaScript library for rendering user interfaces (UI).
Open-source
It is maintained by Meta (formerly Facebook) and a community of individual developers and companies.

Features

What made React stand out and popular?

- React adheres to the declarative programming paradigm
- React lets you break down the UI into reusable components
- Streamlines the process of building and composing components
- React uses a virtual DOM
- It is easy to learn and use

Component based Approach

React is based on a component-based architecture that allows developers to break down their user interface into small, reusable components.

This makes it easier to manage and maintain complex UI

Developers can focus on developing and testing individual components separately.

Each component consists of well-defined functionality that can be inserted into an application without requiring modification of other components

Declarative-What React Simplifies

Consider the task of adding a element

```
const target = document.getElementById("target");
const wrapper = document.createElement("div");
const headline = document.createElement("h1");

wrapper.id = "welcome";
headline.innerText = "Hello World";

wrapper.appendChild(headline);
target.appendChild(wrapper);
```


```
const { render } = ReactDOM;
const Welcome = () => (
  <div id="welcome">
    <h1>Hello World</h1>
  </div>
);
render(<Welcome />,
document.getElementById("target"));
```


Virtual DOM

- React uses a virtual DOM, which is a lightweight representation of the actual DOM.
- React updates the virtual DOM instead of the actual DOM
- React then compares the virtual DOM with the actual DOM and only updates the necessary parts of the UI

Virtual DOM

Every time the DOM changes, the browser has to do two intensive operations: repaint and reflow

Whenever a change is required , React marks that Component as **dirty.**

React updates the Virtual DOM relative to the components marked as dirty

React batches much of the changes and performs a unique update to the real DOM.

Repaint and Reflow the browser must perform to render the changes are executed just once

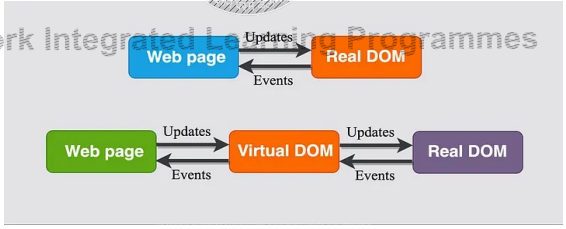
Reconciliation

Reconciliation - Determines the most efficient way to update the UI when data changes.

Compare virtual DOM trees

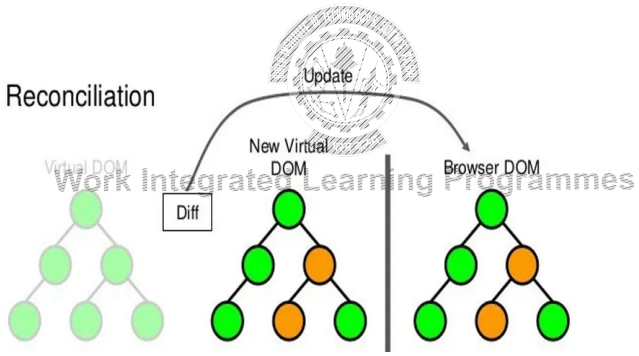
Identifies what changed and updates only that

Enables efficient UI rendering



Diffing Algorithm

- The diffing algorithm is an essential part of React's reconciliation process.
- It helps React decide exactly what needs to be updated on the screen.



Reconciliation Strategies

React offers different reconciliation strategies depending on the type of changes detected during diffing

- **Update strategy**
- **Reordering strategy**
- **Keyed strategy**

Important Javascript features

Data types
Using var, let and const
Conditionals and Loops
Using objects, arrays and functions
ES6 Arrow functions
In-built functions such as map(), forEach() and promises.
Destructuring Arrays and Objects
Modules

Alternatives to Create React App

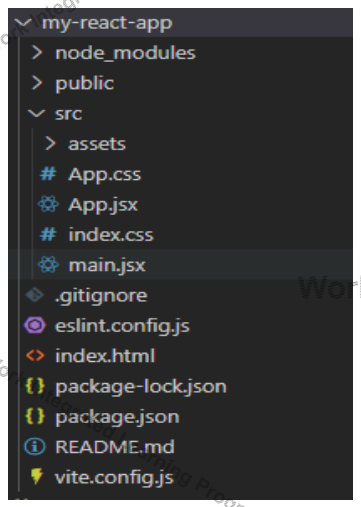
Vite: Known for its speed, Vite is optimized for modern JavaScript and React projects. It provides near-instant startup, fast hot-module replacement (HMR), and excellent configurability. Vite has become very popular for both small and large-scale React applications.

Next.js: Next.js is versatile and can handle single-page applications. It offers built-in routing, API routes, and server-side rendering (SSR) options, making it ideal for production-level React apps.

Remix is a framework that emphasizes full-stack React applications with a focus on web standards and optimization for faster performance. It's a good choice for projects where routing and server-side data fetching are important.

Parcel is a zero-config bundler that's easy to use and has great performance. It's a viable option for those who prefer minimal setup while still needing good development speed.

Folder Structure



React Elements -Smallest building elements

The elements that make up an HTML document become DOM elements when the browser loads HTML and renders the user interface.

The browser DOM is made up of DOM elements.

Similarly, the React DOM is made up of React elements.

A React element is a description of what the actual DOM element should look like.

Create a React element to represent an h1 using React.createElement:

- `React.createElement("h1", { id: "listitem-0" }, "Web Technologies");`

During rendering, React will convert this element to an actual DOM element:

- `<h1 id="listitem-0">Web Technologies</h1>`

ReactDOM

ReactDOM contains the tools necessary to render React elements in the browser.

ReactDOM contains the render method.

```
const Litem1 = React.createElement("h1", { id: "listitem-0"
  }, "Web Technologies");
ReactDOM.render(Litem1, document.getElementById("root"));
```


JSX

A JavaScript extension that allows us to define React elements using a tag-based syntax

JSX (JavaScript XML) is a syntax extension for JavaScript

This looks like HTML, but it's actually JavaScript.

In React, rendering logic and markup live together in the same place -> components.

Each React component is a JavaScript function

React components use a syntax extension called JSX to represent that markup.

JSX looks a lot like HTML, but it is a bit stricter and can display dynamic information.

JSX looks like HTML, but it's compiled into JavaScript by tools like Babel

```
const element = <h1>Hello</h1>;
```

is compiled into

```
const element = React.createElement("h1", null, "Hello");
```

Embedding Expressions

Return a single root element
JSX requires tags to be explicitly closed
JSX uses camelCase for attribute names, unlike HTML.
Embed JavaScript using {} inside JSX.

- let lessonName = “Web Development";
- <h3>{lessonName}</h3>
- Math: {2 + 2}
- Function calls: {getMessage()}
- Conditional expressions: {isLoggedIn ? "Logout" : "Login"}

Vanilla JS vs React – Syntax Differences

Concept	JavaScript (HTML/DOM)	React (JSX)
CSS class attribute	class	className
Event handling	onclick="handleClick()"	onClick={handleClick}
Conditional rendering	if...else or ? :	{condition && <Element />}
Commenting	<!-- comment -->	{/* comment */}
Fragments (no wrapping div)	Not applicable	<></> or <React.Fragment>
HTML attribute naming	for, tabindex, maxlength	htmlFor, tabIndex, maxLength

Mapping Arrays with JSX

To render multiple JSX elements in React, you can loop through an array with the .map() method and return a single element.

```
function courseListItems() {
  const courses = [“Web Technologies", “Java", “C++"];

  return courses.map((course) => <li key={course}>{course}</li>);
}
```

add a unique key to identify each list item uniquely

Index as a key is an anti-pattern

A key is the only thing React uses to identify DOM elements.

React compares the previous and new virtual DOM using these keys.

If keys are just indices, React assumes the identity of the element hasn’t changed

Using index as a key is acceptable only when:

- **The list is completely static**
- **The list never changes in size or order**
- **No element gets added, removed, or moved**

Components

Components let you split the UI into independent, reusable pieces.
Components allow us to reuse the same structure with different pieces of data

Types

Functional Components

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

Class Components

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Rendering a Component

- function Course(props) {
- return <h1>Course: {props.name} , {props.credits} </h1>;
- }

- const element = <Course name="Java" credits="4" />;
- ReactDOM.render(
- element,
- document.getElementById('root')
-);

Exporting a component

There are two primary ways to export values with JavaScript:

Default exports

```
function Welcome() {
  return <h1>Hello from Welcome Component!</h1>;
}
export default Welcome;
```

Named exports

```
export const welcome = "Welcome to React!";
export const goodbye = "Goodbye! good luck!";
```

A file can have only one default export, but it can have many named exports

Importing a component

How you export your component dictates how you must import it.

Syntax	Export statement	Import statement
Default	`export default function Button() {}`	`import Button from './Button.js';`
Named	`export function Button() {}`	`import { Button } from './Button.js';`

Props

Props is how Components get their properties.

Starting from the top component, every child component gets its props from the parent.

When initializing a component, pass the props in a way similar to HTML attributes:

Props are Read-Only

Whether you declare a component as a function or a class, it must never modify its own props.

Props

Passing JSX as Props

```
<Card content={<p>This is a card.</p>} />
```

Passing Functions as Props (Callbacks)

```
<Counter onIncrement={() => setCount(count + 1)} />
```

Children Prop

Every component automatically receives props. children for nested content.

```
function Layout({ children }) {
  return <div className="layout">{children}</div>;
}
```


Presentational vs container components

In React components are often divided into 2 big buckets:

- presentational components and
- container components.

Presentational components are mostly concerned with generating some markup to be outputted.

They don't manage any kind of state, except for state related the presentation.

Container components are mostly concerned with the "backend" operations.

They might handle the state of various sub-components.

They might wrap several presentational components.

They might interface with Redux.

Presentational components are concerned with the look,
container components are concerned with making things work.

State

Components need to “remember” things

React Class components have a built-in state object.

You can add state to a component with a useState Hook.

The state object is where you store property values that belongs to the component.

When the state object changes, the component re-renders.

State in Functional components

Earlier, Functional Components were stateless.

React Hooks made it possible to have state in Function Components.

Hooks are a new addition in React 16.8.

They let you use state without writing a class.

Hooks allow you to reuse stateful logic without changing your component hierarchy.

Points to be Noted

Do Not Modify State Directly - Use setState()

- For example, this will not re-render a component:
- this.state.comment = 'Hello';

State Updates May Be Asynchronous

React may batch multiple setState() calls into a single update for performance.

Because this.props and this.state may be updated asynchronously, you should not rely on their values for calculating the next state.

A component may choose to pass its state down as props to its child components

Benefits

They let you use state without writing a class.

It's hard to reuse stateful logic between components

Hooks allow you to reuse stateful logic without changing your component hierarchy

Complex components become hard to understand.

In many cases it's not possible to break these components into smaller ones because the stateful logic is all over the place.

useState is a Hook

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```


Hooks

React provides a few built-in Hooks like useState.

You can also create your own Hooks to reuse stateful behavior between different components.

Rules of Hooks

Hooks are JavaScript functions, but they impose two additional rules

Only call Hooks at the top level. Don't call Hooks inside loops, conditions, or nested functions.

Only call Hooks from React function components. Don't call Hooks from regular JavaScript functions.

Built in Hook

useState

const [count, setCount] = useState(0);

useEffect

By using this Hook, you tell React that your component needs to do something after render.

useEffect Hook is componentDidMount, componentDidUpdate, and componentWillUnmount combined.

Conditional Rendering

Conditional rendering as a term describes the ability to render different UI markup based on certain conditions.

Conditional rendering in React works the same way conditions work in JavaScript.

Use JavaScript operators like if or the conditional operator to create elements representing the current state, and let React update the UI to match them.

If/else

element variables

Ternary operator

Short Circuit Evaluation with &&

Event Handling in React

Event handling is how your application responds to user interactions like clicks, typing, etc..

React's event handling system is similar to DOM event handling, expect for a few improvements and differences

React Events Are Synthetic Events

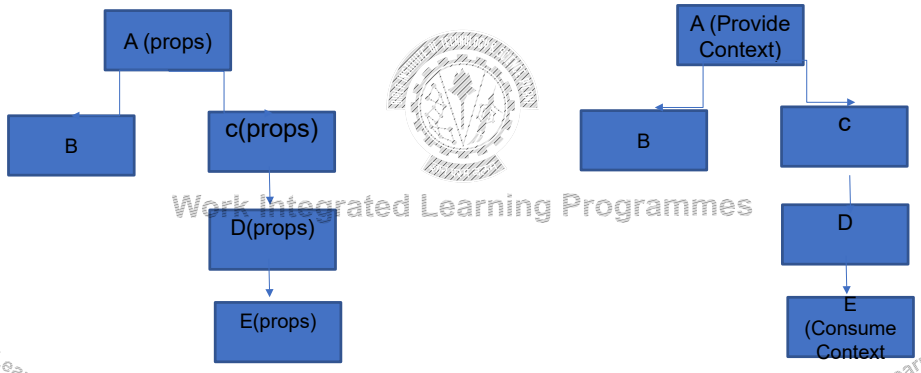
- React wraps native browser events inside a SyntheticEvent.
- Synthetic events are consistent across all browsers.
- They wrap the browser's native event to provide a consistent interface.

React uses camelCase instead of lowercase for event names.

- DOM: onclick → React: onClick
- DOM: onchange → React: onChange

Context

In a typical React application, data is passed top-down (parent to child) via props. When you had to pass props several components down your component tree. It results in prop drilling.



React Context

There are two use cases when to use it:

When your React component hierarchy grows vertically in size and you want to be able to pass props to child components without bothering components in between.

When you want to have advanced state management in React. Doing it via React Context allows you to create a shared and global state.

Responding to events

React lets you add *event handlers* to your JSX.

Built-in components like <button> only support built-in browser events like onClick.

References

https://react.dev/learn

Thank you

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani
Pilani Campus

innovate achieve lead

BITS Pilani
Pilani Campus

CS13: React Router

Routing

SPAs dynamically update sections of a page without full-page reloads
SPAs introduce challenges, such as managing browser history and routing.

React Router

React Router is the most popular routing library for React
Enables navigation among views
Handle routing *declaratively*.
<Route path="/home" component={Home} />

React Router

React Router includes three main packages:

- react-router: This is the core package for the router
- react-router-dom: It contains the DOM bindings for React Router. In other words, the router components for websites

To get started:

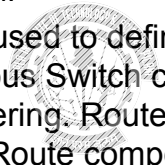
- npm install — save react-router-dom

Using React Router

The React Router API is based on three components:

Router: At the core of React Router v6 is the Router component, which provides routing capabilities to the application.

Routes: The Routes component is used to define route configurations within the application. It replaces the previous Switch component and allows for more flexible route matching and rendering. Routes can be nested and include route elements defined with the Route component.



The Route component defines a route within the application and specifies the component to render when that route matches the current URL. Routes in React Router v6 use an element prop instead of component, and they match paths inclusively by default.

<Router>

React Router v6 offers a single <Router> component that abstracts away the underlying routing strategy

The main job of a <Router> component is to create a history object to keep track of the location (URL).

When the location changes because of a navigation action, the child component is re-rendered.

The react-router-dom package offers three higher-level, ready-to-use router components, as

BrowserRouter: The BrowserRouter component handles routing by storing the routing context in the browser URL and implements backward/forward navigation with the inbuilt history stack

HashRouter: Unlike BrowserRouter, the HashRouter component doesn't send the current URL to the server by storing the routing context in the location hash (i.e.,i ndex.html#/profile)

MemoryRouter: This is an invisible router implementation that doesn't connect to an external location, such as the URL path or URL hash. The MemoryRouter stores the routing stack in memory but handles routing features like any other router

<Route>

It renders some UI if the current location matches the route's path.

<Route path="/about" component={About}/>

By default, routes are inclusive, more than one <Route> component can match the URL path and render at the same time.

<Link>

Create a navigational link to navigate to particular URL

```
<ul>
<li> <Link to="/">Home</Link> </li>
<li> <Link to="/about">About</Link> </li>
</ul>
```


<Outlet>

An <Outlet> should be used in parent route elements to render their child route elements.

This allows nested UI to show up when child routes are rendered.

If the parent route matched exactly, it will render a child index route or nothing if there is no index route.

Redux

Redux

innovate achieve lead

React Redux is the official Redux UI binding library for React.
Redux is a predictable state container for JavaScript apps.

Used for application State Management

It is inspired by Facebook's Flux Architecture

Work Integrated Learning Programmes

Need for Redux

In React, data flows from one parent component to the child component.

The Data flow is unidirectional

The child components cannot pass data to parents

The non-parents components cant communicate with each other

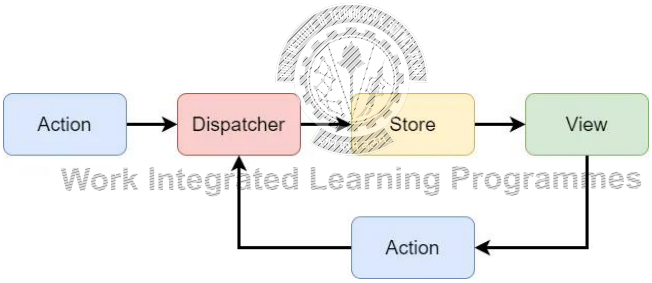
Redux offers a Store- to store all your application state in one place-single source of truth

Components can dispatch the state changes to the Store- instead of sending it to each components

Components need the data can subscribe to the State

Flux Architecture and Unidirectional Data Flow

Flux is an architectural pattern used in building Single Page Applications (SPAs) that guarantees unidirectional flow of data



Principles of Redux

A single object tree stores all of your application state

State is read only and changes are triggered by actions.

The state can only be manipulated by pure functions that are triggered by your actions

Actions

An action is simply an object that describes a change you want to make to your state.
Actions are payload on information that send data from your application to the store.
A standard pattern for actions is the following structure:

```
const action = {
  type: 'ACTION_TYPE',
  payload: 'Some data'
};
The payload is the data that will be used to transform our state.
const increment = {
  type: 'INCREMENT'
};
Some actions like incrementing a number do not require any additional data
```

Action Creator

Function which create actions

```
export const addNumber = (number) => ({
  type: ADD_NUMBER,
  payload: number
});

const action = addNumber(7);
```

Action creators are simply a function that allow us to abstract away the creation of actions, allowing us to easily dispatch an action without having to define all of its properties.

Reducers

A reducer is the pure function that we will use to transform our store state.

Actions describe the data that needs to be change, But How the state change happens is the responsibility of the Reducer

Reducers are triggered whenever an action is dispatched and receive both the current state of that reducer (which will be undefined to begin with) and the action that was dispatched.

```
export const count = (state = 0, action) => {
  switch (action.type) {
    case ADD_NUMBER:
      return state + action.payload;
    default:
      return state;
  }
};
```

Reducer

Takes in the prevstate and action, and determines what sort of update needs to be done based on action type

It returns the newstate value

It returns the prevstate, If no change occurred

Reducers are pure functions, they do not modify the passed original state, but make their own copy and updates them

Single Store , Multiple reducer

Root reducer slices up the state based on keys

Store

The Store is the object that hold application state.

Create a Store
export const store = createStore(count);

store.dispatch(action) is the method that is used to dispatch an action and subsequently trigger our reducers.

store.getState() simply returns the current state of the store.

Register listeners via the subscribe() method

Store Calculates the state changes and communicate to the Root reducer

Combining Reducers

For most applications we are going to want to store more than a single number

```
export const store = createStore(combineReducers({
  count,
  someOtherReducer
}));
```

Behind the scenes is creating another function that calls all our our reducers with the state that is relevant to them.

References

https://react.dev/learn

innovate achieve lead

BITS Pilani
Pilani Campus

Thank you

BITS Pilani
Pilani Campus

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 14 Testing

innovate achieve lead

What is Frontend Testing?

Ensuring that the user-facing part of a web app functions, performs, and looks as expected.

Frontend Testing covers

- **UI interactions,**
- **accessibility,**
- **Responsiveness and**
- **behavior under real conditions.**

Why Test Frontend?

Modern apps (React, PWAs, Flutter web) are complex and dynamic.

Need to deliver consistent UX across browsers, devices, and OS platforms.

The frontend is what users interact with

The bugs in frontend directly impact the business.

Key Drivers for Frontend Testing

Cross-Platform Complexity:
- Test on desktops, mobiles, browsers, OS

Continuous Quality:
- Automated testing across evolving platforms

Velocity vs. Quality:
- Maintain release speed without sacrificing stability

Compliance & Security:
- Privacy, accessibility, cybersecurity mandates

Performance:
- Load time under 3 seconds is business-critical

Testing as a Strategic Investment

Early detection of UI and performance issues
Supports agile workflows with automated feedback
Improves team confidence and customer satisfaction

Categories of Frontend Testing

Functional Testing
- Does it work as expected?

Non-Functional Testing
- How well does it work?

API Testing
- Are the frontend and backend interactions correct?

Visual Testing
- Does it look right?

Cross-Browser & Device Testing
- Is it consistent everywhere?

Exploratory Testing
- Can we uncover unexpected issues?

Coverage Analysis
- Are we testing enough?

Functional Testing

Ensures the application behaves as expected from the user's perspective

- **Unit Testing**
  - Test the smallest components/functions in isolation
- **Integration Testing**
  - Test the interaction between components/services
- **End-to-End (E2E)**
  - Testing – Simulates real user scenarios (e.g., login/logout)
- **Form and Link Testing**
  - Validates inputs, buttons, and navigation
- **Localization Testing**
  - Ensures language and locale support
- **UI and Layout Testing**
  - Validates visual structure and responsiveness
- **Compatibility Testing**
  - Tests across devices/resolutions
- **Usability Testing**
  - Verifies user-friendliness and intuitive flows

Non Functional Testing

Focuses on attributes like performance, security, accessibility, and reliability

Subtypes:

- **Security Testing**
  - Protects against vulnerabilities (SAST, DAST, OWASP)
- **Performance & Load Testing**
  - Measures response under load, latency
- **Accessibility Testing**
  - Ensures compliance (WCAG, ADA, 508)
- **Compliance Testing**
  - Validates industry and legal standards (e.g., HIPAA, PCI-DSS)

API Testing

Verifies data exchange between the frontend and backend via APIs
Ensures API endpoints return correct responses
Checks integration with microservices and third-party systems
Often bundled with E2E or integration tests
Tools: Cypress, Playwright, Postman

Visual Testing

Detects unintended visual regressions across UI versions
- Compares screenshots or DOM rendering before and after changes
- Helps identify CSS or layout drift

Cross-Browser & Device Testing

Ensures consistency across environments

- Different browsers (Chrome, Firefox, Edge, Safari)
- Different operating systems and screen resolutions
- Includes mobile vs. desktop validation

Exploratory Testing

Manual, unscripted testing to catch unexpected bugs.

- Performed by QA engineers or developers
- Complements automated testing
- Useful for complex user interactions or rare workflows

Coverage Analysis

Ensures thorough and effective test suites

- **Code Coverage (white-box):**
  Measures % of code executed (branch, statement, function, loop)
- **Test Coverage (black-box):**
  Measures how many of the requirements/features are tested

Testing React Applications

React apps involve UI components, interactions, and side effects.
Unit testing these requires:

- A test runner to run test files.
- A browser-like environment to render the DOM.
- A way to render React components in tests.
- Tools to simulate user interactions.
- Expressive assertions on the DOM structure and content.

Tools required for Testing

Tool	Purpose	Key Features
Vitest	Test runner	Fast, Vite-native, Jest-compatible syntax
@testing-library/react	Component testing	User-focused rendering and querying
@testing-library/jest-dom	DOM matchers	Adds readable, expressive assertions
@testing-library/user-event	User interaction simulation	Mimics real-world user events
jsdom	Browser-like environment	Allows DOM manipulation in Node.js

Vitest

Vitest (pronounced as "veetest") is a next-generation testing framework powered by Vite.

- Test runner, built specifically for Vite.
- Watch mode and filtering options.
- Vitest starts in watch mode by default in the development environment
- Vitest only reruns the related tests based on the changes.
- Runs tests in parallel for speed.

@testing-library/react

UI/component testing library for React.

- Encourages testing from the user's perspective.
- Abstracts DOM querying to reflect real-world usage.
- render() for rendering React components in tests.
- screen.getByText, getByRole, etc. for querying DOM.
- Works well with Vitest and Jest.
- Tests behavior, not implementation details.

Jest-dom and user-event

jest-dom

- Adds custom matchers like toBeInTheDocument() to expect() for DOM assertions.
- Makes test assertions more expressive and readable.
- Eliminates need for verbose manual DOM checks.

user-event

- Simulates realistic user interactions.
- Mimics how a user types, clicks, selects, etc.
- Works with React Testing Library for interaction tests.

jsdom

Emulates a browser-like environment in Node.
React components often rely on the DOM to render.
Allows DOM APIs (like document, window) in the test environment

What is Snapshot Testing?

Snapshot testing captures a component's rendered output and compares it over time.

Think of it as taking a photo of your component's UI structure and checking if it unexpectedly changes later.

Snapshots are stored in a __snapshots__ folder by default

When to Use Snapshot Testing

For pure UI components (no side effects).
To catch unexpected changes in output after a code change.
When you want quick regression checks.

Use **smaller, focused components** to keep snapshots meaningful.

Cross-Browser Test with Playwright

What can be tested?

- UI Rendering: Layout, colors, fonts
- Functional Features: Navigation, modals, forms, buttons
- Media: Videos, images
- Responsive design: Behavior on different screen sizes
- Performance: Load time, responsiveness

Tools for Cross-Browser Testing:

- Selenium (with browser drivers)
- Playwright
- BrowserStack

Playwright is an open-source end-to-end testing framework

Playwright enables reliable end-to-end testing for modern web apps

Playwright creates a browser context for each test.

Playwright supports all modern rendering engines including Chromium, WebKit, and Firefox

Integration testing

Integration testing verifies how multiple components or modules work together in a system.

Focus is on

- Component interaction
- State propagation
- Events and DOM updates
- API interaction (mocked)

Integration testing bridges the gap between isolated unit tests and full end-to-end testing.

Suppose you have:

- A <SearchBar /> that filters
- A <ProductList /> that displays items
- Both used in a parent <App /> component

An integration test would render <App /> and test:

- Typing in <SearchBar /> correctly updates <ProductList />
- State and props are flowing correctly
- User actions lead to expected UI outcomes

innovate achieve lead

BITS Pilani
Pilani Campus

Thank you

BITS Pilani
Pilani Campus

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 15 Security

Agenda

innovate achieve lead

- Basic Authentication
- API Keys
- JWT
- OAuth

Work Integrated Learning Programmes

Security

Authentication and authorization are two foundation elements of security:
Authentication is the process of verifying who a user is.
Authorization is the process of verifying what they have access to.

Basic Authentication

HTTP provides a general framework for access control and authentication.

In basic HTTP authentication, a request contains a header field in the form of
Authorization: Basic <credentials>

The credentials is the Base64 encoding of username and password joined by a single colon

Basic authentication is typically used in conjunction with HTTPS to provide confidentiality.

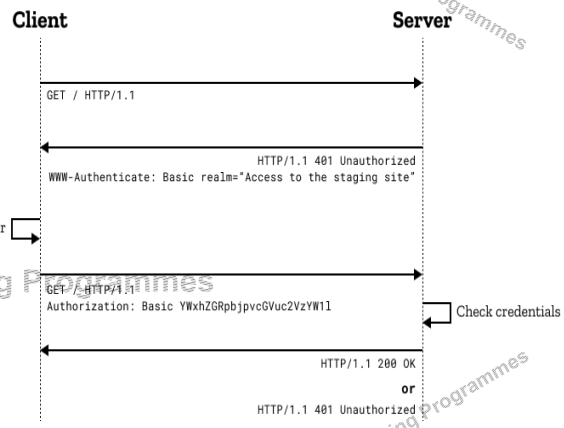


Image Ref:https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication

Basic Authentication

Here are some reasons why it is still used:

- Simplicity
- Compatibility
- Statelessness

Limitations

- Security
- Single Factor
- Risk of Credential Exposure

API Keys

An **API key** is a unique identifier used to authenticate and authorize the access to an **API**

Instead of a username and password, the client application is issued a unique API key, typically a long alphanumeric string.

The API key is sent in the HTTP request as a parameter in the query string or the request headers

(e.g., api_key=your_api_key or Authorization: API-Key your_api_key).

API keys are commonly used for machine-to-machine communication or applications interacting with the API.

Some APIs use API keys to enforce rate limits.

API Keys

Security: Treat API keys as sensitive information. Avoid hardcoding them in client-side code or exposing them publicly.

Rotation: Regularly rotate API keys to enhance security. You can invalidate a key and issue a new one if a key is compromised.

Scopes: Consider using different keys for different purposes (e.g., read-only vs. administrative access).

HTTPS: Always use HTTPS to transmit API keys securely.

Examples: **Google Maps API, Cloud Services**:

Token-based authentication system

Token-based authentication allows users to verify their identity, and in return receive a unique access token.

Token-based authentication is different from traditional password-based technique.

In stateless communication, each request that the user makes to the server contains all the necessary information for authentication, typically in the form of token.

The server validates the token and responds accordingly for each request.

Types Of Tokens

Opaque tokens

The opaque token is a random, unique string of characters the authorization server issues.

The opaque token does not pass any identifiable information

To validate the token and retrieve the information on the token and the user, the resource server calls the authorization server and requests the token introspection.

Structured token:

Its format is well-defined so the resource server can decode and verify the token without calling the authorization server.

JWT is a structured token

JSON Web Tokens (JWT)

JSON Web Tokens (JWTs) are a format of tokens used in web development and security.

JWT is a standard way to securely represent claims, such as user identity and roles, between two parties.

A JWT has a payload, which is a JSON object that contains information about the user, such as their identity and roles, and other metadata, such as an expiration date.

It's signed with a secret that's only known to the creator of the JWT.

The secret ensures a malicious third party can't forge or tamper with a JWT.

JSON Web Tokens (JWT)

JSON Web Tokens consist of three parts separated by dots (.), which are:

- Header
- Payload
- Signature

Therefore, a JWT typically looks like the following: xxxxx.yyyyy.zzzzz

JSON Web Tokens (JWT)

Header

The header typically consists of two parts: the type of the token, which is JWT, and the hashing algorithm such as

HMAC SHA256 or RSA.

Then, this JSON is Base64Url encoded to form the first part of the JWT.

Header:

```
{
"alg": "HS256",
  "typ": "JWT"
}
```

Payload

The second part of the token is the payload, which contains the claims.

Claims are statements about an entity (typically, the user) and additional metadata.

The payload is then **Base64Url** encoded to form the second part of the JWT.

Payload:

```
{
   "sub": "1234567890",
   "name": "John Doe",
    "admin": true
}
```

JSON Web Tokens (JWT)

Signature

To create the signature part you have to ta
payload, a secret, the algorithm specified i

```
HMACSHA256(
  base64UrlEncode(header) + '.' +
  base64UrlEncode(payload),
  secret)
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaXNTb2NpYWwiOnRydWV9.
4pcPyMD09olPSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```

The signature is used to verify that the ser

The output is three Base64 strings separat

Uses of JWT

Information Exchange
Single Sign on
Authentication and Authorization

OAuth

OAuth

OAuth (Open Authorization) is an open standard for access delegation.

It is commonly used as a way for internet users to grant websites or applications access to their information on other websites.

It specifies a process for resource owners to authorize third-party access to their server resources without providing credentials.

OAuth works over HTTPS

Without OAuth

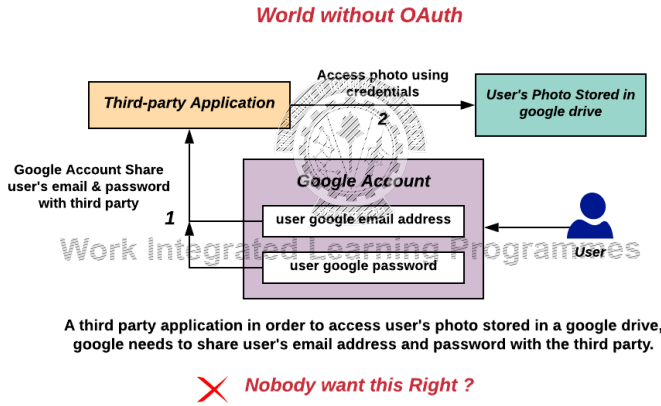


Image Ref: By Devansvd - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=109591037

OAuth

OAuth is a delegated authorization framework for REST/APIs.

It enables apps to obtain limited access (scopes) to a user's data without giving away a user's password.

It decouples authentication from authorization and supports multiple use cases addressing different device capabilities.

It supports server-to-server apps, browser-based apps, mobile/native apps, and consoles/TVs.

OAuth

Analogy: If you have a hotel key card, you can access your room.

How do you get a hotel key card?

You have to do an authentication process at the front desk to get it.

After authenticating and obtaining the key card, you can access the room and resources permitted across the hotel.

Similarly ,App requests authorization from User

OAuth Flow

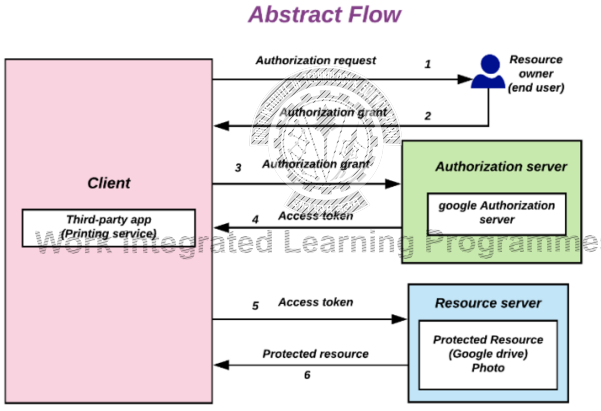


Image Reference: By Devansvd - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=109591026

OAuth Central Components

OAuth is built on the following central components:

- Scopes and Consent
- Actors
- Tokens
- Flows

Scopes

Scopes are what you see on the authoriz
permissions.

They're bundles of permissions asked fo

These are coded by the application deve

Actors

The actors in OAuth flows are as follows:

Resource Owner: owns the data in the re

Resource Server: The API which stores

Client: the application that wants to acces

Authorization Server: The main engine

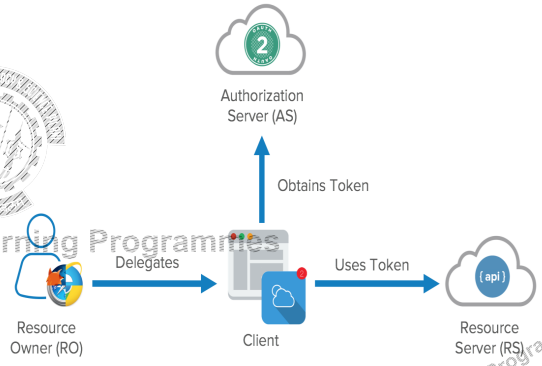


Image Reference:https://developer.okta.com/blog/2017/06/21/what-the-heck-is-oauth

Tokens

Access tokens are the token the client uses to access the Resource Server (API).

They're meant to be short-lived.

Refresh Tokens can be used to get new tokens

The OAuth spec doesn't define what a token is

Usually JWT is used

Tokens are retrieved from endpoints on the authorization server.

The two main endpoints are the authorize endpoint and the token endpoint.

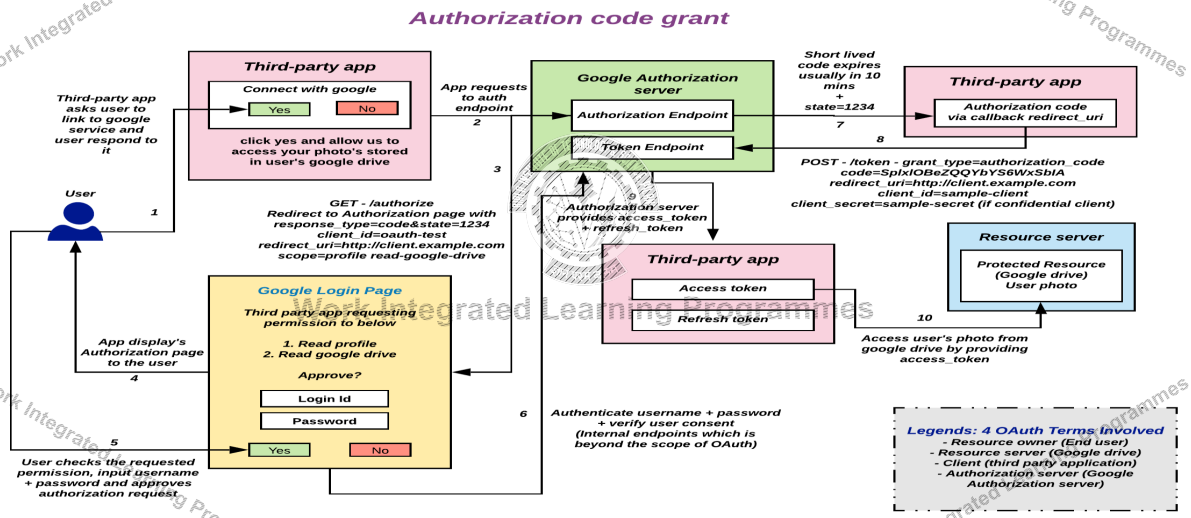
Flows

OAuth framework specifies several grant types for different use cases.

OAuth grant types

- Authorization Code
- Client Credentials
- Implicit Flow
- Resource Owner Password Flow

Authorization code



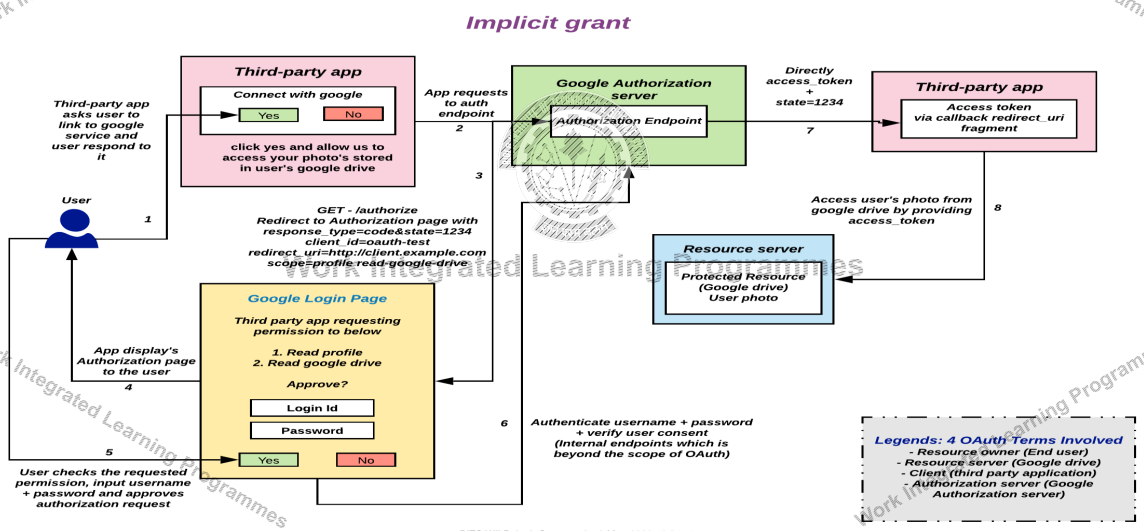
Authorization code flow

Use Case: Regular web apps executing on a server.

Example: A web application that needs to securely retrieve an access token.

The client (web app) exchanges an authorization code for an access token. It's considered safe because the token is passed directly to the server without going through the user's browser.

Implicit Flow



Implicit Flow

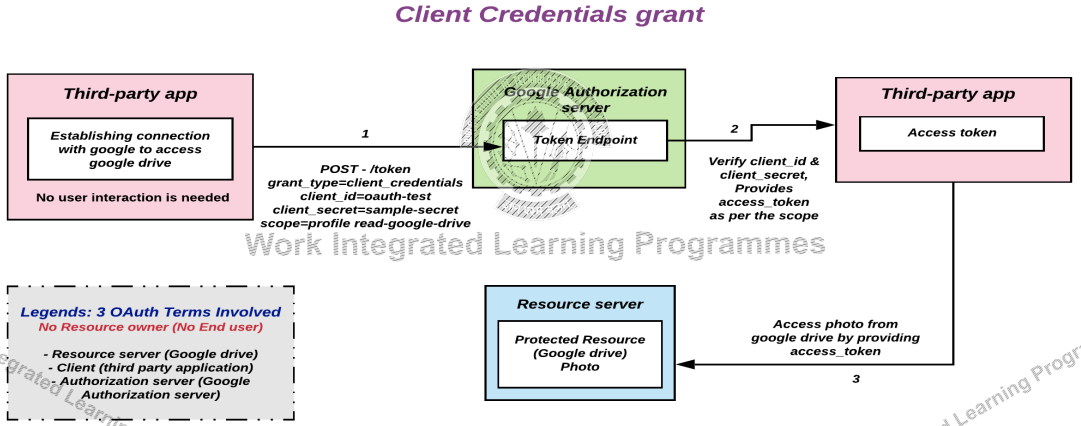
An access token is returned directly from the authorization request.

It typically does not support refresh tokens.

Since everything happens on the browser, it's the most vulnerable to security threats.

An SPA is a good example of this flow's use case.

Client Credentials flow



Client Credentials Flow

For server-to-server scenarios, a Client Credential Flow is used

In this scenario, the client application is a confidential client that's acting on its own.

It's a back channel only flow to obtain an access token using the client's credentials.

It supports shared secrets or assertions as client credentials

Use Case: Machine-to-machine authorization where no end-user interaction is needed.

Example: A cron job that imports data to a database using an API.

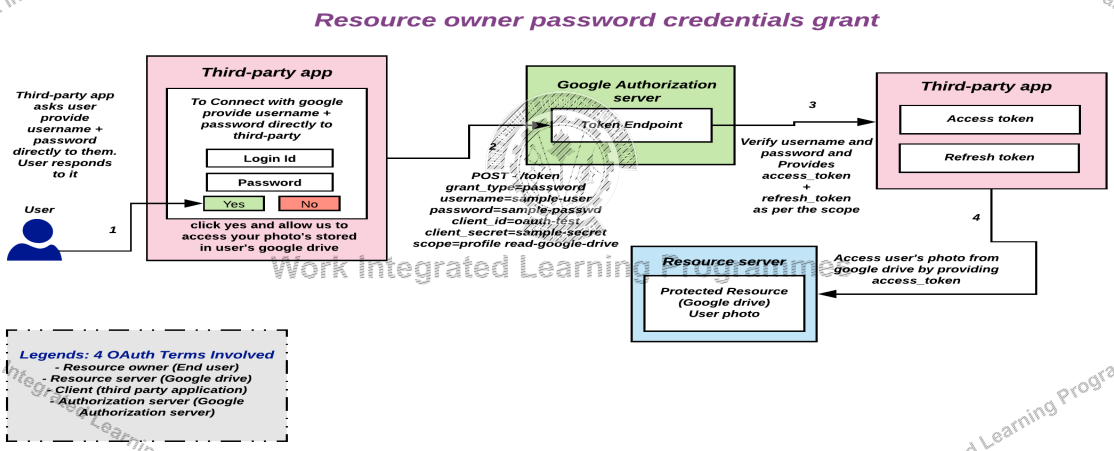
How It Works: The client (e.g., the cron job) directly obtains an access token from the authorization server using its client ID and client secret.

Resource Owner Password Flow

It's a legacy grant type for native username/password apps like desktop applications.

In this flow, you send the client application a username and password and it returns an access token from the Authorization Server.

Resource Owner Password Flow



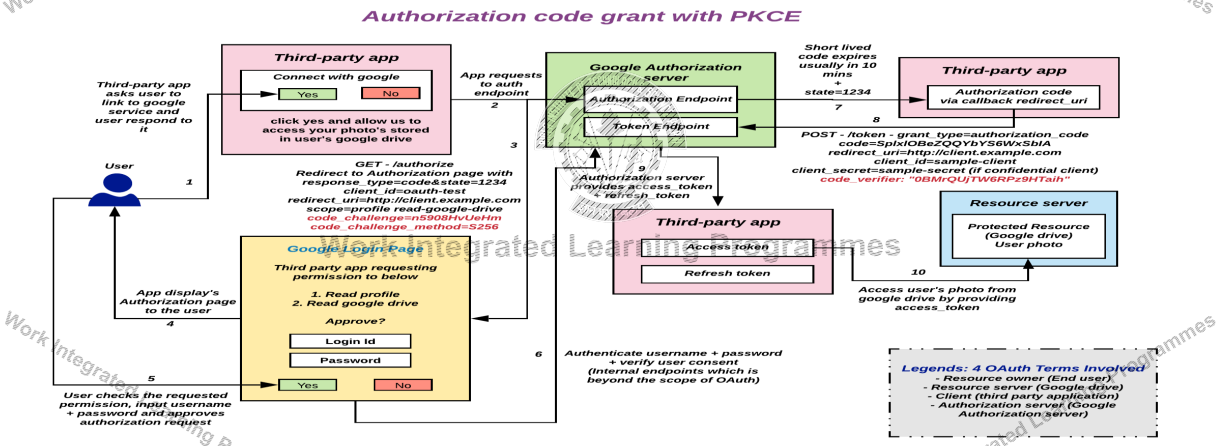
Authorization code with PKCE

This flow is an extension to Authorization grant flow.

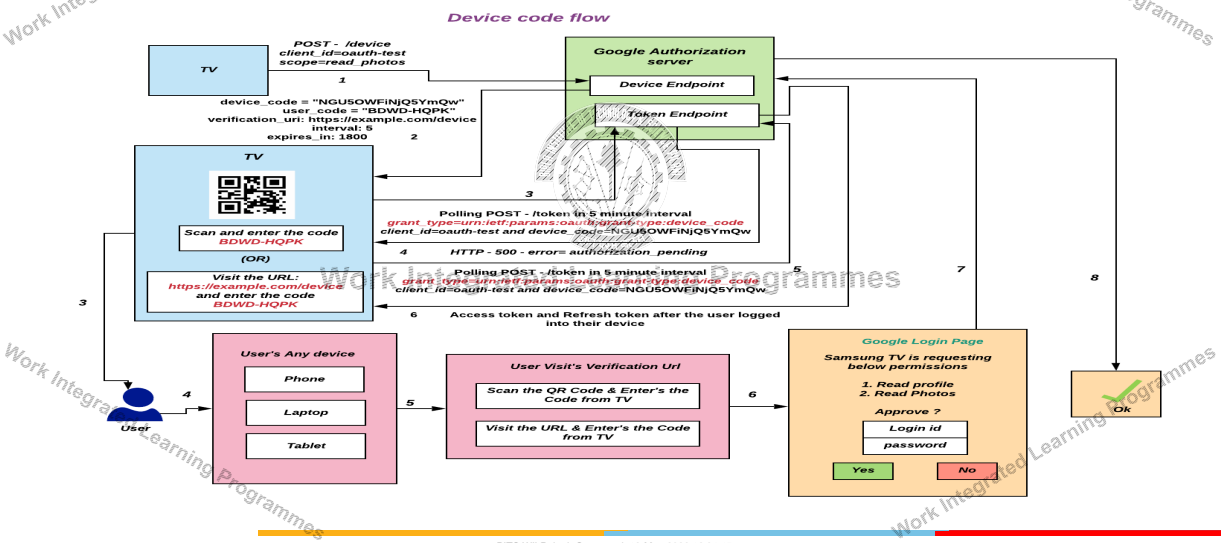
Authorization code grant is vulnerable to authorization code interception attacks when used with public clients

Proof Key for Code Exchange(PKCE)

Authorization code with PKCE



Device Code Flow



pseudo-authentication using OAuth

OAuth is an authorization protocol, rather than an authentication protocol.

OAuth does not provide user's information via an access token

Access tokens are meant to be opaque.

They're meant for the API, they're not designed to contain user information.

Custom Hacks were used to fill this gap

Using OAuth on its own as an authentication method may be referred to as pseudo-authentication

OpenID vs OAuth

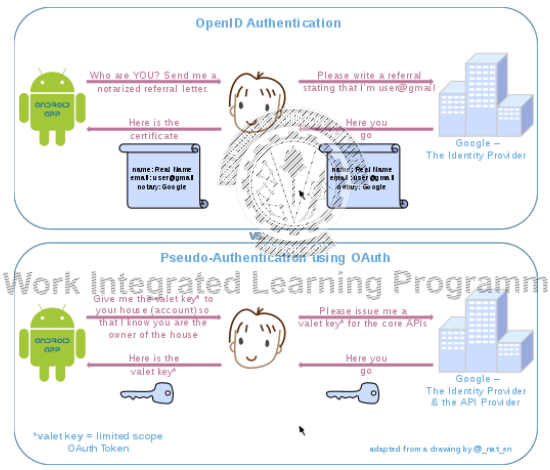


Image Reference: https://commons.wikimedia.org/wiki/File:OpenIDvs.Pseudo-AuthenticationusingOAuth.svg

OpenID Connect

OAuth is directly related to OpenID Connect (OIDC).

OIDC is an authentication layer built on top of OAuth 2.0.

OpenID Connect (OIDC) extends OAuth 2.0 with a new signed id_token for the client and a UserInfo endpoint to fetch user attributes

OpenID Connect is the standard for identity provision on the Internet.

OpenID Connect

What it adds:

- ID token
- User endpoint to get more userinfo
- Standardized

Its formula for success: simple JSON-based identity tokens (JWT), delivered via OAuth 2.0 flows that fit web, browser-based and native / mobile applications.

References

Demystifying OAuth 2.0 - A Tutorial & Primer :: Devansvd — Personal website

https://blog.postman.com/pkce-oauth-how-to/

https://auth0.com/docs/get-started/authentication-and-authorization-flow/which-oauth-2-0-flow-should-i-use

Thank you

Full Stack Application Development- SE ZG503

Akshaya Ganesan

CSIS, WILP

BITS Pilani
Pilani Campus

innovate achieve lead

BITS Pilani
Pilani Campus

Lecture No: 16 Accessibility

UX based on Technologies used

Consider the technologies that people use to experience our websites

- devices,
- browsers,
- operating systems, and
- assistive technologies

How to design for them

Responsive design is about creating fluid designs through HTML and CSS that adapt seamlessly to screen size, resolution, and aspect ratio, so that the layout remains optimally usable and attractive

Accessible design is about creating designs that are usable and enjoyable by people with disabilities, including physical, sensory, cognitive, and neurological problems

Universal design is about creating designs that are usable and enjoyable by everyone, regardless of age, status, culture, ethnicity, or ability

Accessibility

Consider the technologies that people use to experience our websites

- devices,
- browsers,
- operating systems, and
- assistive technologies

Accessible design is about creating designs that are usable and enjoyable for people with disabilities.

Designing for accessibility

The most important way to design for assistive technologies is to design with different users in mind, following usability guidelines

Designing for keyboard input
Designing for Screen Readers
Careful color and color contrast choices

Web accessibility is about removing barriers that prevent people with disabilities from accessing websites.

Principles of accessibility

– The **Web Content Accessibility Guidelines** (**WCAG 2.0**) by W3C defines

Four principles of accessibility.

Perceivable: Information and other interface elements must be visible to everyone.
Operable: Everyone must be able to navigate around any website.
Understandable: The information on websites must be easily understandable.
Robust: The content and its interpretation must be reliable, and compatible with different devices, browsers, and assistive technologies.

Guidelines

Using semantic HTML elements.

Use Text Alternatives for non text content like images

-

Ensure that form elements are accessible.

Arrange the HTML document with a logical and hierarchical structure.

Use appropriate heading elements

Ensure that all interactive elements are accessible via the keyboard.

Maintain sufficient contrast between text and background colors to enhance readability.

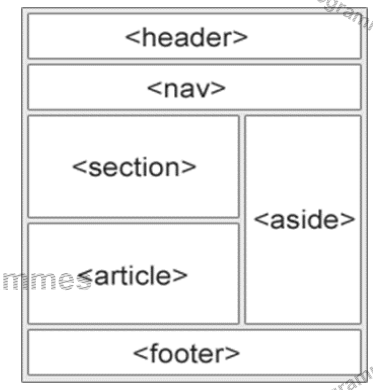
An army of elements

```
<div class="container">
    <div class="row ">
        <div class="col-2 bg-primary">first column</div>
        <div class="col-6 bg-secondary">second column</div>
        <div class="col-4 bg-primary">third column</div>
    </div>
</div>
<hr/>
<div class="container">
    <div class="row ">
        <div class="col-sm-2 bg-primary">first column</div>
        <div class="col-sm-6 bg-secondary">second column</div>
        <div class="col-sm-4 bg-primary">third column</div>
    </div>
    <hr/>
    <div class="row ">
        <div class="col-md-2 bg-primary">first column</div>
        <div class="col-md-6 bg-secondary">second column</div>
        <div class="col-md-4 bg-primary">third column</div>
    </div>
```

Semantic Elements

header: <header>.
navigation bar: <nav>.
main content: <main>,
Main content can be organized into subsections by
- <article>, and
- <section>

sidebar: <aside>; often placed inside <main>.
footer: <footer>.



ARIA

The Web Accessibility Initiative's Accessible Rich Internet Applications specification (WAI-ARIA)
ARIA provides additional information to assistive technologies about the roles, properties and states of elements.

Example

```
<button aria-expanded="false" onclick="toggleCollapsible()">
  Toggle Section
</button>
<div class="content" aria-hidden="true">
    <p>This is the content of the collapsible section.</p>
</div>
```


Self Reading

https://testsigma.com/tools/accessibility-testing-tools/

innovate achieve lead

BITS Pilani
Pilani Campus

Performance

Core Web Vitals

innovate achieve lead

Web Vitals is an initiative by Google
- Provide essential quality signals
- To deliver a great user experience on the web.

Core Web Vitals:
- Subset of Web Vitals
- Apply to all web pages

The Vitals focuses on three aspects of the user experience
- loading,
- interactivity, and
- visual stability

Metrics

Largest Contentful Paint (LCP):
Measures loading performance.
Time to render the largest visible content

Interaction to Next Paint (INP):
Measures interactivity.
Responsiveness of page to user interaction

Cumulative Layout Shift (CLS):
Measures visual stability.
Stability of layout

First Contentful Paint (FCP)

- The time it takes for the browser to render the first piece of DOM content (like text or images)
- After the user has navigated to the page

Time to First Byte (TTFB):

- The time between the request for a resource and when the first byte of a response begins to arrive.
- TTFB includes DNS, server response etc.,
- Reducing latency in connection setup time and on the backend can lower your TTFB.

Optimization

What does a High FCP or LCP mean?

- Slow initial load or rendering of large elements
- Common Issues include high TTFB, large JS/CSS files
- Render-blocking resources (e.g., fonts, CSS)

Optimization Techniques:

- Use lazy loading for images
- Use a CDN
- Optimize critical rendering path

What does a High INP mean?

- Poor responsiveness to user input (delayed reactions)
- Common Issues include Long JS tasks, inefficient event handlers

Optimization Techniques:

- Optimize React rendering using useMemo, useCallback
- Avoid heavy synchronous computations in event handlers

What does a High CLS mean?

- Layout shifts during load or interaction
- Issues like ads or images without fixed dimensions,
- Dynamic content

Optimization Techniques:

- Always set width and height on images/videos
- Reserve space for dynamic content

Example in React

React.lazy():

- This function allows for the dynamic import of components,
- The code is loaded only when they are needed

<Suspense>:

- This is a React component that allows you to "wait" for code to load
- Display a fallback UI while waiting
- It is used to wrap React.lazy() loaded components.

```
const Chart = React.lazy(() => import('./Chart'));

function App() {
  return (
    <div className="App">
      <Suspense fallback={<div>Loading...</div>}>
        <Chart />
      </Suspense>
    </div>

  )
}
```


Progressive Web App

Progressive Web Apps(PWA)

Web applications that use modern web capabilities to deliver app-like experiences.

Runs in a browser but behaves like a native app.

Responsive
Installable
Works offline
App-like interface
Secure (HTTPS)
Discoverable (indexed by search engines)
Re-engageable (push notifications)

PWA vs Traditional Web App vs Native App

Feature / Criteria	Progressive Web App (PWA)	Traditional Web App	Native App
Performance	High (cached assets, fast load)	Moderate (network-dependent)	Very High (compiled for the platform)
Installability	Yes (via browser prompt)	No	Yes (through app stores)
Access to Device Features	Limited (Camera, GPS, Push, etc.)	Very Limited (few APIs)	Full access (Bluetooth, NFC, etc.)
Distribution	Web (URL, shareable)	Web (URL)	App Stores (Apple, Google, etc.)
Offline Capability	Yes (via Service Worker)	No	Yes
Update Mechanism	Auto via browser	Auto via browser	Requires app store updates
Development Cost	Low (one codebase for all)	Low	High (platform-specific development)
User Engagement	High (push notifications, install)	Low	Very High (deep integration)

Core concepts

Service Worker:
- **A service worker is a JavaScript file that runs in the background**
- **Enables Offline support, background sync, push notifications**

Web App Manifest:
- **This is a JSON file that provides metadata about your app**
- **Enables Installability, app-like behavior on launch**

App Shell:
- **Minimal HTML, CSS, and JavaScript required**
- **Enables Instant load time**

HTTPS:
- **HTTPS is mandatory for PWAs.**

1. **When a user first loads the app, the manifest enables installability.**
2. **The app shell is cached by the service worker and displayed instantly.**
3. **Any network content (like articles, product data, etc.) is fetched and also cached for offline use.**
4. **All of this is only possible under the secure environment of HTTPS.**

Features

App Shell separates the core structure from dynamic content

Caching Strategy:
- Static (App Shell), Dynamic (API responses)
- Common strategies: Cache First, Network First, Stale-While-Revalidate

Offline Experience
- Serving cached assets when offline
- Graceful Degradation

Ensures secure content delivery and integrity

Service Worker

A script that runs in the background, independent of the web page.

Acts as a network proxy, handles caching and push notifications

Service workers have a lifecycle that dictates how they're installed, separately from PWA installation.

The folder the service worker is located determines its scope.

1. Registration:
   - register() is called in the main JavaScript:
2. Install:
   - Triggered when the browser first registers the service worker or detects a new version.
   - cache essential assets for offline use.
3. Activate:
   - Happens once the old service worker is deactivated.
   - clean up old caches.
   - The Service Worker will control all the pages that fall under its scope.
4. Fetch
   - Once activated, the service worker intercepts all fetch requests made by your web app.
   - Respond from the cache or make a network request.

Manifest File-Example

name: 'My Vite PWA': This is the full name of your app

short_name: 'VitePWA': A shorter version of the app name

start_url: '/': '/' launches the root page

display: 'standalone'
Defines the display mode when your app is launched as a PWA.

```
manifest: {
        name: 'My Vite React PWA',
        short_name: 'VitePWA',
        description: 'A Vite + React Progressive Web App',
        theme_color: '#ffffff',
        background_color: '#ffffff',
        display: 'standalone',
        start_url: '/',
        icons: [
          {
            src: 'icon-192-192.png',
            sizes: '192x192',
            type: 'image/png',
          },
          {
            src: 'icon-512-512.png',
            sizes: '512x512',
            type: 'image/png',
          },

        ],
      },
```

Thank You!